UNIVERSITY
OF OSLO

Master's thesis

Kubernetes in Edge Systems: Low Overhead Network Capacity Estimation

Kjetil Marstrander Omang

Informatics: Programming and system architecture

60 ECTS study points

Department of Informatics

Faculty of Mathematics and Natural Sciences

May 14, 2025

Kjetil Marstrander Omang

Kubernetes in Edge Systems: Low Overhead Network Capacity Estimation

Supervisors:

Carsten Griwodz

Frank Trethan Johnsen

Acknowledgements

This thesis is submitted in partial fulfillment of my Master's degree in Informatics at the University of Oslo.

I want to thank my supervisors, Frank Trethan Johnsen and Carsten Griwodz, for their patient guidance and support throughout the project.

Thank you to my friends and family for listening to me rant incoherently about problems they did not fully understand. Additional gratitude goes to my partner, Siv, and our cats, Lotta and Linus, for always being there.

And finally, thank you to my fellow students on the eighth floor of Ole Johan Dahls Hus for keeping me sane, and reminding me that one should take breaks, even when I insisted I was "almost finished".

AI Declaration

In this scientific work, generative artificial intelligence (AI) has been used. All data and personal information have been processed in accordance with UiO's regulations, and I, as the author of this document, take full responsibility for its content, claims, and references. The following provides an overview of the use of generative AI.

Specifically, large language models were used to:

1. Check spelling and grammar correction
2. Refine language and style
3. Auto-complete code fragments in the implementation of software associated with this thesis
4. Break down concepts to improve my own intuition and understanding

GitHub Copilot provided code suggestions, OpenAI's ChatGPT helped with textual improvements, and Grammarly helped with spelling checks.

Abstract

Edge networks pose significant challenges for network performance optimization, particularly in environments where rapid topology changes and unpredictable cross-traffic complicate bandwidth estimation. This thesis presents the design, implementation, and evaluation of a low-overhead available bandwidth estimation tool tailored for such dynamic edge networks. The proposed tool operates non-intrusively, capable of providing real-time network metrics to the Kubernetes control plane to enhance scheduling decisions in edge scenarios. Leveraging the Common Open Research Emulator (CORE) for network emulation and the traffic generator MGEN for synthetic load generation, we could simulate semi-realistic networks to evaluate the tool's performance. The experimental evaluation demonstrates that the tool can estimate available bandwidth without network overhead; however, it lacks precision for less optimal scenarios. Ultimately, this research contributes to the growing body of knowledge on the practical use of passive bandwidth estimation and lays a foundation for future developments aimed at robust, real-time network performance monitoring in Kubernetes clusters deployed on the edge.

Contents

1 Introduction . . . 1
1.1 Kubernetes in Edge Networks . . . 1
1.2 Motivation . . . 2
1.3 Scope . . . 3
1.4 Method . . . 5
1.4.1 Literature review as method . . . 5
1.4.2 Experimental prototyping . . . 6
1.5 Structure . . . 6
2 Background . . . 8
2.1 Kubernetes Architecture . . . 8
2.1.1 Pods, Services, and Deployments . . . 8
2.1.2 Master Node . . . 9
2.1.3 Nodes . . . 9
2.2 Edge Computing . . . 10
2.3 Kubernetes at the edge . . . 11
2.4 Understanding Bandwidth . . . 12
2.4.1 Capacity and Bottleneck Capacity . . . 12
2.4.2 Available Bandwidth (ABW) . . . 13
2.4.3 Effects of Cross-Traffic and Queueing . . . 14
2.4.4 Achievable bandwidth . . . 16
2.5 Bandwidth Estimation . . . 16
2.5.1 Active Probing . . . 17
2.5.2 Passive Bandwidth Estimation . . . 19
2.5.3 Factors Influencing Estimation Accuracy . . . 20

2.6 Bandwidth Estimation in Radio Networks . . . 24
2.6.1 Physical Layer Variability and Interference . . . 24
2.6.2 Link-Layer Shared Medium Access . . . 25
2.6.3 Network-Layer Dynamics and Topology . . . 26
2.6.4 Limitations of Traditional Estimation Techniques . . . 27
2.6.5 Implications and Rationale . . . 29
3 System Design . . . 30
3.1 Emulation . . . 30
3.1.1 NREC . . . 31
3.1.2 Emulation Tools . . . 32
3.1.3 Load Generators . . . 34
3.1.4 Integrating CORE and MGEN . . . 35
3.2 Bandwidth Estimation Tool . . . 36
3.2.1 Design and Architecture . . . 36
3.2.2 Measurement window . . . 37
3.3 Data Collection . . . 38
3.3.1 Evaluation and Performance Metrics . . . 39
4 Implementation . . . 41
4.1 Overview . . . 41
4.1.1 Libraries and Crates . . . 41
4.2 Emulation Testbed Setup with CORE . . . 42
4.2.1 CORE Installation and Configuration . . . 43
4.2.2 Traffic Generation Scenarios . . . 44
4.3 Packet Capturing and Parsing . . . 45
4.4 Packet Analysis Pipeline . . . 47
4.5 Concurrency Model with Tokio . . . 50
4.5.1 Channel message design . . . 52
4.5.2 gRPC Communication via Prost and Tonic . . . 53
4.6 Available Bandwidth Estimation Algorithm . . . 54
4.6.1 Filtering of the probe gap measurements . . . 55
4.6.2 Regression algorithm . . . 56
4.6.3 Limitations and considerations . . . 56

5 Results . . . 58
5.1 Experimental Setup . . . 58
5.2 General Observations . . . 60
5.2.1 Cross-traffic Timing . . . 60
5.2.2 Timestamping Anomaly . . . 62
5.3 Accuracy . . . 63
5.3.1 Systematic over-prediction . . . 64
5.3.2 Impact of Regression Basis and Data Volume . . . 66
5.3.3 Experiment 2: Underestimation . . . 68
5.3.4 Experiment 8: Narrowing the Capacity Gap . . . 69
5.4 Discussion . . . 70
5.5 Using robust regression . . . 71
5.6 Performance Evaluation . . . 73
6 Conclusion . . . 75
6.1 Research Questions . . . 75
6.2 Contributions . . . 77
6.2.1 Passive Bandwidth Estimation Framework . . . 77
6.2.2 Experimental Evaluation setup . . . 77
6.2.3 Dataset for Analysis . . . 77
6.2.4 Proof-of-Concept gRPC API . . . 77
6.3 Experiences . . . 78
6.4 Future work . . . 78
Appendices . . . 94
A Tool Configuration . . . 95
B Installation . . . 97
B.1 Database configuration . . . 99
B.2 CORE Installation . . . 100
C Additional Figures . . . 102

Chapter 1

Introduction

Kubernetes, often abbreviated as K8s, is an open-source container orchestration system originally developed by Google (The Kubernetes Authors, 2024a). It is designed for managing large-scale, cloud-based systems. Kubernetes excels in handling volatile software environments, managing failures, and making informed decisions regarding CPU, memory, and overall resource availability. In recent years, Kubernetes has evolved to become one of the most widely used frameworks for hosting cloud platforms and is utilized by most of the largest cloud providers in the world, such as Amazon Web Services (AWS), Microsoft Azure, Google Cloud, and IBM Cloud (Bernstein, 2014).

1.1 Kubernetes in Edge Networks

Kubernetes was initially developed to orchestrate containers within stable server clusters connected via high-speed, reliable networks (Burns et al., 2016). Such environments benefit from dedicated link capacities that operate independently of one another, ensuring consistent performance. Extending Kubernetes to manage edge nodes that utilize radio-based connections introduces significant challenges. Wireless and cellular networks operate over shared media, where the activity of one node can interfere with others due to limited spectrum availability and potential signal interference (P. Gupta & Kumar, 2000). This shared nature results in variable network capacity, as increased usage by one node can degrade the performance for neighboring nodes.

In high-bandwidth, robust networks with numerous users, probing is a common method for assessing network capacity. Probing involves transmitting data packets to evaluate metrics such as response time and packet loss (Dovrolis et al., 2001). However, in

radio-based networks, this approach poses some challenges. Each probing transmission consumes bandwidth that could otherwise be utilized by legitimate traffic, leading to potential congestion and overall performance degradation. Moreover, excessive probing can exacerbate network inefficiencies and even disrupt normal operations. Wireless connections fluctuate in quality and availability due to environmental factors and user mobility, complicating capacity measurements and increasing the risk of interference.

Consequently, to avoid operational disruptions, Kubernetes needs to adapt its resource management strategies when deployed in these volatile environments. This thesis aims to explore the feasibility of running a Kubernetes cluster at the edge in wireless networks. For this to work effectively, the Kubernetes scheduler would benefit from having real-time information about the network links, enabling it to make informed scheduling decisions. This information should be available through an Application Programming Interface (API) from all nodes in the cluster, providing metrics such as available bandwidth, congestion levels, and signal quality when needed.

1.2 Motivation

Wireless networks typically rely heavily on infrastructure to operate effectively. For instance, a typical 5G network includes base stations positioned every few hundred meters to ensure coverage and connectivity. In contrast, Mobile Ad Hoc Networks (MANETs) and Vehicular Ad Hoc Networks (VANETs) are decentralized networks that do not depend on pre-existing infrastructure (Akyildiz et al., 2002). Instead, they form dynamically as mobile devices connect directly to one another, creating flexible and self-organizing networks.

In tactical edge scenarios, such as rescue missions or military operations, network infrastructure might not be available or feasible to deploy. In these cases, MANETs are utilized to enable communication by setting up ad hoc infrastructure using tactical routers and relay devices. However, these networks are inherently volatile (Luo et al., 2008). The bandwidth, latency, and networking technologies can vary significantly within the network and across different situations, depending on factors such as node mobility and environmental conditions. This variability makes developing robust and generic solutions for MANETs particularly challenging (Johnsen et al., 2023).

Recently, Nicolai Vatne explored the possibility of deploying Kubernetes clusters in

disaster relief operations, where network stability is a major concern (Vatne, 2022). His work discusses the use of MQTT ("MQTT Version 5.0", 2019) as a messaging protocol to enhance communication reliability and the potential of K3s (K3s Project Authors, 2024), a lightweight Kubernetes distribution suitable for such scenarios. Vatne's findings suggest that single-board computers like the Raspberry Pi can effectively manage resource allocation and network traffic in edge environments, provided there is sufficient bandwidth. However, he also identified challenges related to running a Kubernetes cluster in unstable environments. As highlighted by Kayal (2020), the Kubernetes scheduler does not consider network state when making decisions, rendering it unreliable in volatile networks where high-capacity links with ample available bandwidth cannot be guaranteed.

Building upon the limitations presented by Vatne (2022), this research aims to enhance the concept of a Kubernetes cluster operating in edge networks, focusing on real-time bandwidth estimation. The primary objective of this thesis is to explore methods to accurately estimate available bandwidth in volatile edge networks to provide useful information to the Kubernetes control plane (Rensin, 2015). By supplying the scheduler with real-time network metrics, such as available bandwidth and latency, Kubernetes can make informed decisions, improving resource allocation and system stability in environments where network conditions are highly variable.

1.3 Scope

The scope of this thesis is to design, develop, and evaluate a nonintrusive bandwidth estimation (BWE) tool tailored for edge networks. The primary aim is to provide real-time network information to the Kubernetes control plane, thereby enhancing scheduling decisions in highly variable network conditions. To achieve this, we:

- Conducted a detailed literature review of existing bandwidth estimation techniques, evaluating their strengths, limitations, and potential gaps in the context of wireless networks.

- Designed and developed a non-intrusive BWE tool by adapting and integrating existing methodologies and tools. The implementation is a prototype made to validate feasibility and as a proof of concept for further research. We did not modify Kubernetes's internal scheduling logic, but we supplied external bandwidth

metrics for potential use in scheduling. Although applicable to broader contexts, the solution is made for isolated edge networks, where link quality and capacity fluctuate.

- Evaluated the tool's performance through emulation with synthetically generated traffic. The evaluation focused on estimation accuracy and adaptability to different link and traffic characteristics.

This research is confined to developing and assessing the BWE tool within the context of Kubernetes clusters operating on the edge. Although the initial goal was to write a generic tool for bandwidth estimation in wireless edge networks, as we explain in Section 2.6, this problem is highly technology- and topology-specific, making it difficult to solve in a high-level emulation environment. Consequently, we moved away from ABW estimation in wireless networks, shifting our focus towards emulated physical networks.

Additionally, we did not account for the security of the tools' operation, and we mainly focus on classic available bandwidth estimation, based on transport-layer traffic analysis, without the use of machine learning. Consequently, we explored the following overarching research question and three sub-questions:

How can we utilize the natural flow of data on the edge to indicate the state of network links in an unstable network?

1. What are the challenges of implementing bandwidth estimation in radio networks?
2. How can data collected via passive network interface listening be used to estimate available bandwidth?
3. How can we communicate the passively gathered data with Kubernetes without causing significant strain on the network?

By addressing these questions, we aim to develop a framework that can help supply additional context to the Kubernetes scheduler and provide an understanding of the bandwidth estimation in general, and the complexity introduced by the properties of wireless networks.

1.4 Method

To address the research questions, we use a combination of literature studies and experimental prototyping.

1.4.1 Literature review as method

A literature review (Snyder, 2019) is a research method that involves the systematic collection, examination, and synthesis of existing research and published works within a specific subject area. This method is used to gain a comprehensive understanding of what is already known, identify knowledge gaps, and establish a theoretical framework for future research.

Following Snyder (2019)'s taxonomy, our review combined systematic database searches, followed by more targeted research and critical synthesis to ensure coverage of both classic bandwidth estimation methods, passive approaches, and strategies targeted to radio networks.

We began by translating our overarching thesis question into specific review objectives.

1. Gain knowledge and intuition around network capacity estimation and related concepts.

2. Explore both active and passive methods for available bandwidth estimation.

3. With a deeper understanding of network monitoring, identify specific challenges related to capacity estimation in wireless networks.

With the review objectives identified, we performed wide literature searches to find research covering bandwidth estimation, focusing on passive listening-based methods. We targeted peer-reviewed studies and built a foundation of relevant research by following the citation networks of the initial searches.

From the gathered research, we identified challenges within available bandwidth estimation and the complexity introduced when applying the theoretical approaches from wired networks to radio networks.

1.4.2 Experimental prototyping

In line with Carr and Verner (1997)'s classification of prototyping approaches as exploratory, experimental, or evolutionary, we adopted the experimental prototyping paradigm for this work. In this model, the primary focus is on creating early, experimental versions of a system to test and evaluate concepts before full implementation.

1. **Prototype Design and Implementation**:

   We began by specifying minimal functional requirements, which were to develop a tool capable of low-overhead available bandwidth estimation, coupled with functionality to publish these findings such that they can be used by Kubernetes to make informed scheduling decisions. An initial prototype was built in the Rust programming language (Klabnik, 2018), using packet capture for passive network traffic listening and analysis.

2. **Iterative Refinement**:

   During the development process, we performed tests to discover bugs, make changes to the filtration mechanism, and adjust timing parameters. In the early stages, these tests were focused on making sure that the central parts of the tool worked as expected, while towards the end of the project, we mainly worked on refining the emulation and data collection infrastructure.

3. **Experimental Evaluation**:

   The final prototype was deployed in an emulation environment, enabling the collection of ground truth measurements and metrics for later analysis. The evaluation process was iterative, starting with a few baseline experiments, which were then modified based on gathered results.

1.5 Structure

The remainder of this thesis is organized as follows: Chapter 2 provides the necessary foundation for the research, including an overview of the Kubernetes architecture and its typical operation, edge computing, and Kubernetes in the context of edge computing. This chapter also defines key bandwidth-related concepts such as capacity, available

bandwidth, and the impact of cross traffic, and reviews existing bandwidth estimation approaches. Finally, in Section 2.6 we explain the difficulties with bandwidth estimation in wireless networks, and why we chose to focus our implementation on wired networks.

Chapter 3 then presents the design of our proposed bandwidth estimation system. We outline the tool's architecture, detailing how the passive monitoring, data processing, and communication interfaces are structured. In addition, we explain the approach to emulation and the tools used during experimentation.

In Chapter 4, we describe the prototype's implementation details and the emulation testbed's configuration. We discuss the technologies used and how CORE and supplementary tools were configured to create the different test scenarios. This chapter also covers some practical challenges encountered during development and how we addressed them.

Chapter 5 presents the experimental findings. We analyze the performance of the bandwidth estimation tool in various scenarios, evaluating and discussing its accuracy in estimating available bandwidth in different scenarios.

Chapter 6 summarizes the contributions of the thesis and reflects on the degree to which the initial objectives were met. We discuss the overall feasibility of passive bandwidth estimation in edge systems and outline the remaining challenges, as well as future work.

Chapter 2

Background

2.1 Kubernetes Architecture

While this thesis primarily focuses on leveraging data flow for bandwidth estimation, understanding the structure of Kubernetes provides context for the project's overarching goal.

Kubernetes is a platform designed to automate the deployment, scaling, and operation of containerized applications, providing a robust framework for managing distributed systems. A Kubernetes cluster is fundamentally composed of master nodes and individual nodes. Here, we introduce the key components that constitute a K8s cluster.

2.1.1 Pods, Services, and Deployments

Kubernetes conveys application intent through three abstractions: Pods, Deployments, and Services. A pod is the smallest schedulable unit, encapsulating one or more containers that share location, namespace, and storage volumes (The Kubernetes Authors, 2025b).

Higher-level orchestration is performed through the definitions of Deployments. A deployment specifies characteristics such as Pod templates, replica counts, and update strategy, while the control plane drives the cluster state to match this specification (The Kubernetes Authors, 2025a).

Network identity is decoupled from the transient lifecycle of individual Pods through the Service abstraction. When a Service is created, the API Server assigns a stable

virtual IP address and port, removing the need for clients to track pod-level addresses (The Kubernetes Authors, 2025c).

2.1.2 Master Node

The *Master node* is the central unit responsible for managing the entire cluster. It coordinates various activities within the cluster, such as scheduling tasks, maintaining the cluster state, and handling API requests. These functions are primarily managed through three core components: the API Server, etcd, and the Scheduler.

The API Server acts as the Kubernetes API's front-end, processing user requests, cluster components, and external applications. It handles all operations on the cluster's resources, ensuring they are correctly validated and configured. The API Server ensures that the cluster operates as expected by managing data for objects such as pods, services, and deployments (The Kubernetes Authors, 2024c).

Etcd is a distributed key-value store for configuration data and state information. It serves as the cluster's backing store, ensuring the consistency and availability of data. Etcd maintains records of the cluster's state, configuration details, and the current status of all nodes and pods. This centralized data storage allows the Master node to effectively track and manage cluster components, facilitating recovery and reconfiguration in case of failure (The Kubernetes Authors, 2024b).

Finally, we have the *Scheduler*. This process assigns tasks to nodes based on resource availability and specific scheduling policies. Its primary goal is to ensure efficient resource allocation across all nodes to maintain performance in the cluster. The Scheduler considers factors such as CPU and memory requirements, and data locality to make informed scheduling decisions that balance the workload across the cluster (The Kubernetes Authors, 2024d).

2.1.3 Nodes

Kubernetes nodes provide the runtime environments necessary for containers to operate. These nodes ensure the reliability and interoperability of containers and pods, facilitating communication between pods and executing commands from the Master node. They also provide information about the status of running containers. Each node runs processes such as the kubelet and kube-proxy, with the optional addition of cAdvisor for enhanced performance monitoring.

The *kubelet* is the primary agent on each node, responsible for ensuring that containers are running and maintaining the node's state. It interacts with the container runtime to manage the lifecycle of containers, including starting, stopping, and monitoring them. The kubelet also monitors the health of the pods and reports this information back to the API Server, contributing to the overall health and stability of the cluster.

The *kube-proxy* maintains network rules on each node, enabling communication between pods across different nodes. When a service is created, it is assigned a cluster-wide virtual IP address. The kube-proxy manages network routing for services, forwarding traffic based on the service's IP address and port, thereby maintaining efficient network traffic management.

cAdvisor (Container Advisor) is an optional component that provides valuable insights into the node's performance and its containers. It collects status information and usage statistics about containers, gathering metrics on resource usage and performance over time. This data is helpful for monitoring and optimizing the performance of the node and its running containers (Rensin, 2015).

2.2 Edge Computing

Edge computing is a distributed computing paradigm that brings computation and data storage closer to data sources, such as sensors, IoT devices, or users. Instead of relying on centralized cloud data centers, edge computing processes data at or near the network's edge, reducing latency and enhancing response times. This approach is particularly beneficial for applications that require real-time data processing, such as autonomous vehicles, industrial automation, and remote monitoring, where minimizing delays and ensuring quick decision-making are important. By decentralizing computing resources, edge computing enhances security and resilience in networked environments. Shi et al. (2016) defines edge computing as follows:

> Edge computing refers to the enabling technologies allowing computation to be performed at the edge of the network, on downstream data on behalf of cloud services, and upstream data on behalf of IoT services.

2.3 Kubernetes at the edge

Deploying Kubernetes in edge computing environments presents some unique challenges, including computational resources, constrained network bandwidth, and the need for autonomous operation during network disruptions. Traditional Kubernetes distributions are often too resource-intensive for such scenarios. Consequently, several lightweight Kubernetes distributions have been developed to address these constraints. This section evaluates three prominent distributions- KubeEdge, MicroK8s, and K3s- and justifies the selection of K3s for our deployment.

KubeEdge

KubeEdge extends native Kubernetes capabilities specifically for edge nodes, introducing components such as EdgeCore and CloudCore, which facilitate seamless cloud-edge communication (Xiong et al., 2018). It is optimized for intermittent connectivity, supporting autonomous edge node operation during network disruptions. These features make KubeEdge particularly suited for applications demanding offline operation and real-time edge processing. However, empirical studies have indicated that KubeEdge has higher resource usage and lower scalability than the presented alternatives, limiting its practicality in resource-constrained environments (Yakubov & Hästbacka, 2025).

MicroK8s

MicroK8s, developed by Canonical, is a CNCF-certified Kubernetes distribution with a minimal footprint, explicitly designed for IoT devices and edge scenarios (Canonical Ltd., 2024). It provides straightforward installation and robust support for popular add-ons like Istio, Knative, and Prometheus, enabling flexible configurations for diverse use cases. Despite its benefits, MicroK8s demonstrates higher CPU and memory usage than other lightweight distributions, such as K3s, and favors extensibility over minimal resource usage (Koziolek & Eskandani, 2023; Yakubov & Hästbacka, 2025).

K3s

K3s, developed by Rancher Labs, is designed for resource-constrained edge computing environments (Dubey, 2021). It features a minimal, single-binary architecture, simplifying deployment and reducing operational complexity. K3s excludes some

Kubernetes features and employs lightweight alternatives for the most important components, lowering its resource footprint. However, it should be noted that the distribution choice depends on the use case. Our choice of K3s comes down to its ability to function on low-powered devices due to its low memory usage and resource requirements, as discussed by Vatne (2022).

2.4 Understanding Bandwidth

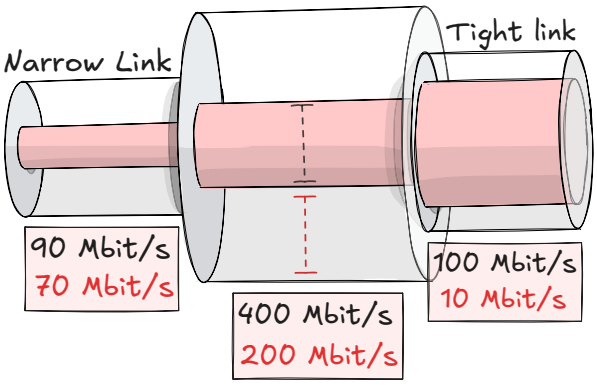

Figure 2.1: Graphical representation of the difference between a narrow (left) and tight link (right). Each pipe joint represents a network link, where the red-shaded part illustrates used capacity, and the outer part shows the residual capacity.

Within network communication, **bandwidth** refers to the maximum rate at which data can be transmitted between two points over a given time, typically measured in bits per second (bit/s). In packet-switched networks, this concept can be further categorized into two terms: **capacity** and **available bandwidth** (ABW).

2.4.1 Capacity and Bottleneck Capacity

The term **capacity** in networks generally defines the maximum data-carrying capability of a network link or path under ideal conditions. Chimento and Ishac (2008) defines nodes as devices such as hosts, routers, Ethernet switches, or any other equipment where the input and output links may have different characteristics. A link is a connection between two nodes, while a path P of length n consists of a series of links $(L_1, L_2, ..., L_n)$ that connect a sequence of nodes $(N_1, N_2, ..., N_{n+1})$. Within this context, a source S is located at N_1 and the destination D, at N_{n+1}.

Note that with this definition, capacity is relative to where in the network stack it is

measured. For instance, the *Nominal Physical Link Capacity*, defined as the theoretical maximum amount of data that a link L can support (Chimento & Ishac, 2008), will always be larger than the IP layer capacity and the protocol-specific network layer capacity. This means that the effective capacity is slightly lower than the raw physical rate due to protocol overhead such as headers, framing, and error correction bits. Thus, when discussing capacity in an end-to-end sense, we consider IP-layer capacity, which can be defined as the maximum rate at which data can be delivered when considering link-layer overhead.

In a network path, the end-to-end capacity is determined by the bottleneck link, which is the link with the lowest capacity on the path and ultimately limits the capacity of the entire path. We adopt the formal definition by Prasad et al. (2003) for end-to-end capacity expressed as:

$$C = \min_{i=1...H} C_i, \tag{1}$$

where C_i is the IP-layer capacity of the ith link, H is the number of links in the path, and C is the minimum of the link capacities C_i, after accounting for link-layer protocol overhead on each hop. The link with the minimum capacity is typically called the **narrow link**, and as visualized in 2.1, the tight link can be defined as the link with the least residual capacity. It is important to note that the narrow link is not always the tight link due to *cross traffic* not necessarily being equally distributed on the path. In network traffic, cross traffic can be defined as any unrelated data flows that compete for bandwidth on the same network path as the probe or measurement traffic, acting as a source of interference, affecting the available bandwidth that can be measured by estimation tools (M. Liu, 2006; X. Liu et al., 2007).

2.4.2 Available Bandwidth (ABW)

Available bandwidth refers to the portion of a link or path's capacity currently unused and thus available for carrying additional traffic. We can formulate this relation as

$$A_i(t) = C_i - U_i(t), \tag{2}$$

where $U_i(t)$ is the current utilized bandwidth on link i at time t. Extending this to a path, the available end-to-end bandwidth at time t is the minimum over all hops, also known as the tight link:

$$A_{path}(t) = \min_{i=1...H} C_i - U_i(t) \tag{3}$$

By definition, available bandwidth can never exceed the capacity of a path. It reflects the ability of the network path to carry data in real-time, given the current traffic conditions. ABW is a dynamic metric that varies from moment to moment as flows start, stop, or change their rates. This makes ABW particularly important for adaptive network management and Quality of Service (QoS) provisioning (Zhao et al., 2009). For instance, specific applications like video streaming or control systems may require a minimum ABW to function properly, so knowing the ABW helps in admission control and routing decisions.

While this definition is accurate when we look at ABW for a single static moment, it does not give us any information about the link utilization over time. At any given time, when we transfer information across a physical medium, the rate at which data is sent is always at the maximum capacity. However, if we take the average instantaneous utilization of a link over a given time interval, we get a more meaningful metric.

Prasad et al. (2003) defines the utilization over a time period $(t-\tau, t)$ as the average utilization $\overline{u}(t-\tau, t)$, given by

$$\overline{u}(t-\tau, t) = \frac{1}{\tau} \int_{t-\tau}^{t} u(x)dx, \tag{4}$$

where $u(x)$ is the instantaneous ABW from Equation 2, and τ, the averaging timescale of the available bandwidth. We then define the available bandwidth on a link as the unused capacity on a link i and the end-to-end available bandwidth as the minimum available bandwidth of all H links as:

$$A = \min_{i=1,...,H} A_i \tag{2.1}$$

2.4.3 Effects of Cross-Traffic and Queueing

Because the amount of cross-traffic determines ABW, it is inherently tied to queueing and traffic behavior when moving from one link to another. If a link's outgoing traffic rate exceeds its capacity, excess packets are queued in buffers, introducing delay. This principle is often used to infer available bandwidth.

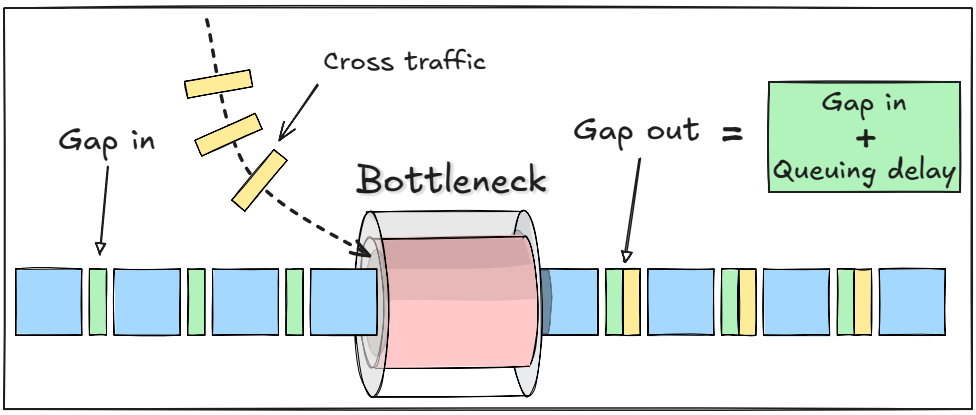

Figure 2.2: The gap between packets increases when cross traffic causes queueing delays.

The queueing delay will increase when a stream's rate surpasses the path's ABW. For example, consider sending two back-to-back probe packets through a network path. If the path's available bandwidth is larger than the rate at which the packets were sent, the spacing between the two packets at the receiver will be roughly the same as when they were sent. However, if the inverse of the input gap, known as the sending rate, is higher than the ABW of the tight link, the second packet in the pair will be enqueued, and the pair emerges with a larger time gap. As a result, the output gap (gout) between packets grows relative to the input gap (gin) once the sending rate exceeds the ABW, as shown in Figure 2.2. This effect is illustrated by the gap response curve, visualized in Figure 4.5, where increasing queueing delays signal that the probing stream has saturated the tight link. Queue buildup indicates that the available bandwidth threshold has been crossed.

It is also worth noting that available bandwidth is time-dependent. Measurements of ABW are more volatile than capacity measurements because even minor changes in network traffic can alter the unused portion of bandwidth. Therefore, when discussing or measuring ABW, one should always consider the time frame of the measurement (Chimento & Ishac, 2008). A single snapshot over a one-second interval may not be representative if the network conditions fluctuate rapidly. For example, an interval of one minute could obscure short bursts of congestion that momentarily drop the available bandwidth. In contrast, shorter windows might capture only a spike in traffic that does not indicate typical conditions.

2.4.4 Achievable bandwidth

In addition to capacity and available bandwidth, it is helpful to consider the achievable bandwidth on a network path. Achievable bandwidth, or achievable throughput, refers to the data rate a particular transport protocol can obtain under current network conditions. This value is inherently influenced by the protocols and traffic dynamics in use. For example, using a tool like iPerf to send as much data as possible over a path will reveal the maximum throughput that can be achieved at that moment. Notably, the achievable bandwidth of a flow is constrained by, but not necessarily equal to, the available bandwidth.

A TCP flow's throughput depends on many factors beyond raw available capacity, including socket buffer sizes, the level of cross-traffic congestion, round-trip time, and packet loss rate (M. Jain & Dovrolis, 2003). Even if a path has spare capacity, a particular flow might not fully utilize it due to protocol overheads or interactions with other traffic. In other words, achievable bandwidth will always be less than or equal to the available bandwidth on a path, with the gap determined by the transport protocol behavior and network conditions. By distinguishing these terms, we recognize that a high available bandwidth does not always guarantee that any given application can obtain that throughput. This distinction is important in dynamic networks since a bandwidth estimation tool must be aware that any introduced traffic might cause protocols such as TCP to temporarily back off due to congestion, thereby changing the available bandwidth at the time of measuring.

2.5 Bandwidth Estimation

Knowing a network's capacity and available bandwidth is helpful for network management, but directly measuring ABW is challenging. **Bandwidth Estimation** (BWE) refers to the techniques and processes used to infer a path's capacity or available bandwidth through observations and measurements. In the context of this thesis, BWE, specifically available BWE (ABE), is needed to enable network-context-dependent scheduling of Kubernetes nodes deployed on the edge. However, achieving consistently accurate BWE across diverse network environments is non-trivial. Numerous methods have been proposed over the years, and researchers have published comprehensive surveys to classify these techniques (Airon & Gupta, 2017; Chaudhari & Biradar, 2015). Broadly,

bandwidth estimation methods fall into two categories: *active* techniques and *passive* techniques. Active methods inject probe traffic into the network to measure how the network responds, whereas passive methods observe existing traffic to infer bandwidth characteristics. Each approach has assumptions, strengths, and weaknesses, which we discuss below.

2.5.1 Active Probing

Active probing techniques deliberately send probe packets or packet trains through the network and analyze their behavior to estimate bandwidth-related metrics. By measuring properties such as round-trip time (RTT), one-way delay (OWD), packet dispersion, or loss rate of these probes, active methods can infer the capacity or available bandwidth of a path or a specified link (Airon & Gupta, 2017). A wide range of BWE techniques exists, and the following section outlines the core idea behind the most prominent ones.

Self-Loading Periodic Streams (SLoPS)

This approach sends periodic streams of packets at an increasing rate until the network's available bandwidth is reached. It is based on the concept of self-induced congestion, as the probe stream rate ramps up, the OWDs of packets are monitored. When the probing rate exceeds the ABW of the path, packets begin to queue at the bottleneck, and their delays increase. By detecting the point at which delays start rising, SLoPS infers the available bandwidth (M. Jain & Dovrolis, 2003). Pathload, introduced by M. Jain and Dovrolis (2003), is a classic implementation of this technique.

Packet Pair/Train Dispersion (PPTD)

In packet pair or packet train dispersion methods, pairs of back-to-back packets (or longer trains) are sent, and the gap between packets is measured at the sender and receiver (Dovrolis et al., 2004). Under the probe gap model, if a path is not saturated, the inter-packet gap remains the same, but if a packet pair encounters a slower link, the pair gets spread out proportionally to the link's capacity. Packet pair techniques are traditionally used to estimate capacity since the dispersion of a closely spaced pair can reveal the bottleneck link's transmission rate (Airon & Gupta, 2017; Prasad et al.,

2003). Tools like Pathrate use this principle to measure bottleneck capacity (Dovrolis et al., 2004).

Variations of packet train dispersion can also be used for ABW: for example, sending longer trains or multiple pairs can help average out noise and detect when additional spacing is introduced due to queueing (Melander et al., 2000; Prasad et al., 2003). (Obara et al., 2012) suggests that packet train dispersion can measure available bandwidth with higher accuracy if certain conditions, such as stable cross traffic, are met.

Variable Packet Size (VPS) Probing

This technique involves sending probe packets of different sizes to analyze how RTT or OWD varies with packet length. The underlying idea is to infer per-hop capacity by seeing how additional bits affect transmission delay. Using the Time-to-Live (TTL) field in the IP header can cause probes of various sizes to expire at specific hops along the path, measuring the RTT to each hop. Plotting the minimum RTT as a function of packet sizes yields a slope corresponding to the bottleneck link's serialization delay, thereby revealing that link's capacity. Variants of this method can map out the capacity of multiple segments in the path. VPS probing is mainly used for finding link capacities and locating the narrow link rather than for measuring available bandwidth, since it does not directly account for cross traffic (Turrubiartes et al., 2005).

Trains of packet pairs (TOPP)

TOPP is a hybrid approach aimed at estimating available bandwidth by combining aspects of packet pair and self-induced congestion techniques. In TOPP, multiple packet pairs are sent back-to-back at different sending rates, and by incrementally increasing the rate of these packet-pair trains, we can observe how the packet gaps change from sender to receiver. At low rates, the dispersion of each pair at the receiver will shrink inversely with the sending rate as long as the path is underutilized. However, once the probing rate nears the ABW, additional increases in rate no longer lead to proportional decreases in the output gap (gout). Instead, the gap stabilizes or widens due to queueing. By finding the point at which this transition occurs, TOPP pinpoints the available bandwidth of the path. Using multiple packet pairs helps average out jitter or timing noise, improving accuracy. This method was first described by Melander et al. (2000),

and further discussed by Prasad et al. (2003) as a reliable way to estimate ABW using probe traffic.

2.5.2 Passive Bandwidth Estimation

Passive bandwidth estimation techniques are non-intrusive methods that infer bandwidth characteristics by observing existing network traffic rather than generating dedicated probe packets (Chaudhari & Biradar, 2015). In passive BWE, the tool or algorithm listens to the flow of data already present in the network, for example by capturing packets on a link or by instrumenting end hosts, and analyzes patterns such as packet timings, sizes, and protocol behavior. Because no extra traffic is injected, passive methods avoid adding load to the network and are ideal for scenarios where minimizing interference is important. These techniques leverage the fact that protocols such as TCP naturally adjust to network conditions, and their observable behavior, such as inter-packet gaps, acknowledgements, and congestion window size, carries implicit information about the available bandwidth and congestion state (Khangura & Fidler, 2017).

Purely passive approaches monitor metrics such as packet OWDs, RTTs, inter-arrival gaps, loss rates, and throughput statistics of ongoing flows to draw conclusions about bandwidth. For example, a steady increase in OWD or RTT over time can indicate that a queue is building up on some link, implying the flow's sending rate is higher than the path's available bandwidth. Similarly, monitoring inter-arrival times between incoming packets helps identify variations caused by network congestion or bottlenecks (Brakmo & Peterson, 1995; Gerla et al., 2001).

Passive observation of a TCP flow can be especially revealing, as it will naturally slow down its sending rate when a congestion event, such as packet loss, occurs. Algorithms like TCP Vegas (Brakmo & Peterson, 1995) monitor the RTT of packets, inferring impending congestion by seeing the RTT rise as the network path neared saturation. Common techniques in passive BWE include analyzing inter-packet arrival patterns, together with acknowledgement gaps, and throughput fluctuations of existing flows to estimate the residual capacity (Airon & Gupta, 2017; Arsan, 2012; Tursunova et al., 2010).

Because passive methods rely on existing traffic, they can struggle in scenarios where traffic is sparse or not sufficiently aggressive to probe the network's limits. In a completely idle network, a purely passive approach has nothing to observe and cannot

determine available bandwidth. Some passive techniques incorporate a slight proactive element to address this limitation, essentially a hybrid approach (Nam et al., 2013). These are sometimes called proactive, passive, or implicit probing methods (Chaudhari & Biradar, 2015). The idea is to inject only a minimal amount of additional traffic when needed or to identify an unknown variable, such as the bottleneck link capacity, which can then be used to increase the accuracy of the ABW estimate. The key distinction from fully active methods is that these probes are used sparingly, so the approach remains non-intrusive.

Khangura and Fidler (2017) proposed a sender-side passive estimation technique that uses the probe gap model to estimate available bandwidth. Instead of actively sending dedicated packet pairs, they exploit the naturally occurring packet pairs in the traffic to measure dispersions. By doing so, entirely at the sender, their method could infer the available bandwidth, showing accurate results in a simple physical network. Khangura (2019) further improved on this concept by incorporating machine learning into the model, improving accuracy in networks where the network experiences more chaotic behavior.

Another recent example is SABES (Statistical Available Bandwidth Estimation), which proposes a method using statistical analysis of passive TCP measurements to infer available bandwidth (Ciaccia et al., 2020). This method is further improved by training the neural network on the observed patterns to improve its estimates.

2.5.3 Factors Influencing Estimation Accuracy

Bandwidth estimation, whether active or passive, is sensitive to a number of factors. Understanding these factors is important for designing an effective BWE tool and interpreting its results. (M. Jain & Dovrolis, 2004) highlights that assumptions about traffic can easily lead to inaccuracies. For example, the burstiness of cross-traffic, the choice of packet sizes for probing, and the measurement interval length all affect the accuracy of bandwidth estimates. This section discusses these factors, particularly in the context of passive bandwidth estimation.

Available Bandwidth variability

One fundamental challenge in bandwidth estimation is that ABW is a time-varying metric. The path's capacity may be fixed, but the available portion can change

momentarily as cross traffic flows start, stop, or change their rates. This variability can occur over multiple time scales (Chimento & Ishac, 2008; M. Jain & Dovrolis, 2002). For example, on a congested link, available bandwidth might drop sharply when a large file transfer begins and then rise again when that transfer completes or enters a less intensive phase. Even on shorter time scales of seconds or less, bursty traffic can cause the available bandwidth to fluctuate around some average value. Because of this, any single measurement of ABW provides only a snapshot of the current network's state. Suppose a bandwidth estimation tool assumes a steady state during the measurement. In that case, it may be misled by changes that occur just after the measurement or even during the probing/listening process.

Tools like Pathload (M. Jain & Dovrolis, 2002) explicitly account for this by reporting a range of values rather than a single point estimate (M. Jain & Dovrolis, 2005). By outputting an upper and lower bound "grey region", Pathload conveys the variability in the ABW during the measurement interval. This approach is helpful because it recognizes that the true ABW might oscillate within a range due to normal network dynamics.

Cross traffic patterns

The pattern of cross traffic significantly influences the accuracy of bandwidth estimation. Different classes of cross traffic, such as bursty, periodic, and more fluid-like cross-traffic, interact with probe traffic in various ways, and tools often make assumptions, simplifying these patterns. Many estimation algorithms assume that cross traffic behaves like a fluid with a constant rate during the measurement interval (Strauss et al., 2003). The rationale behind this assumption is that to measure the cross-traffic impact on the probe traffic accurately, the cross-traffic needs to be equally distributed across the measurement interval. For instance, perfectly fluid constant-rate cross traffic would cause equal queueing delays unrelated to when the probe packet was sent. In reality, cross traffic is usually not at a constant rate and is likely not fluid-like.

Bursty cross traffic is characterized by intermittent surges in packet transmissions, often with large packets at Maximum Segment Size (MSS). For example, web browsing or cloud backup traffic often comes in bursts. Regarding network behavior, bursty traffic creates short-term spikes in queue lengths, posing a challenge for bandwidth estimation algorithms. The transient effect of bursty traffic on ABW estimation is that it partly

breaks the analytical models assuming smooth cross traffic. If the estimation algorithm does not account for burstiness, its results can be erratic or biased, and in turn less accurate (Guerrero & Labrador, 2008; K.-J. Park et al., 2009).

Packet Sizes and Probe traffic pattern

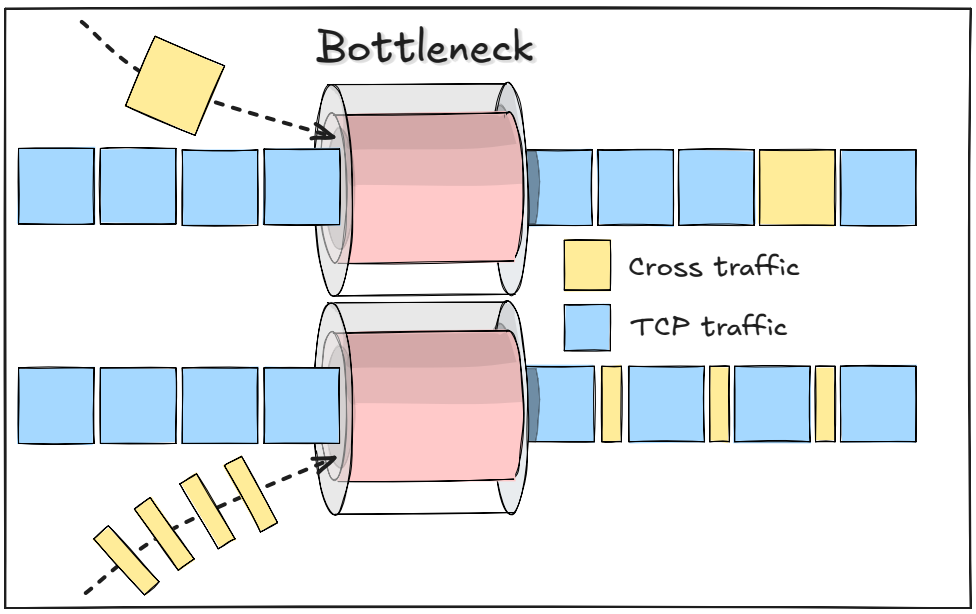

Figure 2.3: Illustration of the impact of cross-traffic packet sizes. When the cross-traffic packet sizes increase, the number of packet pairs experiencing a gap decreases. The blue boxes can be interpreted as active probing traffic or TCP traffic for passive, sender-side listening.

The characteristics of the probe and cross-traffic can significantly influence bandwidth estimation accuracy, and are one of the key differences between how different tools operate. Packet size matters because network routers and links deal in packets of specific sizes, not an infinitely divisible fluid flow. If we assume fluid cross-traffic, no matter the probe packet size, each probing packet will experience queueing delay in direct relation to the utilization of the link. On the other hand—as visualized in figure 2.3—if all the cross traffic consists of large MSS-sized packets, any pair of packets is more likely to land next to one another in the queue, showing no signs of congestion. Similarly, if we send more packets with a lower size as probes, we increase the likelihood that at least some of the packets experience queueing.

In active bandwidth estimation, the tool chooses both the size of each probe and the timing pattern, such as packet pairs, trains, or chirps. In contrast, passive techniques

are limited to observing existing traffic and must infer network characteristics based on whatever happens to traverse the link. This difference has several consequences:

- Due to the lack of control over packet sizes, passive estimators must work with the packet sizes appearing in the traffic trace. To be able to infer ABW, we typically have to use constant packet sizes as changes in queueing delays can be directly attributed to the network capacity and cross traffic interference (Dovrolis et al., 2004; Huang et al., 2024).
- Active tools can space probes in a controlled fashion to detect the onset of queueing. Passive methods must observe inter-arrival gaps determined by application and transport layer behavior, which may be bursty or synchronized with higher-layer timers. As a result, passive methods must perform analysis on less predictable datasets, often requiring more data.
- Passive tools cannot control the probe intensity of existing traffic, so the measured data can be both too high, shifting the network ABW, such as with iPerf, or too low, not causing any queueing delays. Consequently, results produced by passive tools have the potential to be as accurate as those produced by active tools when the observed traffic behaves as expected. Still, they might produce invalid results when these patterns change.

Aspect	**Active**	**Passive**
Control over probes	Full (size, timing, intensity)	None (depends on existing traffic)
Intrusiveness	Can be intrusive	Non-intrusive
Sample density	Adjustable	Variable (low in quiet periods)
Model assumptions	Known probing pattern unknown cross traffic	Infer both probe and cross traffic patterns
Applicability	Controlled and on-demand	Long monitoring or post analysis

Table 2.1: Main differences between active and passive bandwidth estimation methods.

Because of these challenges, passive bandwidth estimation techniques often rely on statistical analysis and long observation windows, fitting a model to the measured packet inter-arrival times, and reconstructing the underlying ABW. This can work well when traffic patterns are relatively stable and flows persist long enough to produce

many samples. However, they may yield wide confidence intervals or produce unusable estimates in highly dynamic or sparse traffic conditions.

2.6 Bandwidth Estimation in Radio Networks

Estimating available bandwidth (ABW) in radio networks is significantly more complex than in wired networks. Wireless links exhibit high variability in capacity and quality due to physical phenomena, and the medium is shared, resulting in complex link - and physical-layer interactions (P. Gupta & Kumar, 2000). This section discusses the physical-layer impairments, medium access issues, and broader network-level dynamics that complicate ABW estimation in wireless networks. It also explains why classical estimation methods, such as the ones mentioned in Section 2.5, often struggle or fail in this context, motivating the decision to shift our experiments from a wireless setting to a controlled testbed using emulated wired links.

2.6.1 Physical Layer Variability and Interference

Wireless channels are highly variable and prone to interference at the physical layer. Unlike a wired link with a fixed capacity and dedicated channel, a radio link operates over a shared spectrum subject to signal propagation effects. Phenomena such as multipath fading, signal attenuation with distance, path loss, and external interference cause the usable capacity of a wireless link to fluctuate over time (Mukta & Gupta, 2020). In practical terms, the *signal-to-noise ratio* (SNR) experienced by a wireless receiver can change rapidly due to the movement of nodes or obstacles, weather, or other devices transmitting on the same frequency. A lower SNR leads to higher error rates, forcing the link to use a more robust modulation scheme or to retransmit lost frames, reducing the adequate bandwidth and increasing latency (Nishimori et al., 2012).

Because these signal conditions are dynamic and often unpredictable, we cannot rely on a consistent physical layer throughput when estimating bandwidth. The connection is stable in wired networks, and errors don't happen very often. In contrast, wireless networks can have fluctuating capacity, so any measurements taken can quickly become outdated. There are some exceptions to this, such as stationary cell towers communicating with each other, which will fluctuate primarily due to weather, especially if they are positioned in a manner where it is highly unlikely that obstacles will appear.

Interference is another challenge observed at the physical layer of wireless communication. Neighboring wireless devices, ranging from Bluetooth devices, telephones, and even microwave ovens, although very unlikely to be found in tactical scenarios, can introduce background traffic or noise on the channel. This interference can be co-channel interference (Aina et al., 2021; Garg, 2007) or adjacent-channel interference, and it effectively reduces available bandwidth for the network by corrupting frames or increasing the idle time of the transmission medium. Moreover, wireless networks are susceptible to the hidden node problem, where a transmitter may not sense another interfering transmitter, but their signals collide at a common receiver (Kosek-Szott, 2012). Hidden nodes cause unpredictable packet collisions and lost capacity that standard channel sensing mechanisms do not detect easily. Overall, the physical-layer unreliability and interference in wireless media result in an ABW that is both time-varying and hard to measure accurately.

2.6.2 Link-Layer Shared Medium Access

At the link layer, wireless networks use a shared medium access protocol, such as IEEE802.11's Distributed Coordination Function (DCF), to coordinate transmissions (Kumar et al., 2004; Shah et al., 2003). All nodes on the same channel contend for access, but only one transmission can effectively use the channel simultaneously in a given collision domain. This contention-based Media Access Control (MAC) means that available bandwidth is not merely a function of unused capacity on a single dedicated link but rather the fraction of time the channel is free for a new flow. If one node is transmitting, others must wait, which means that the usable bandwidth for any one flow depends on how the MAC protocol performs its scheduling of transmissions among contenders (Sarr et al., 2008).

802.11 WLANs typically use the distributed coordination function (DCF), which combines carrier-sensing and random backoff intervals to avoid collisions. Even under steady traffic, this introduces variability in packet delays. Unlike a wired link, where a router's queue might implement first-come-first-served scheduling, a wireless MAC does not strictly serve packets in order of arrival. A neat First In First Out (FIFO) queue assumption does not hold in 802.11 DCF (Bredel & Fidler, 2008). Instead of one flow simply filling any leftover capacity behind another in a queue, multiple wireless flows share the channel through a pseudo-random access protocol that aims for a form

of fairness over time. For example, two active flows on a WiFi link will roughly split the capacity, regardless of which started first, due to the DCF backoff algorithm giving each a chance to transmit (Bianchi, 2000). This means the new flow's "fair share" of the capacity might be much lower than the actual residual capacity if other devices are active. Consequently, traditional bandwidth estimation tools that assume a fixed bottleneck may measure this fair share rather than the actual available bandwidth.

Another link-layer challenge is the overhead and optimization features of wireless protocols. Techniques like automatic repeat request (ARQ) for error recovery and dynamic rate adaptation mean the effective link throughput can change without warning (Paris et al., 2015). If a few packets are lost due to noise, 802.11 will retransmit them (up to a limit), causing variable delays that an active probing tool might misinterpret as congestion. Rate adaptation algorithms adjust the physical modulation/coding rate based on recent SNR or packet losses, which means that under poor conditions, the link may shift to a lower bit rate, reducing the capacity available to all flows (Acharya et al., 2010; Wong et al., 2006). This change is not signaled to the network layer (Aboba, 2007) and cannot be known by an end host directly, meaning that an ABW measurement taken just before such a rate drop will immediately become invalid (Mingzhe Li et al., 2008). Similarly, modern WiFi standards such as 802.11n/ac/ax introduce features like frame aggregation, where multiple packets are bundled into a single transmission to improve efficiency (Skordoulis et al., 2008). Aggregation changes the timing of packets and how airtime is used, which can confuse probe-based estimation tools that think each probe packet faces separate transmission delays (Farshad et al., 2014).

2.6.3 Network-Layer Dynamics and Topology

Network-layer factors beyond a single wireless link add further complexity to bandwidth estimation in radio networks. Wireless networks, particularly ad hoc or multi-hop networks, often have dynamic topologies and multi-hop paths with several wireless links. Each hop's capacity may fluctuate independently due to local signal conditions, and interference can occur between hops (K. Jain et al., 2003; Padhye et al., 2005). Even in a wired path, where the capacities are stable, available bandwidth estimation is a significant challenge due to the possibility of several bottlenecks having a cumulative effect on packet gap measurements (Khangura & Akın, 2021; Xiliang Liu et al., 2008). This is made significantly more complicated in wireless networks since we can have several

bottlenecks *and* they can shift with environmental changes or node mobility. Moreover, the interference range of a wireless transmission is often larger than the communication range (Iyer et al., 2006). This means that even nodes that are not direct neighbors can cause interference on the channel, effectively reducing the available bandwidth without being part of the path's logical topology. For example, in a wireless mesh, two flows on nearby links can each degrade the other's throughput even if they don't share a router. All these factors lead to spatial and time variability in available bandwidth.

Wireless ad hoc networks also lack a centralized scheduling of transmissions, relying on distributed protocols that react to conditions (Agarwal et al., 2001). Route changes or link outages due to mobility will change which links are carrying traffic, sometimes introducing transient congestion or underutilization. Each node in such a network has only a local view of channel usage. As a result, each host has imprecise knowledge of the overall network status, and links change dynamically in ad hoc networks. This makes coordination for bandwidth measurement difficult, as a sender might not know that a given time interval was atypically idle or busy.

2.6.4 Limitations of Traditional Estimation Techniques

Because of the above challenges, bandwidth estimation tools and techniques that work well in wired networks often struggle in wireless environments. For instance, tools using the probe-gap model, such as Spruce and IGI, send pairs of back-to-back packets and examine the one-way delays, inferring available bandwidth from how much these probes were delayed by queueing. Tools such as Pathload and PathChirp use the probe-rate model, which gradually increases the sending rates of probe traffic until they detect increased delay or loss (Cottrell, 2003; M. Jain & Dovrolis, 2002). These approaches assume a path with a single bottleneck link with fixed capacity, fluid-like cross traffic, and FIFO queueing. In wireless networks, however, those assumptions break down. The medium access method is not FIFO, and the bottleneck capacity can fluctuate. A train of probe packets might, instead of inducing a clean queue buildup, cause the wireless nodes to back off or share airtime with cross traffic in a way that masks the true available capacity. It has been observed that standard probe-based tools yield unreliable available bandwidth estimates on wireless links Abdelaziz et al. (2007), Bergfeldt et al. (2011), and Lakshminarayanan et al. (2004).

Specific problems arise for each class of tool. Probe-gap tools can be fooled by

variable transmission times, but in WiFi, even if the channel is free of cross-traffic, the spacing between two packets at the receiver can increase due to random backoff or a drop in link rate, not necessarily because another flow consumed capacity on the same channel (Haider et al., 2018). The inherent variability of the wireless channel thus introduces noise into the delay measurements unrelated to available bandwidth (Feng & Shu, 2007). Likewise, self-induced congestion algorithms like Pathload face difficulty converging on a correct estimate in wireless conditions. Pathload expects that as it increases the sending rate, delays will, at some point, increase past a threshold when probes exceed the available bandwidth. In reality, a wireless path might show sporadic delay spikes even at low rates due to retransmission and/or interference, and thereby not show a clear indication of congestion at the true ABW rate because the MAC is dividing airtime among nodes (Takano et al., 2016). The distributed and 802.11 fairness mechanisms cause active probe tools to provide inaccurate estimates in wireless scenarios (D. Gupta et al., 2009). In other words, the tool might think the path is saturated when it is not, or it might fail to see that the channel was busy because the probes waited for their turn. Even advanced probe techniques struggle with issues like the optimal probe packet size or the need to disable MAC features. In some studies, RTS/CTS handshaking is always used to simplify modeling, which is not always true in practice.

Due to these limitations of active probing, some research has advocated passive estimation in wireless networks by measuring idle/busy times to infer ABW (Khan & Veitch, 2008; H. J. Park et al., 2010). As explained in Section 2.5, passive approaches avoid injecting extra traffic, thus not disturbing the network, which provides an advantage in a wireless context where any additional packets can contribute to collisions or interfere with primary traffic. Passive monitoring can directly observe how much airtime is free, but this does not provide information about the end-to-end available bandwidth, only a per-hop estimate. Other approaches, such as the one described by Sarr et al. (2008), combine collision probability estimation with channel utilization measurements to estimate ABW and improve quality of service.

However, passive methods require the presence of enough regular traffic and may need calibration or lower-layer information that is not exposed to typical applications. Azevedo et al. (2018) attempts to improve existing active probing methods by implementing a detection algorithm for frame aggregation, improving the overall estimation accuracy.

2.6.5 Implications and Rationale

This thesis originally intended to deliver and validate an available bandwidth estimation tool that could operate in heterogeneous ad hoc infrastructures with volatile conditions. However, as the above sections outline, ABW estimation is generally a hard problem to solve, and even harder in wireless networks. Our initial testbed was an emulated wireless multi-hop network, and the goal in mind was to use packet capture to estimate ABW on this network. Still, early measurements made it clear that the problem was more complex than anticipated. Additionally, emulating different types of radio technologies requires intricate knowledge of signal propagation and interference models, and even if configured correctly, it might still fail to emulate realistic behavior accurately. Regarding research question 1, the preceding section underscores the complexity and inherent unpredictability of bandwidth estimation in wireless networks, validating our decision to refocus the prototype on more controlled, wired scenarios.

The resulting scope is then narrowed to creating a passive, sender-side estimator that operates on wired links whose capacity, delay, and loss can be dialed through emulation software. The tool captures traffic passively, extracting real-time TCP packet-gap statistics to derive ABW estimates. Because the emulator supplies a verifiable ground truth, we can show how link impairments and cross-traffic patterns impact accuracy. By publishing both the code and the evaluation setup, we aim to provide a baseline tool that can be extended to work in wireless scenarios at a later point in time.

Chapter 3

System Design

Building upon the preceding analysis of bandwidth estimation methods, this chapter presents an overview of the system architecture and the various discussions and considerations that shaped this thesis.

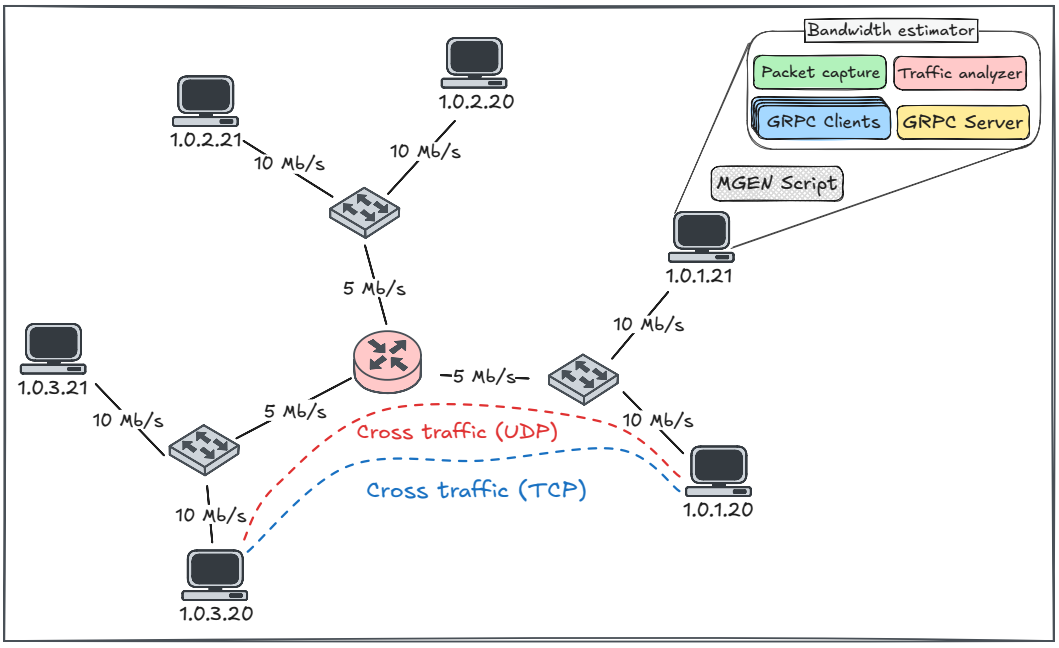

Figure 3.1: Illustration showing the topology used during experimentation. Each pair of nodes has two bottleneck links with equal capacity, and each node runs both an instance of the implemented tool and an MGEN instance for load generation.

3.1 Emulation

Simulation relies on mathematical models and abstract representations to approximate network behavior. It is efficient for exploring theoretical network scenarios, and because

it works within simulated time, it does not need to meet the same requirements as emulated networks. This enables the simulation of more complex environments and realistic models of the physical properties. On the other hand, emulation needs to keep up with real-time systems and, therefore, has to take certain shortcuts, such as using less computationally expensive models. It can, however, utilize real network hardware and software to transfer real and synthetic data while emulating the medium on which data is transferred.

Network emulation has evolved as a response to the limitations of network simulation and real-world testing. Initially, emulation allowed real-world traffic to interact with simulations, providing a platform to test algorithms and protocols under controlled conditions (Fall, 1999). Over time, the development of network emulators like Dummynet (Rizzo, 1997), NIST-Net (Carson & Santay, 2003), and the Linux Traffic Control subsystem ("Introduction to Linux Traffic Control", n.d.) has provided tools to emulate increasingly complex networks.

Emulation tools create a controlled environment where network protocols and systems interact with synthetically generated network conditions. At a fundamental level, these tools replicate the behaviors of physical network components by using virtualization techniques such as containerization and Linux namespaces. Virtual nodes can be configured with network interfaces that mimic physical ones, allowing the operating system's network stack to process actual data packets in real time. These emulators manipulate packet scheduling and queueing disciplines through mechanisms such as Linux traffic control (tc) to introduce network impairments like delay, jitter, packet loss, and bandwidth restrictions. Since a typical modern computer can transfer data locally at speeds over 10s of gigabits per second, it can emulate large virtual networks when working with impaired links.

It should be noted that even though emulation provides a test environment for software, it does not perfectly represent reality. Instead, it enables us to do low-cost tests before validating results in real networks.

3.1.1 NREC

At the start of the design and implementation phase of the project, we used our own hardware to implement the core parts of the tool. Though this worked well initially, due to the need to scale up the simulation without causing performance

issues, we migrated to the Norwegian Research and Education Cloud (NREC). NREC provided the infrastructure to quickly spin up virtual machines while not overloading the CPU, making the development process more manageable. A common issue regarding network emulation is that the devices share CPU, memory, and network hardware, unlike real physical networks, where each node is a separate physical device. This is especially problematic when analyzing high-fidelity parameters, such as packet gap measurements. To perform ABW estimation, we needed relatively high precision timestamping capability down to a few tens of microseconds, for hundreds or even thousands of packets per second. By switching to NREC, we aimed to minimize timestamping anomalies, though as discussed in Section 5.2.2, we encountered errors even there.

3.1.2 Emulation Tools

Our project required an environment that could mimic real-world conditions yet remain flexible and manageable. Practical constraints ruled out a full-scale physical testbed, making emulation a natural choice. Emulation allows us to introduce real and synthetic data flows under controlled conditions, preserving actual networks' timing and dynamic properties.

We evaluated several emulation tools based on their ability to interface with real network hardware and software, ease of configuration, and support for wireless emulation. The following section outlines the main characteristics of five well-known network emulation tools under open-source licenses.

Tool	Interface/Software	Wireless support	Emulation approach
GNS3	API, GUI	Partial (No link-layer emulation)	Descrete-event
Mininet	Script-based CLI	Available through the Mininet-WiFi fork	Traffic control
Netem	Command-line	No	Traffic control
NS3	Script-based (C++/Python)	Limited	Descrete-event
CORE	GUI, scripted (Python), gRPC	Yes (With EMANE for link-layer emulation)	Traffic control

Table 3.1: Emulation tools and their corresponding attributes.

GNS3 (Graphical Network Simulator-3) is a widely used GUI-based network emulation framework that primarily targets the emulation of real network hardware, especially

legacy and Cisco devices, using a graphical user interface. Its ability to integrate with actual router images and combine virtual components makes it a good candidate for environments where replicating vendor-specific hardware behavior is required. It is commonly used as an educational learning tool, focusing on usability (Chan & Haythorne, 2017). Additionally, although we did not end up using wireless emulation, at the time of choosing our preferred tool, GNS3 did not natively support link-layer wireless emulation, prompting us to look at other tools (Han et al., 2022).

Mininet leverages lightweight virtualization using Linux namespaces and containers to create virtual networks on a single host (Lantz et al., 2010; Muelas et al., 2018). It is primarily developed to emulate software-defined networks, providing prototyping and testing of OpenFlow-based networks. Despite its performance for network emulation, it focuses more on SDN-related research. Initially, this was chosen as the preferred tool, due to the fork *Mininet-WiFi* (R. R. Fontes et al., 2015) providing the functionality needed for wireless emulation. However, at the time of development, it did not support enough configuration of wireless properties, such as modifying the send rate to lower than 1 Mbit/s or introducing synthetic packet loss.

NetEm is a network emulation tool integrated within the Linux kernel's traffic control (`tc`) mechanism, providing capabilities to emulate network impairments such as delay, jitter, and packet loss. Empirical studies have validated that NetEm can accurately replicate delay and packet loss. However, jitter emulation can display lower variability than configured, reflecting effects not explicitly specified by the user (Jurgelionis et al., 2011). Beyond its standalone use, NetEm serves as a foundation component for several higher-level network emulation tools such as CORE (Boeing Research & Technology division, 2025).

NS3 is primarily known as a discrete-event network simulator, and implements detailed modeling of protocol behaviors and network interaction using script-based configurations in C++ or Python (Riley & Henderson, 2010). Running it in *emulation mode* makes it possible to extend its functionality to interact with live network components in real time (H. Fontes et al., 2015), but since it is primarily a discrete-event simulator, the configuration process is less straightforward than the other tools mentioned. Furthermore, Mishra and Rath (2020) discusses issues encountered when building large

heterogeneous emulated networks in ns-3, resulting in random errors in networks with more than 30 nodes.

CORE (Common Open Research Emulator) provides a flexible and relatively easy-to-use environment for network emulation. Similar to most other network emulators, it supports link modifications through tc. Still, it stands out by providing both a Google Remote Procedure Call (gRPC) API (gRPC Authors, 2021) and a GUI for real-time changes to emulated links (Ahrenholz et al., 2008).

These tools are similar in function and use many of the same emulation techniques. The main differences in these tools depend not on their ability to emulate simple networks but rather on how they integrate with other tools and their interfaces. As we explain in Section 3.1.3, CORE integrates with MGEN for load generation and with EMANE for physical- and link-layer emulation of wireless networks, which helps with achieving more realism in test-scenarios (Ahrenholz et al., 2011).

3.1.3 Load Generators

Emulated networks do not inherently produce much traffic, except for small amounts generated by node discovery mechanisms and some routing-related messages. However, if we want to estimate ABW and not just capacity in general, we need to introduce some load on the network, both as general cross traffic and TCP traffic, which can be used for sender-side bandwidth estimation. We have two primary ways of injecting this traffic on the network: replaying recorded network traces or generating new synthetic data.

We mainly looked at tools capable of generating synthetic data because we needed to test different network conditions with differing link characteristics. Replaying real network traces would require a significant time investment in data collection, which we decided was not worthwhile given the relatively small benefit of the added traffic realism.

In the following paragraphs, we introduce the three candidate load generation tools we examined, summarized in the table below. We selected these tools primarily because they are open source, freely available, and each offers distinct capabilities that complement our requirements.

Iperf3 is a tool, built for testing achievable bandwidth on end-to-end network paths, but can double as a synthetic traffic generator as it can be configured with different

Traffic Generator	Key Features
Iperf3	Simple throughput and latency measurements
Trex	High-performance, versatile, configuration heavy
MGEN	Versatile, simple UI, pre-defined traffic patterns

Table 3.2: Key features of the candidate load generators.

target throughput and protocols (ESnet, 2024). Still, it lacks in terms of generating realistic traffic, as it does not support traffic patterns other than simple UDP or TCP streams with target throughputs. In the initial stages of the implementation, we often used Iperf3 to perform tests, but we moved away from using it as our primary load generator when seeking to add more realism to the test setup.

Trex (Systems, 2024) is an open-source traffic generator from Cisco that is capable of fast, stateless, and stateful packet streams. While its high throughput makes it attractive for stress-testing routers and firewalls, deploying it in our CORE setup would have meant anchoring the generator outside every virtual namespace and binding it to DPDK-capable interfaces. This would add an extra layer of complexity that outweighed the relatively simple cross-traffic we needed for our experiments. Additionally, running a single TRex instance requires at least 4GB of memory and four virtual processors, which is not a lot when running a single instance, but quickly adds up when running several instances (Swann et al., 2020).

MGEN emerged as the optimal choice for our testbed. Developed by the Naval Research Laboratory (NRL) (Naval Research Labratory, 2021), it offers a balanced approach, capable of generating both constant and bursty traffic conditions with minimal configuration overhead. Additionally, its native support within CORE simplifies integration, facilitating both the injection of cross traffic and passive TCP measurements for bandwidth estimation.

3.1.4 Integrating CORE and MGEN

Our testbed is constructed around two core components: CORE for network emulation and MGEN for traffic generation. CORE enables the creation of dynamic network scenarios, including emulating wireless behavior. CORE can be extended for enhanced wireless realism with EMANE (Extendable Mobile Ad Hoc Network Emulator), which

provides configurable link-layer and physical-layer properties. Our primary reason for choosing CORE as our preferred emulator was its support for link-layer wireless emulation and traffic generation. However, as discussed in Section 2.6.5, the wireless features became less important when we moved away from wireless networks.

As visualized in Figure 3.1, we constructed an environment where nodes are connected via emulated physical links, all sharing exactly two bottlenecks, with each other node in the network.

Despite some inherent limitations, the combination of MGEN and CORE yields a comprehensive test environment for evaluating the bandwidth estimation tool. While emulation offers a cost-effective and flexible alternative to physical testbeds, it also imposes certain limitations. The need to operate in real time forces the use of simplified models, which may not fully capture the complexity of real-world networks. Furthermore, real networks are often complex in terms of traffic conditions, nodes' computational resources, and protocols, making them difficult to model in an emulation context. These limitations frame the context for interpreting the performance of the bandwidth estimation tool and highlight the trade-offs between simulation accuracy and practical feasibility. Chapter 5 discusses how these limitations impacted our experiments.

3.2 Bandwidth Estimation Tool

With the emulation environment and traffic generator chosen, we next describe the design of the bandwidth estimation tool itself. Providing real-time network metrics by passively capturing traffic, the tool extracts network parameters such as round-trip time (RTT) and throughput using libpcap for packet capture and pnet for packet parsing.

In this section, we outline the core components of the tool, while Chapter 4 further expands upon the specific implementation details.

3.2.1 Design and Architecture

There are four main components in the tool, each responsible for a distinct function:

1. **Packet Capturing Module**: At the foundational level, the packet capturing module functions as a dedicated producer, continuously monitoring network traffic via the library libpcap (The Tcpdump Group, 2025). To mitigate the risks of packet loss during large bursts when the incoming rate exceeds processing

capacity, packets are queued and only discarded when the queue is saturated. This mechanism ensures that measurements are retained, thereby preserving the fidelity of the network metrics.

2. **Packet Processing Pipeline**: This module acts as the central hub by processing the captured packet stream, handling responses from the thread pool (such as timeouts during RPC calls), and triggering the publication of measurement data. This module performs TCP stream analysis and generates data points for subsequent analysis.

3. **Data Analysis Module**: Although currently integrated within the processing pipeline, the data analysis module has been written with future decoupling in mind. It accepts packet data from the packet processor, such as packet gaps and RTTs, and stores it until a bandwidth estimate is requested.

4. **Inter-Device Communication Setup**: Implements a gRPC client-server architecture where each tool instance runs a gRPC server and a client. This setup enables inter-node communication for exchanging link-state information, coordinating health checks (via hello requests), and initiating bandwidth tests using tools like iperf3 or pathload. Since each node is also supposed to be a Kubernetes node, the tool needs to keep track of which nodes we care about and which we don't. This is done by keeping track of sent and received hello requests broadcast during the startup phase.

3.2.2 Measurement window

The tool's operation is governed by a configuration parameter that sets the duration for data collection before forwarding it via the gRPC thread to the Kubernetes controller. This interval directly affects the accuracy of the bandwidth estimation. For example, a 1-second interval estimates the average available bandwidth over that brief period, whereas a 10-minute interval provides a longer-term average that might not reflect real-time fluctuations. An interval that is too short may yield insufficient data points and be overly influenced by outliers, while an excessively long interval could mask current network conditions. Therefore, selecting a fitting middle-ground interval is needed to provide accurate measurements.

3.3 Data Collection

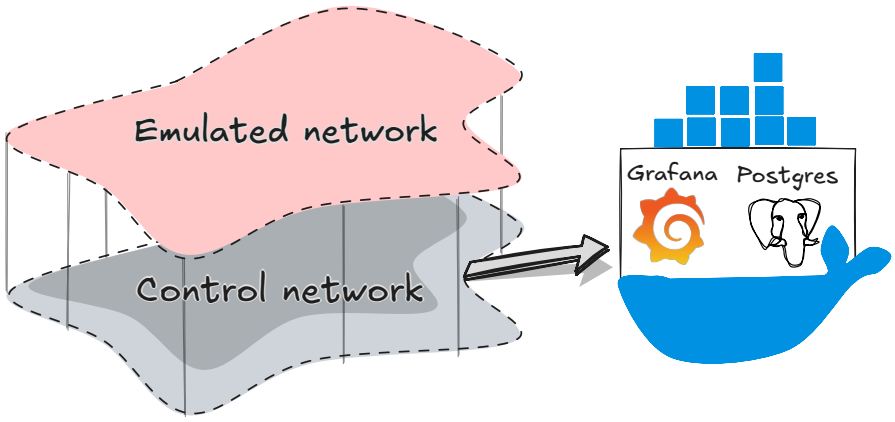

Figure 3.2: Data collection setup: Docker containers running Grafana and PostgreSQL, receiving data from the host machine through the control network.

As a way of getting real-time visualizations of the collected data from the implemented tool, as well as storing the test data for later analysis, we mainly used three technologies: Protocol Buffers (Google LLC, 2025), PostgreSQL 17 (Group, 2025), and Grafana (Labs, 2025), to gather measurements, store results, and view the data in real-time. This enabled us to continuously observe and refine the bandwidth estimation tool with low network overhead.

We considered using a representational state transfer (REST) API with JSON (Bray, 2017) payloads or using message brokers like MQTT ("MQTT Version 5.0", 2019), as seen in related work (Vatne, 2022). However, JSON carries a higher overhead in message size due to the cost of verbose text encoding and parsing. Since our goal is to provide real-time updates, potentially over low-bandwidth links, the extra overhead of JSON could be significant. Protobuf, on the other hand, encodes data in a very compact form. For example, a bandwidth report containing two IP addresses, a timestamp, and an ABW value might be on the order of only tens of bytes in the protobuf binary format. In contrast, a JSON equivalent could be a few hundred characters long.

Perhaps the most compact way of transmitting data would be to serialize raw Rust structs using a format such as `bincode` (Overby et al., 2025). However, this would force the receiving end to deserialize using Rust. One of the main benefits of using protobuf, other than providing compact messages, is that it works across languages without relying on string-based parsing. This works by giving each programming language a way to

generate code based on the `.proto` files, which describe the message structure. So, for instance, in Rust, we can use the library *prost* or *protobuf-rs* to generate code for serializing and deserializing protobuf messages, which can then be sent to other clients sharing the same .proto file or parts of it.

We can use gRPC to enable API-like behavior, such as requesting data or pushing changes. gRPC is built upon protobuf and can act as a partial replacement for typical HTML-based APIs. These can be easier to debug at the human level, but potentially introduce more network load and complicate real-time processing.

On the data storage side, PostgreSQL was selected because it is open source, supports time-series data using the TimescaleDB extension (Timescale, 2025), and nicely integrates with Grafana. Its primary use was to facilitate data collection for later analysis. It formed the basis of the data visualizations in Chapter 5, but was not related to the tools' functionality.

Grafana was then used to visualize the growing dataset in real time. Grafana can render graphs showing how the data is structured by regularly querying the PostgreSQL database. This helped us reiterate the tool's approach to bandwidth estimation and visualize network properties such as latency and throughput early on to improve our intuition on these concepts. Additionally, it helped us save time by allowing us to restart experiments when uncaught errors occurred during startup.

A dedicated *control network* was added in CORE to avoid interfering with the emulated network's traffic. Each virtual node gained an additional interface connected to the host machine, where a small TCP server awaited incoming protobuf messages. This arrangement minimizes overhead on the emulated network by offloading monitoring traffic to a separate channel.

The server then inserted incoming measurements into the PostgreSQL database, which Grafana accessed for visualization. This design pattern allowed the impaired, emulated links to remain isolated from the additional traffic generated by sending large amounts of datapoints and link-state summaries to the data collection server.

3.3.1 Evaluation and Performance Metrics

To assess the effectiveness of the implemented tool, we conducted evaluations within a controlled emulation testbed. This environment allowed us to vary network conditions

and compare the tool's estimations against known ground truths. The following metrics were employed to evaluate the tool's performance:

Estimation Accuracy

Estimation accuracy measures how closely the ABW estimates align with the ground truth available bandwidth configured in the testbed. When examining accuracy, we examine the median error and the mean absolute error to determine the results' bias and the overall variability of the estimates depending on different environmental factors.

Responsiveness to Network changes

This metric evaluates the tool's ability to adapt to changes in network conditions, such as changes in cross traffic or bottleneck capacity. We introduced controlled changes in cross traffic and measured the observational ABW error, relative to the bottleneck capacity, to determine whether or not it managed to reflect these changes over time.

Robustness to Traffic Variability

Given that real-world networks often experience bursty and unpredictable traffic patterns, we tested the tool against various traffic patterns with different packet sizes to assess its robustness within more chaotic scenarios. The variability in estimation accuracy across these patterns indicates the tool's resilience to different traffic behaviors.

Initially, we planned to do performance testing, to analyze CPU and memory utilization, but we ended up focusing on the ABW estimation results instead, as this allowed us to spend more time testing different experiment permutations. By employing these metrics, we aim to provide a comprehensive evaluation of the tool, and in Chapter 5 we discuss the gathered results and show how the tool performed.

Chapter 4

Implementation

The bandwidth estimation tool (see Section 3.2 for design) was implemented in Rust to balance high performance, memory safety, and extensibility. Rust (edition 2021, Klabnik, 2018) offers low-level control and high efficiency akin to C/C++, with memory safety guarantees that reduce runtime errors. Additionally, Rust's modern concurrency model —via the Tokio framework (Tokio Contributors, 2025)—enables the application to handle many tasks asynchronously with minimal overhead. This capability helps the tool keep up with high-speed packet capture from the network interface card (NIC).

4.1 Overview

The tools' primary function is to observe existing traffic flows, compute available bandwidth in real time, and publish these insights via a gRPC-based interface. By avoiding heavy active probing, the tool is particularly suited for resource-constrained edge nodes with limited network capacity.

The implementation leverages several Rust libraries (crates) to achieve these goals for packet capture, analysis, concurrency, and network communication between nodes. The following sections describe the overall test setup, packet capturing methodology, data analysis pipeline, concurrency model, and how the tool communicates its findings to the Kubernetes Master node.

4.1.1 Libraries and Crates

Implementing the tool required leveraging several Rust libraries (crates) to handle packet capture, packet parsing, concurrency, and network communication. We highlight a few

of the most important crates and their roles in the system:

- **pcap (2.2.0)** provides packet capture capabilities by interfacing with the libpcap library. This allows the tool to sniff network traffic on an interface in promiscuous mode and capture raw Ethernet frames. Using the `pcap` crate provides C-level packet capture capability with the convenience of Rust bindings, avoiding the need to write a custom packet capture framework from scratch (Bowe et al., 2024).
- **pnet (0.35.0)** - A Rust-based packet parsing library used to dissect raw Ethernet frames into higher-level protocol structures. The tool uses pnet to parse Ethernet frames, IP packets, and transport layer segments (TCP/UDP) (pnet Contributors, 2024).
- **Tokio (1.44.2)** is a commonly used framework for Rust asynchronous (async) programming, offering lightweight *tasks* and async channels, which are used extensively in the tool's design, discussed in Section 4.5 (Tokio Contributors, 2025).
- **Tonic (0.13.0)** - A high-performance gRPC library for Rust built on the *Tokio stack* and uses the protocol buffers implementation *Prost*. It handles the tool's RPC functionality, networking, serialization, and async request handling (tonic Contributors, 2025).
- The **procfs (0.17.0)** crate offers a Rust interface to the Linux proc filesystem. It was used to read from `/proc/net` to obtain information about connection states, as detailed in Section 4.4 (Chin, 2024).
- **Serde (1.0.219)**, although not used directly, should also be mentioned, as it provides a serialization/deserialization framework used for just about everything from deserializing captured packets to reading TOML-configuration files (Serde Contributors, 2025).

4.2 Emulation Testbed Setup with CORE

In Chapter 3, we explained the choice of CORE as our emulator of choice, which allowed us to create network topologies with configurable link characteristics, providing a controlled and semi-realistic testbed for testing the implemented tool.

4.2.1 CORE Installation and Configuration

```
# Start the network listener
vcmd -c /tmp/pycore.1/node_n -- \
    network_listener -c /path/to/config.toml &
PIDS+=($!) # Store process ID

# Start mgen script
vcmd -c /tmp/pycore.1/node_n -- \
    mgen input /path/to/script.mgn output /path/to/log &
PIDS+=($!) # Store proces ID
```

Listing 1: Example script showing initialization of mgen and network_listener. PIDS are used later to stop the processes.

CORE was installed on the Linux distribution Ubuntu 22.04 and its required dependencies, including Python, protoc, and others, further specified in Appendix A. Appendix B.2 presents installation instructions for CORE 9.2.0 and MGEN 5.1.1.

We created network scenarios using the graphical user interface, then modified the XML files generated by CORE. Each virtual node in CORE runs as a separate Linux namespace, behaving like an independent host. We set link parameters for our tests, such as bandwidth, latency, and packet loss. The actual parameter values are specified in Section 5.1.

As part of the node setup, each node in the emulated network was configured to run the Rust-based bandwidth estimation tool on its default interface, monitoring traffic on that interface. This was done using automated shell scripts by sending startup commands through the CLI `vcmd` provided by CORE. Listing 1 shows an example of the usage of `vcmd`, and another bit of logic for storing process IDs for later termination. We did this to be able to control how the script termination was handled.

In addition to the primary network, we introduced a separate control network interface, which is easily added by enabling it in the GUI or changing the global configuration of CORE. By default, the control network interface is called `ctrl0`, which can be changed in the CORE configuration.

4.2.2 Traffic Generation Scenarios

We generated traffic flows between the CORE nodes using the MGEN traffic generator to drive the network and produce measurable events. Following the MGEN 5.1.1 documentation (Naval Research Labratory, 2021), we created a few simple MGEN scripts to generate reproducible, semi-randomized TCP flows and steady *periodic* UDP flows with different intensities.

Listing 2 shows an example of a typical script we used during testing. First, to accept UDP and TCP connections from other clients running MGEN, we tell it to *LISTEN* to specified ports. Then we start four UDP streams, each sending $20 \cdot 8192$ byte-sized MGEN messages each second to four other nodes. The following two lines specify two bursty TCP flows, sending $2 \cdot 60000$ byte bursts every second, lasting for two seconds, occurring randomly every 15 seconds on average.

```
LISTEN UDP 5321,5320,5221,5220
LISTEN TCP 6321,6320,6221,6220

ON 1 UDP DST 10.0.2.20/5120 PERIODIC [20 8192]
ON 2 UDP DST 10.0.2.21/5120 PERIODIC [20 8192]
ON 3 UDP DST 10.0.3.20/5120 PERIODIC [20 8192]
ON 4 UDP DST 10.0.3.21/5120 PERIODIC [20 8192]

ON 5 TCP DST 10.0.2.20/6120 BURST \
    [RANDOM 15.0 PERIODIC [2 60000] EXP 2.0] RETRY -1/5
ON 6 TCP DST 10.0.3.20/6120 BURST \
    [RANDOM 15.0 PERIODIC [2 60000] EXP 2.0] RETRY -1/5
```

Listing 2: An example of a typical MGEN script starting four UDP streams and two TCP streams, as well as listening on a range of ports.

We classified these streams into different types of traffic to emulate network behavior, where the background traffic would act as a more fluid-like traffic, giving a baseline available bandwidth to look for in the results. The TCP flows were added to test the estimator and introduce more chaotic behavior in the network. MGEN supports several different traffic patterns, including the ones in Listing 2, (e.g., `PERIODIC`, `BURST`) such as `POISSION` and `JITTER` (Naval Research Labratory, 2021). Since our experiments involved several nodes, each one supposed to send some amount of TCP traffic across the network,

we needed some way of making sure that they all didn't send bursts at the same time, causing large, periodic spikes in the network. Still, we wanted somewhat bursty traffic to test our bandwidth estimation tool, so we used the `RANDOM` mode for the `BURST` pattern on all the TCP flows. For instance, `RANDOM 15.0` causes MGEN to send bursts at an average interval of 15.0 seconds. For UDP traffic, we used the `PERIODIC` pattern for most of our experiments, and later we tried the `JITTER` pattern to test the impact of less regular cross traffic. In Section 5.2.1, we discuss the effects of these patterns in more detail.

All traffic was confined to the emulated topology within CORE, with the known link bandwidth acting as the ground truth for the maximum available bandwidth. Utilizing the gRPC API provided by CORE for event listening, we could fetch information about throughput on each link, giving us insight into the real-time ABW.

Note that in an actual deployment scenario, the traffic would be generated by applications running on the Kubernetes nodes, but using MGEN allowed us to have reproducible tests. Initially, the goal was to recreate the exact conditions for each test using MGEN's SEED option, which allows the seed to be set for patterns with uniform random distribution. However, we quickly realized that this would not result in an accurate reproduction of the gathered results, because emulations share CPU, memory, and I/O resources with the host machine, leading to contention and consequently timing noise, which impacts the packet inter-arrival times we measure.

4.3 Packet Capturing and Parsing

The first functional component of the bandwidth estimation tool we implemented was the packet capture module. This module passively listens on a network interface and captures packets for analysis. We utilized libpcap through the Rust pcap crate to achieve this. The tool opens the interface in promiscuous mode. It sets the snaplen option to 138 bytes to capture only the Ethernet, IP, and transport layer headers, as we don't need any application layer information. Figure 4.1 shows a rough representation of what a `snaplen` of 138 bytes might capture. A typical packet rarely contains this many bytes in header information, but to be safe, we assume the IP header and application layer header to be 60 bytes each. For instance, the maximum size of an IP header is 60 bytes when all options are used (Postel, 1981).

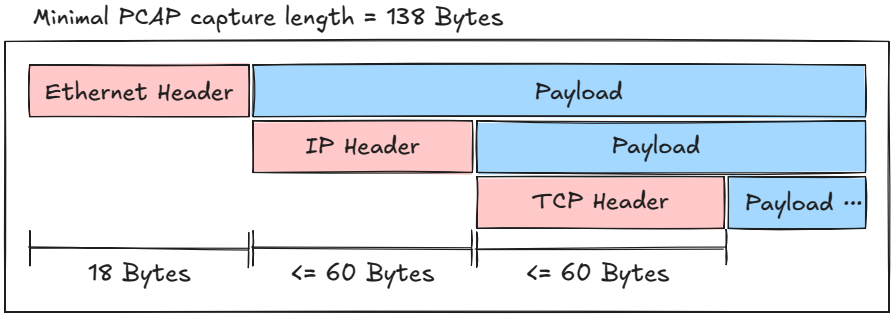

Figure 4.1: The minimal capture length to use when only capturing header information is 138 bytes (*This figure is not to scale!*)
. A combination of headers is unlikely to ever reach this size, as some IP/TCP options are rarely used.

Captured packets are obtained as raw byte buffers and metadata with additional information, such as timestamp and length. Rather than copying packet data around unnecessarily, we handle these using references whenever possible. Each captured packet is handed off to the analysis pipeline via a `tokio::sync::mspc` (multi-producer, single-consumer) channel as a reference to the "packet" (byte vector). This decoupling allows packet capture to run independently, making packets less likely to be dropped due to buffer overflow.

Once a raw packet is pulled from the capture channel in the analysis task, we perform zero-copy parsing of its headers and metadata using the `pnet` crate. The overall workflow is roughly outlined below:

1. The libpcap header gives us a `timeval` with either macro or nanosecond precision. We convert this into Rust's `SystemTime`.

2. We then wrap the raw byte slice into an `EthernetPacket` struct, extracting the source and destination MAC addresses and the Ethertype without copying the payload buffer.

3. With the payload buffer slice, we dispatch on Ethertype to parse either the IPv6 or IPv4 header, extracting the next-header field for IPv6 or the next-level-protocol for IPv4.

4. With the protocol header, we match the protocol on either TCP, UDP, ICMP, or "other", recorded by the protocol number. For TCP, we read the most important

fields, such as the seq/ack numbers, flags, and window size, and parse some of the option fields present.

5. For each packet, we record the packet direction by comparing the MACs to our interface MAC, and mark "intercepted" packets that neither originate nor terminate on our interface IP address. This allows intermediate nodes in a multihop wireless network to perform packet analysis, even though the node itself is not the TCP sender.

All of this is performed in the function `ParsedPacket::from_packet(&OwnedPacket, &PCAPMeta)`, which returns a fully typed `ParsedPacket` struct with no heap allocations or payload copies beyond the original buffer reference. To preserve memory, the returned struct discards most of the information contained in the header, and when passed to longer-term storage, the individual packets are stripped of stream-specific fields such as seq/ack numbers and IP addresses.

4.4 Packet Analysis Pipeline

Once packets are parsed into the relevant structure, the tool proceeds through a pipeline of analysis to assemble the high-level information needed for bandwidth estimation. Figure 4.2 illustrates the logical flow from an incoming packet to the computed probe gap metrics the estimator uses. The pipeline consists of several stages described below.

Stream Identification and Assembly: Each packet is associated with a flow identified by the TCP/UDP 5-tuple or 3-tuple for protocols without port information. We maintain an internal state for active flows, primarily to group packets in the same traffic stream. In the case of TCP, we keep track of seen sequence numbers to detect retransmissions or out-of-order arrivals. Initially, we experimented with implementing a complete TCP state machine, tracking connection establishments, termination, and sequence dynamics, but this proved unnecessarily complex for our needs. Instead, we adopted a simplified, minimal TCP stream reassembly approach. We do not track every TCP state or maintain a full in-order byte stream reconstruction. The main goal is to know which bytes have been delivered to each TCP connection and to be able to map ACKs to the data packets they acknowledge.

Still, to manage memory and avoid unbounded growth of per-flow storage, we periodically check for terminated flows to clean up their state. We leveraged a

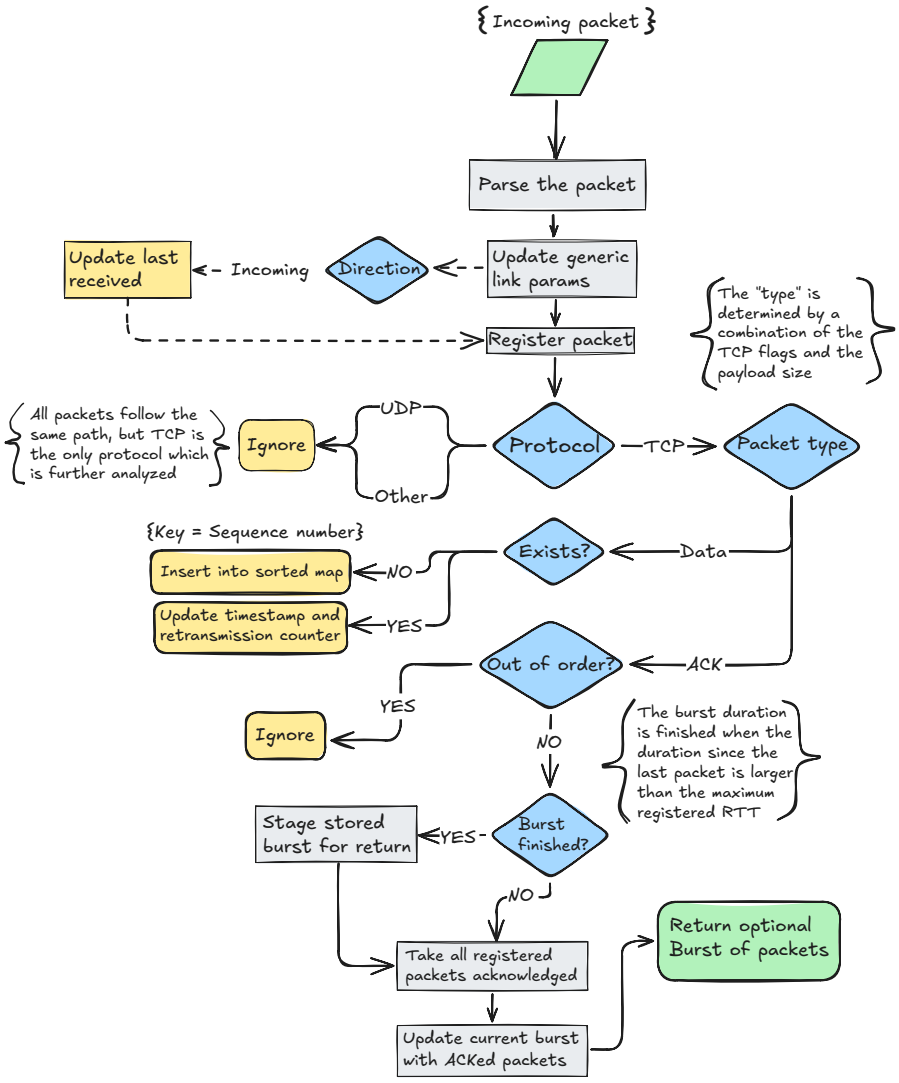

Figure 4.2: Flow chart of the logical flow from incoming packet to return a `Burst` if it has finished. The `Burst` is then processed into packet-gap datapoints for analysis.

separate thread that periodically reads kernel connection tables using the library `procfs`, specifically `/proc/net/tcp` and `/proc/net/udp`, to detect closed connections. When a connection is no longer present, such as after a FIN/FIN-ACK exchange or timeout, we free that flow's associated buffers and state.

ACK Handling and Burst Formation: The core of the passive bandwidth estimation logic relies on analyzing ACKs in TCP flows. In TCP, ACKs signal that one or more data segments have been successfully delivered. When the tool recognizes an ACK without a payload, it checks the ACK number against the registered sequence numbers and stores the ACKed packets in an intermediate struct. We call this struct a `Burst`, which essentially acts as a consecutive train of packets grouped by acknowledgement.

For each collection of ACKed packets, the following metrics are calculated:

- Total length (*len*): The combined payload sizes of each acknowledged packet.
- Gap in (*gin*): the gap between the previous ACKed collection's last packet and the current collection's first.
- Gap out (*gout* or *gack*): In the context of the probe gap model, the output gap is usually measured as the gap between two packets when they arrive at the receiver. However, we must use the acknowledgement gaps since we only have sender-side information.

One challenge we faced was deciding when a burst was "finished" and ready to be analyzed. ACKs arrive frequently in a live TCP flow, and a long burst of data might be partially acknowledged when the time since the last measurement is larger than the measurement window. The tool implements a delay mechanism to avoid splitting what conceptually is a single burst of back-to-back packets into smaller pieces. When an ACK arrives, we do not immediately output a `Burst` until we are reasonably sure the data burst is complete. We base this on the RTT of the flow; specifically, we wait until the time since the last captured data packet exceeds the maximum RTT observed for that flow. The intuition is that, in simple terms, TCP transfers packets back-to-back until the maximum flight-size is reached, then it waits until it receives an ACK before starting to transmit again.

As part of the filtering mechanism of the ABW estimation, a `Burst` is not returned before the time since the last registered packet is larger than the maximum registered RTT for that specific TCP stream. We chose to do this because we observed during testing that the measurements would get thrown off if the measurement interval landed within the TCP Slow-start phase, resulting in a higher concentration of high or low send rates, skewing the final ABW value.

Once a `Burst` is identified and considered finished, we record the gin, gout, and len and add it to a pool of datapoints for the represented link. When the measurement window is up, we use the recorded datapoints to estimate the available bandwidth.

4.5 Concurrency Model with Tokio

The tool's architecture leverages Rust's asynchronous programming model, with Tokio serving as the underlying runtime for task scheduling, timers, and inter-task communication. This approach enables the system to handle multiple concurrent operations efficiently without relying on OS-level threads for each component.

The implementation is organized into several asynchronous tasks coordinated through Tokio's primitives, primarily bounded multi-producer, single-consumer (MPSC) channels. These channels decouple the system's processing stages, allowing each task to operate at its own pace while maintaining thread safety.

- The capture task interfaces with `libpcap` to retrieve packets from a specified interface. Since the underlying packet capture API is a synchronous implementation in C, the task is executed using `tokio::task::spawn_blocking`, spawning a thread to avoid blocking the async runtime. Captured packets are immediately sent into a channel for downstream processing.

- The analysis task consumes packets from the capture channel and executes the parsing and burst analysis pipeline, as outlined in Section 4.4. It produces `Burst` records, which are temporarily stored until a bandwidth estimation is triggered.

- The communication task manages gRPC-based message exchange, handling outgoing link-state messages to Kubernetes (or any external data consumer), and coordination between nodes.

- The housekeeping task performs lightweight background maintenance, specifically doing operations requiring reading from disk, such as monitoring active connections via `/proc/net`, informing the analysis task which traffic streams should be retained or discarded.

To ensure that memory usage does not grow too large but still allows bursty network traffic, the bounded channels are configured to be large enough to accommodate short traffic bursts. If the analysis task cannot process packets fast enough, the channel may temporarily buffer packets, eventually dropping them when under sustained overload. High traffic scenarios generally increase the CPU usage of our tool, and when the average load increases above the processing capability of the machine, packets will be dropped.

To partially improve this issue, we could pass a Berkeley Packet Filter (BPF) to libpcap, restricting it to only capture TCP packets.

Within certain tasks, notably the communication task, it is necessary to await multiple asynchronous events concurrently. For this, Tokio's `select!` macro is used, similar to the `select` system call in C (Kerrisk, 2025). This allows the task to respond dynamically to diverse events, such as:

- Receiving a heartbeat message to check node liveness,
- sending a link state update to the scheduler,
- or handling shutdown signals for graceful termination.

A minimal example demonstrating `select!` usage is provided in Listing 3, showing parts of how the analyzer task works.

```
tokio::select! {
    Some(cap_ev) = self.packet_stream.recv() => {
        match cap_ev { // Capture event
            CapEvent::Packet(packet) => {
                // Pass the packet into the analyzer pipeline
            }
            ...
            CapEvent::Error(e) => {
                // Handle potential errors
            }
        }
    },
    _ = measurement_window.tick() => {
        // Perform the bandwidth measurement
    },
    ...
}
```

Listing 3: Example usage of the tokio::select! macro from the data consumer/analysis task, listening on incoming packets and other events.

This concurrency pattern allows a single task to respond efficiently to whichever event happens first, without blocking on one and missing others. The net effect is that our tool's main loop is event-driven, processing bursts as soon as they arrive, sending updates at intervals, and reacting to control messages.

```
pub enum CapEvent {
    Packet(OwnedPacket),
    IperfResponse(IperfResponse),
    Protobuf(PbfMsg),
    PathloadResponse(String),
    PingResponse(Result<Duration, SurgeError>),
    Error(AnyError),
}
```

Listing 4: Event enumeration for passing events between tasks. The main use for this specific enum is to send captured packets, but it also includes variants for ping responses and potential active measurements.

4.5.1 Channel message design

We defined internal enumeration types for channel message payloads to carry different events with associated data. For example, we have an enum for events to be read by the analysis thread containing various information, such as captured packets and active probing tool results. An example of such an enum can be seen in listing 4, showing parts of the core data consumer loop.

This allowed the analyzer task to handle various types of inputs in a single stream. A similar enumeration could also be used for control messages. Still, as of now, the tool handles shutdown by having the main thread listen on interrupts, killing threads when receiving one. Spawning a Tokio task using tokio::spawn causes very little overhead since a task uses a single heap allocation and roughly 64 bytes of memory. This means we can afford a dedicated task for each key activity and spawn sub-tasks when necessary to handle timeouts. However, we had to be mindful of how Tokio handles concurrency. A task differs from a dedicated thread as it is not scheduled by the OS scheduler but by Tokio's async reactor. This means that by spawning a task, unless using the `tokio::spawn_blocking` call, we spawn what is typically known as a "green thread", which handles concurrent async operations cooperatively. Still, if heavy computations are done on one of these threads, tasks may not yield to other tasks frequently enough, which might overload the reactor. For instance, the available bandwidth regression analysis is fast when done in small chunks, but if it were something heavier or done on larger chunks of data, we would consider moving it to a dedicated thread pool to keep the event loop responsive.

4.5.2 gRPC Communication via Prost and Tonic

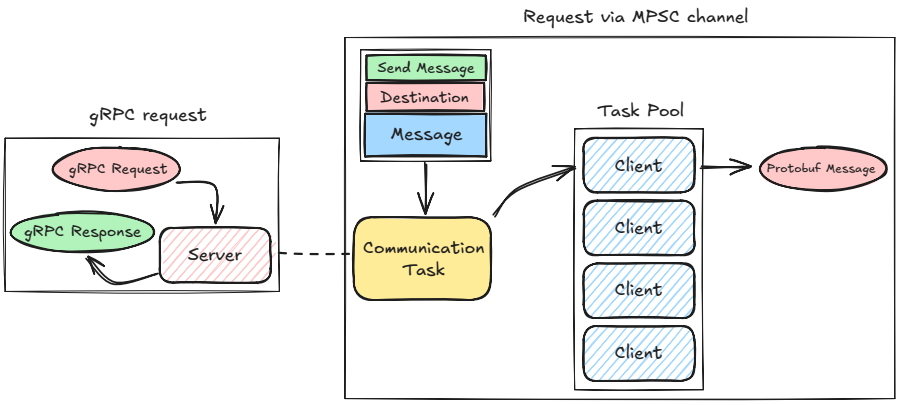

Figure 4.3: This figure shows how the communication tasks handle incoming requests from the MPSC channel and gRPC requests.

Once the tool computes an estimate of the available bandwidth and other metrics, it needs to communicate these results to external systems, such as the Kubernetes scheduler and the data collection server. As explained in Section 3.3, we chose gRPC for this communication, using Tonic, Prost, and Serde for data serialization.

We created a `.proto` file for the service using *proto 3* (Authors, 2025), defining the RPC calls and message structures. Using Prost, we could generate Rust structures from these message definitions and use Tonic to implement the client and server stubs. In our architecture, there are two main ways to communicate ABW measurements to a receiver: via client-side streaming (gRPC Authors, 2025), or through server-side streaming. The main difference between these two is that client-side streaming does not require the ABW data consumer to know each of the clients' IP addresses. Server-side streaming allows the tool to receive subscription requests from external sources, which will then return a stream of ABW measurements.

As mentioned in the scope (1.3), we did not consider security during the development. Consequently, the API can be accessed by any external source that implements the client interface. In a production setting, using Transport Layer Security (TLS) should be considered (Rescorla, 2018).

4.6 Available Bandwidth Estimation Algorithm

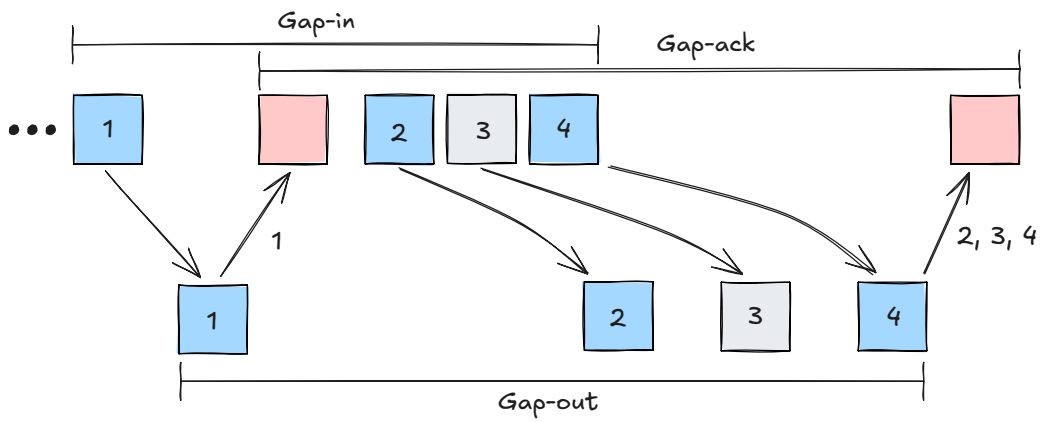

Figure 4.4: Gap-in is the gap between TCP packets sent from the estimator. Gap-out is the gap between packets when received, while Gap-ack is the gap between acknowledgements when the sender receives them.

This tool's available bandwidth estimation algorithm is conceptually grounded in the probe gap model by passively observing TCP flows following the methodology presented by Khangura and Fidler (2017). This section answers research question 2, and in Chapter 5 we verify that it manages to estimate ABW.

The idea is to interpret TCP packet bursts as natural probes. When TCP transmits a large segment, this is typically fragmented into smaller MTU-sized packets, which are sent back-to-back at an increasing rate until a congestion event occurs. This behavior is beneficial for us since the inter-arrival times of packets only start growing when the sending rate is larger than the available bandwidth. However, since we are measuring at the sender side of the TCP connection, we do not have access to the typical output gap, as one would in client/server-based estimation techniques. Therefore, we use the proposed ACK-gap (gack) method, using ACK gaps in place of output gaps.

Section 4.4 describes how we get these measurements. Still, we also need to consider that a single ACK can acknowledge several packets in many cases, especially in congested scenarios. So for each $\{gin, gack\ len\}$ measurement, we divide the derived values by the number of ACKed packets.

From these three metrics, we can calculate two values, later used for the regression

to derive the available bandwidth estimate:

1. The data rate in bits per second: $\frac{len}{gin}$.
2. The relationship between gin and $gout$: $\frac{gout}{gin}$.

With these two values plotted against each other, we can perform simple linear regression to find where the regression line intercepts $y = 1$, indicating the available bandwidth. We performed this analysis in the same way as Khangura and Fidler (2017), by setting a maximum value for gin as the average $gout$, corresponding to the smallest 10% gin values, limiting the gin to instances when the send rate would be more likely to fit the upward trending part of the gap response curve. This filter can be modified to better fit a given scenario. A possible improvement to this technique could be to implement a dynamic threshold based on the observed data.

4.6.1 Filtering of the probe gap measurements

Unsurprisingly, we found that raw measurements can be noisy. For example, small bursts might show erratic gap ratios due to high timing anomalies or cross-traffic burstiness. We incorporated filters to minimize the effect of these issues and avoid some of the observed problems:

1. If the number of data points when calculating the ABW is very low, for instance if no substantial TCP burst has been fired, we discard the calculated value and clean out the registered data points to avoid skewing newly gathered observations.
2. During testing, we observed some anomalies in the inter-arrival times of packets, which we further discuss in Section 5.2.2. We filter out all values where $gin < MSS/C$ or $gout < MSS/C$, where C is the outgoing link's physical capacity. However, this introduces a new value that needs to be known during runtime, which might be problematic in a real scenario due to the additional configuration needs.
3. As we described in Section 4.4, we wait until a burst is finished before allowing the data to be used for analysis. This adds an implicit filter since we only calculate packet gaps between ACKed packets within one `Burst`. So for instance if we have a burst B_n with ACKed packets A_n, then for the bursts: $B_1[A_1, A_2]$ and $B_2[A_3, A_4]$ we count only the pairs $(A_1, A_2), (A_3, A_4)$ and not (A_2, A_3) for gap calculations.

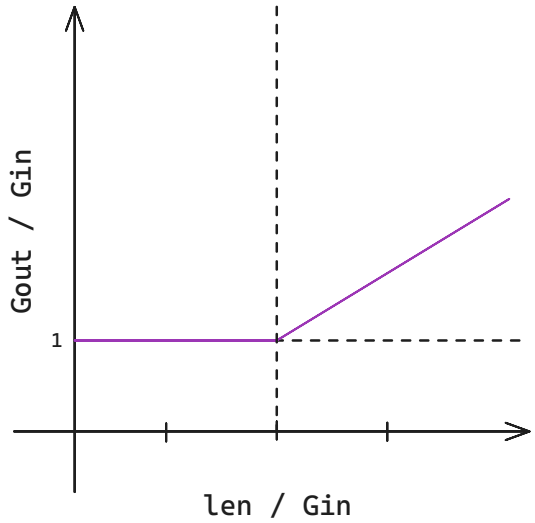

Figure 4.5: The gap response curve. The point at which *gout* exceeds *gin* is the available bandwidth, marked at the intersection of the horizontal and vertical dotted lines.

4.6.2 Regression algorithm

We implemented two methods of regression: *Robust least squares regression* and *simple linear regression*. The reason why we tested robust regression was to see if it could help overcome issues with outliers and improve the accuracy. However, as later discussed in Section 5.5, we discovered that the effect this had was to essentially strengthen the patterns present in the data, which did not necessarily improve the estimation accuracy.

4.6.3 Limitations and considerations

While the implemented ABW estimation algorithm covers the basic required functionality, there are some limitations and opportunities for further tuning.

Reliance on TCP traffic

Our approach depends on observing TCP flows and the relationship between the send rate and the acknowledgement gaps. If a link has mostly UDP or non-TCP traffic, the current implementation would not have any way to estimate available bandwidth. Fronteddu et al. (2022) describes a method of passive ABW estimation based on receiver-side UDP traffic analysis, but it only works in networks where the sender uses a modified protocol exclusively sending packets back-to-back.

Parameter tuning

Some thresholds, like what constitutes a "small burst" to ignore, or the fraction of the smallest gaps to average for the baseline, are currently fixed or empirically chosen. These might need further tuning or adaptation. For example, the 10% smallest `gout` selection was based upon the heuristic from Khangura and Fidler (2017), and does not necessarily fit all scenarios. In addition, the RTT-based burst cutoff might need adjustments, as it might cause the loss of some datapoints with larger gaps caused by congestion to be lost.

Impact of congestion on gathered data

When a path becomes congested, two independent mechanisms combine to reduce the number of acknowledgements, reducing the amount of ACK-gap samples visible to the bandwidth estimation tool. First, because each TCP ACK is triggered by the arrival of data, a highly congested path, where packet loss is frequent, naturally produces fewer valid ACKs for the estimation. Second, congestion on the path prompts receivers, proxies, or queue disciplines to batch or discard acknowledgements. Sooriyabandara et al. (2002) (RFC3449) describes ACK filtering/decimation that intentionally lets one cumulative ACK cover many data segments to relieve upstream pressure. NIC-level receive-side coalescing and interrupt moderation further aggregate multiple arriving segments before the TCP stack runs, so the receiver emits one ACK that acknowledges three or more packets at once. Because the implemented algorithm relies on the time gaps between successive ACK sequences to infer ABW, every instance of delayed ACK batching, ACK filtering, or packet loss removes potential measurement points.

Chapter 5

Results

The following chapter presents the results gathered from testing the tool outlined in chapters 3 and 4, discussing the degree to which the tool managed to use passively gathered data to estimate ABW.

5.1 Experimental Setup

The evaluation was conducted in a controlled network environment to assess the bandwidth estimation tool under various conditions. Each bottleneck link was configured with a fixed capacity, and cross-traffic was generated using MGEN. By varying the cross-traffic patterns and network parameters across experiments, we could emulate a range of network conditions and stress-test the estimator in less optimal scenarios.

We deployed the estimator on each edge node, where it passively collected packet gap measurements from ongoing TCP flows. For ground truth comparison, we obtained the actual throughput on each link via CORE's gRPC API. We computed the true available bandwidth by subtracting the known load from the link's capacity. This allowed us to directly compare the tool's reported ABW against the known ABW over time, thereby evaluating accuracy. The ground truth throughput measurements arrived around every 3 seconds, so to be able to compare this to the estimated values, following a different timescale, we did the following calculations:

1. For each bottleneck link (the experiments' narrow link), we calculate the moving average throughput matching the experiments' measurement window.

2. Then, for each ABW timeseries we find the corresponding two bottleneck links'

throughput measurements, choose the narrow link (the one with the highest throughput) for the two closest measurements, and interpolate the value between to get the value closest representing the data the estimate is based upon.

3. From the interpolated ground truth measurements, we calculate the signed error, and then the error in % of the bottleneck capacity for comparison.

A total of 11 experiments (including scenario variants) were defined to explore factors such as traffic burstiness, cross-traffic characteristics, and link capacity differences. Experiment 1 was a baseline scenario with a moderate bottleneck capacity and no added latency jitter or random packet loss. In this baseline, cross-traffic was configured to load each bottleneck with 2 Mbit/s. The topology for all experiments remained the same as illustrated in Figure 3.1 from Chapter 3, and a sample MGEN configuration is provided in Listing 2 for reference. The scripts are configured such that each node initiates UDP flows to nodes on separate subnets (e.g., 10.0.3.21 sends traffic to subnet 10.0.2.0/24 and 10.0.1.0/24 peers).

Table 5.1 summarizes the experiment configurations. For instance, in Experiment 2, we reduced the bottleneck link capacity and introduced latency, jitter, and random packet loss to simulate a more challenging environment. Some experiments employed "fluid" cross-traffic, meaning we broke the same cross-traffic volume into many small packets instead of a few large packets. The goal of these fluid traffic variants was to see if a more continuous, finer-grained stream of packets would improve the estimator's accuracy compared to the burstier traffic pattern of the normal scenarios.

We also tested scenarios with different bottleneck capacities. Notably, in experiments 4 and 5, the bottleneck link capacity was set to only 1 Mbit/s, with a capacity of 4 Mbit/s for the outgoing links, and we reduced the TCP burst size by a factor of 10 to simulate low-traffic scenarios. These two experiments represented more volatile conditions where the estimator had fewer packets to observe per measurement window, allowing us to examine the performance in sparse conditions.

Finally, some experiments were designed to observe the estimator's behavior over time under different cross-traffic loads. In experiments 6, 7, and 8, the cross-traffic intensity and pattern were varied during the run to showcase how the estimated bandwidth tracked changes in the ABW over time. For instance, in experiment 7, we used the PERIODIC traffic pattern in MGEN to simulate highly regular traffic, while in

Experiment	Traffic Pattern combination	OWD	Jitter	Bottleneck/ Upstream Capacity	Loss	Packet buffer
exp1	Periodic+bursty	12 ms	0 ms	5/10 Mbit/s	0 %	75
exp1_fluid	Periodic+fluid	12 ms	0 ms	5/10 Mbit/s	0 %	75
exp1_jitter	Jitter+fluid	12 ms	0 ms	5/10 Mbit/s	0 %	75
exp2	Peridoic+bursty	22 ms	2 ms	3/5 Mbit/s	2 %	30
exp2_fluid	Periodic+fluid	22 ms	2 ms	3/5 Mbit/s	2 %	30
exp3	Periodic+fluid	12 ms	0 ms	5/10 Mbit/s	0 %	75
exp4	Periodic+fluid	42 ms	8 ms	1/4 Mbit/s	4 %	10
exp5	Jitter+fluid	42 ms	8 ms	1/4 Mbit/s	4 %	10
exp6	Periodic+fluid	12 ms	0 ms	5/10 Mbit/s	0 %	75
exp7	Varied	12 ms	0 ms	5/10 Mbit/s	0 %	75
exp8	Jitter+varied	12 ms	1 ms	7/10 Mbit/s	0 %	75

Table 5.1: Experiment configurations. Each parameter is based on each path having two bottleneck links with compounding effects, but with the same bottleneck capacity.

experiment 8, we introduced randomness in traffic timing and narrowed the gap between the bottleneck capacities and the outgoing link capacities. In all experiments, we used a measurement window of 35 seconds, with TCP bursts initiated on average every 15 seconds. This window ensured that each ABW estimate was based on at least one TCP burst, providing a mix of high-activity and low-activity periods for each measurement.

5.2 General Observations

Before presenting and discussing the estimation results, we highlight some general observations from the experiments. In particular, we explain how the cross-traffic pattern timing influenced the experiments, and discuss the timestamping anomaly first introduced in Section 4.6.1.

5.2.1 Cross-traffic Timing

In a highly periodic cross-traffic scenario, as used in Experiment 1, traffic was sent in bursts of large packets sent back to back every second from each node. Even if the average cross-traffic load was below the outgoing link capacity, due to how we synchronized these bursts, starting all streams simultaneously, they consistently overflowed the packet buffer, leading to a large amount of packet loss before entering the bottleneck. This resulted in the observed throughput on the bottleneck links being significantly lower than expected. This effect was partly mitigated when we configured experiments with

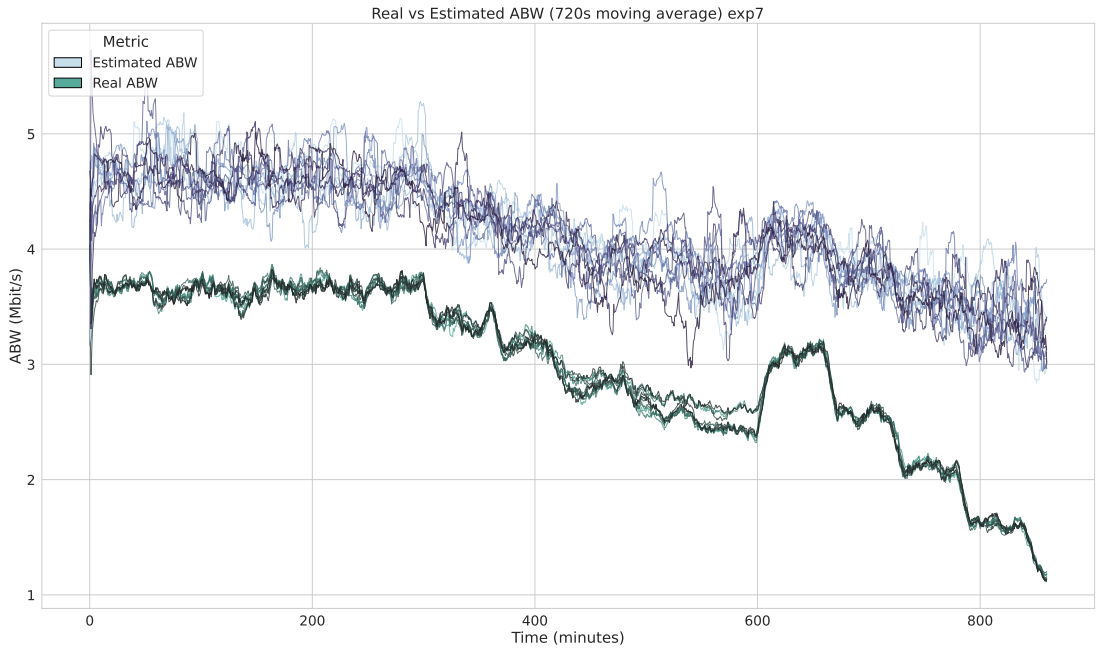

Figure 5.1: Ground truth ABW plotted against the estimated ABW for experiment 7, showing how the algorithm reacted to ABW changes over time.

jitter in the packet transmissions because the packets were more evenly distributed. For example, by randomizing each flow's inter-packet intervals by ±50% variation around the mean, the combined traffic from all nodes becomes smoother, avoiding momentary queue overload. We used this approach in Experiment 8, which was a modification of Experiment 7, introducing a slight jittered timing.

An example of this behavior can be seen in Figure 5.1, which is a plot of the ground truth ABW (green tint) and the estimated ABW (blue tint) for all sender/receiver node pairs. The figure is divided into three five-hour sections, each configured with the same cross-traffic intensities, increasing every hour, then resetting each 5 hours. It clearly shows that the real throughput on the bottleneck links was not equal throughout the experiment, as configured. It also highlights a consistent overestimation of the available bandwidth, which we discuss in more detail in Section 5.3.1.

Although not all experiments behaved as expected, these runs still provided valuable insights. In general, we observed that highly bursty traffic can lead to a split distribution of measurement data where some reflect an uncongested state, while others capture a heavily congested moment. This split dataset poses challenges for any estimation model, as there is a lack of a general trend indicating the rate at which congestion occurs. Using finer-grained traffic tended to mitigate this by ensuring a more continuous utilization of

the bottleneck link, thereby providing more consistent measurements due to the concept outlined in Section 2.5.3. For instance, in the fluid variation of experiment 1, a larger number of the gathered data showed signs of congestion. While this did not improve the estimation error, the nonfluid variant was not able to capture any cross-traffic at all, suggesting that smoothing out the cross-traffic helped capture higher fidelity estimates.

It is important to keep in mind that the issue we discussed here impacted all our experiments and was mainly due to a misunderstanding of how the MGEN patterns interacted with each node's outgoing link. We configured each outgoing link with a 0-packet buffer and a capacity significantly higher than the bottleneck links, but we did not consider what would happen when all UDP streams were started simultaneously. Introducing packet timing jitter and reducing packet sizes helped mitigate this effect, but it did not eliminate it entirely.

5.2.2 Timestamping Anomaly

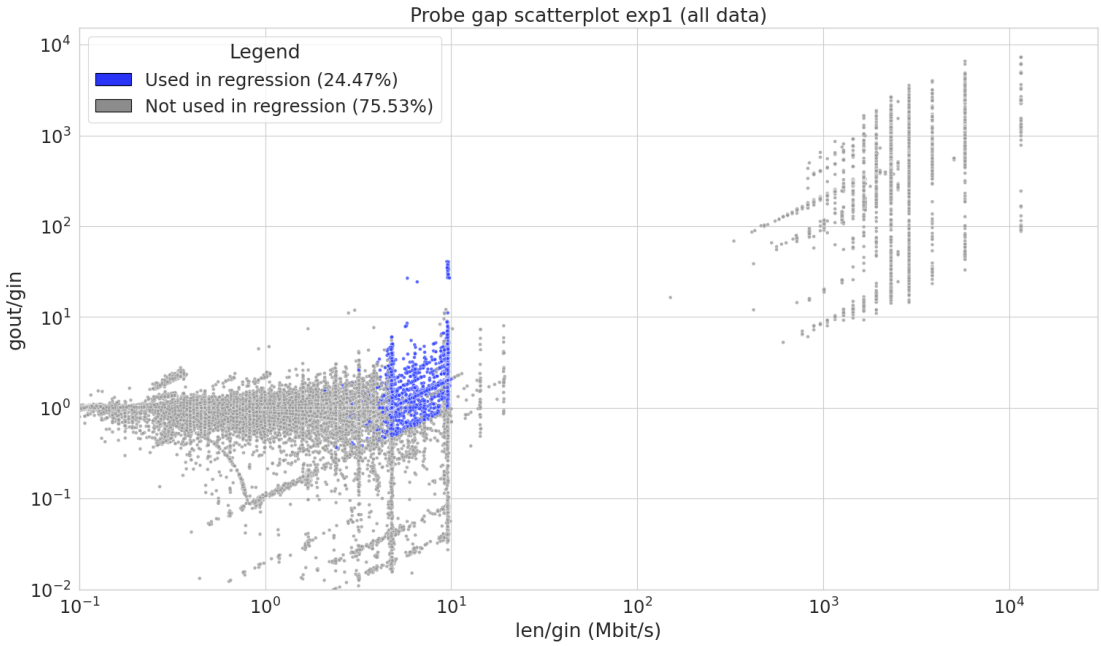

Figure 5.2: All the gathered probe gap measurements from experiment 1, including anomalies.

During the experiments, we observed an anomaly in the raw packet timing. Occasionally, some of the datapoints were registered with implausibly short input and output gaps, showing a far higher transmission rate than what the network link capacities should allow. For example, in Experiment 1, no two packets should ever be observed back-to-back faster than at around a 10 Mbit/s rate. Yet we saw some reported

gaps corresponding to the NIC's capacity on the order of gigabits per second. These anomalous readings resulted in error rates on the order of 10% of the *NIC capacity*. In other words, they appeared as outliers around 10000% off, relative to the true 5 Mbit/s bottleneck capacity in that experiment.

We suspect this issue is due to how packets are timestamped in the NIC. It appears that under certain conditions, packets were being timestamped in batches rather than individually. A possible reason for this happening is that we use NIC packet timestamping for the packet capture, and since all experiments were done in an emulation, the NIC is tasked with timestamping for all the nodes in the network, on top of its regular networking tasks. As a result, during large traffic bursts, packets were likely enqueued, yielding artificially small gap measurements.

To avoid these anomalies skewing the data, we applied a pre-processing filter to eliminate impossible values. Specifically, as mentioned in Section 4.6.1, we filter out any samples where the implied input or output gap corresponds to a throughput higher than the physical link capacity. Figure 5.2 shows all the packet gap measurements from Experiment 1, including the anomalies.

5.3 Accuracy

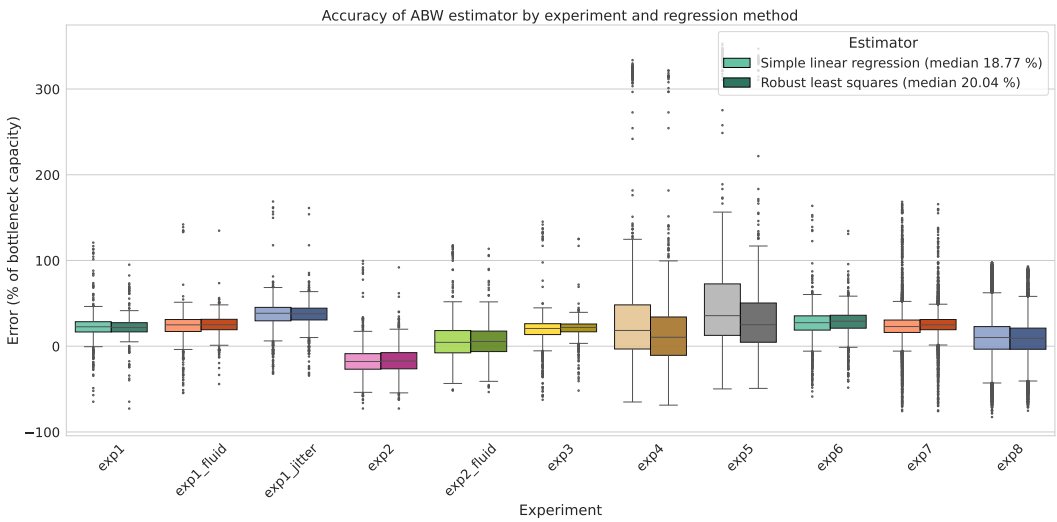

Figure 5.3: Boxplot showing the error distribution as a percentage of the bottleneck capacity for all experiments. Each boxplot is plotted twice, showing the error when robust and simple regression are used.

Having examined the general behavior of the data, we now turn to the core performance of the estimation tool: its accuracy in reporting available bandwidth. We

Experiment	**Simple** (%)	**Robust** (%)
exp1	22.69	21.79
exp1_fluid	24.96	25.19
exp1_jitter	38.42	37.90
exp2	-17.84	-17.31
exp2_fluid	4.52	5.54
exp3	20.70	21.87
exp4	18.37	10.46
exp5	35.66	25.09
exp6	27.47	29.21
exp7	22.97	24.93
exp8	10.17	9.26

Table 5.2: Median error relative to bottleneck capacity by experiment and regression method.

evaluate accuracy by comparing the estimator's reported ABW to the ground truth ABW and calculating the error as a percentage of the true bottleneck capacity. We retain the sign of the error to highlight any systematic, directional bias.

Table 5.2 shows the median error observed in each experiment for both regression models we tested. For a visual perspective, Figure 5.3 presents boxplots showing the distribution of errors for all experiments, with each experiment plotted for both regression models. As an example, an error of +20% on a 5 Mbit/s bottleneck indicates an over-prediction of 1 Mbit/s, whereas an error of -20% means it under-estimated the ABW by the same amount. As a baseline, we expected to see relatively large errors given the complexity of passive estimation, but some of the results were still surprising.

5.3.1 Systematic over-prediction

One immediate pattern we recognized is that the estimator consistently overshot the ground-truth ABW in almost every scenario, with a couple of notable exceptions discussed later. The median errors in Table 5.2 show this clearly, and the boxplots in Figure 5.3 reinforce that most distributions are centered above zero. The magnitude of overestimation is about 20-30% in most cases, but some have errors closer to 40%. This outcome was initially surprising as we expected it to yield estimates closer to 10% as is reported by Khangura and Fidler (2017), the author of the algorithm we use.

The ground truth ABW in our comparison is derived from the measured cross-traffic from CORE, as explained in Section 5.1. Importantly, this ground truth includes *all* traffic on the bottlenecks, including the TCP bursts used by the estimator in place of

probe traffic. The algorithm treats all this additional traffic the same way as an active probing tool does, ignoring the consumed ABW.

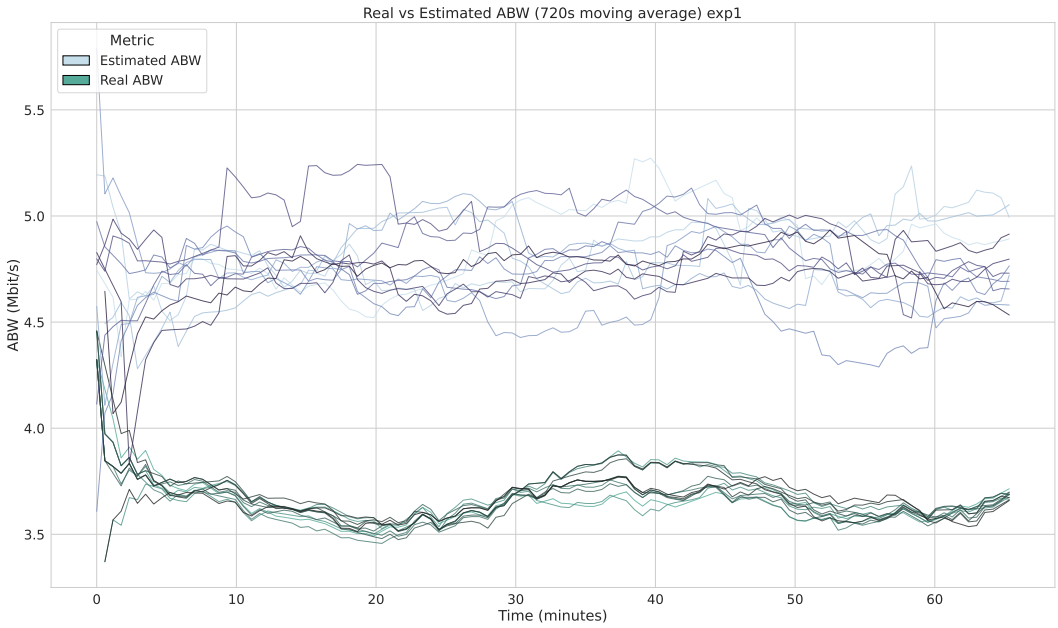

Figure 5.4: Ground truth ABW plotted against estimated ABW for experiment 1, showing the consistent overestimation.

In practical terms, this leads to an overestimation. For example, in Experiment 1, plotted in 5.4, each node periodically sends short TCP bursts lasting for around 2 seconds every 15 seconds on average. Across all nodes, these bursts amount to a bit over 1 Mbit/s of average load on the bottleneck. However, this is only when using a long averaging timescale. The average number of bursts firing during a single second is 1.07, so while burst collisions are likely to occur, most of the short reductions in ABW end up being ignored. This leads to a cumulative effect of the ignored ABW consumption by the TCP stream, and the likelihood that the TCP burst is fired at a point in time where no other burst is firing, skewing the error upwards. The effect of this bias is that the tool often reports an ABW closer to the ABW, only accounting for the less bursty traffic.

It is worth noting that this bias might be less pronounced in scenarios where traffic is less regular. It is highly unlikely to encounter a network where six nodes at such frequent intervals send large TCP bursts like this. If the measurement window were longer and bursts were less frequent, the bias would, in turn, be lower, as the average load would be significantly lower. A 2 Mbit burst over a 35-second measurement window would, in turn, lower the average ABW by around 60Kbit/s, significantly less than what is seen

in these experiments.

5.3.2 Impact of Regression Basis and Data Volume

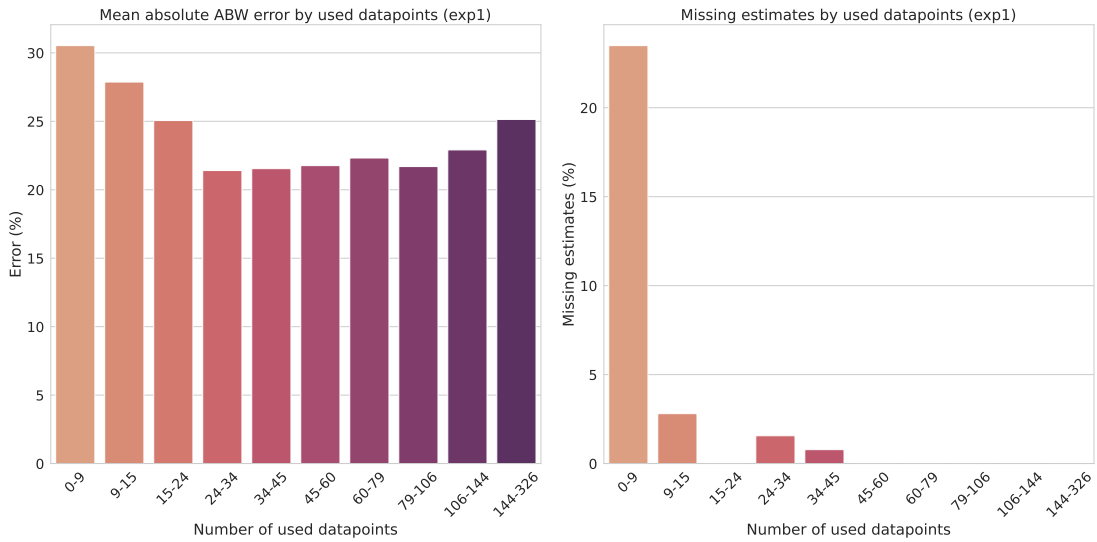

Figure 5.5: This figure shows the mean absolute ABW estimation error divided into buckets based on the number of measurements within each range of used datapoints. To the right is the number of estimates that yielded negative values or values higher than the physical capacity of the nearest link.

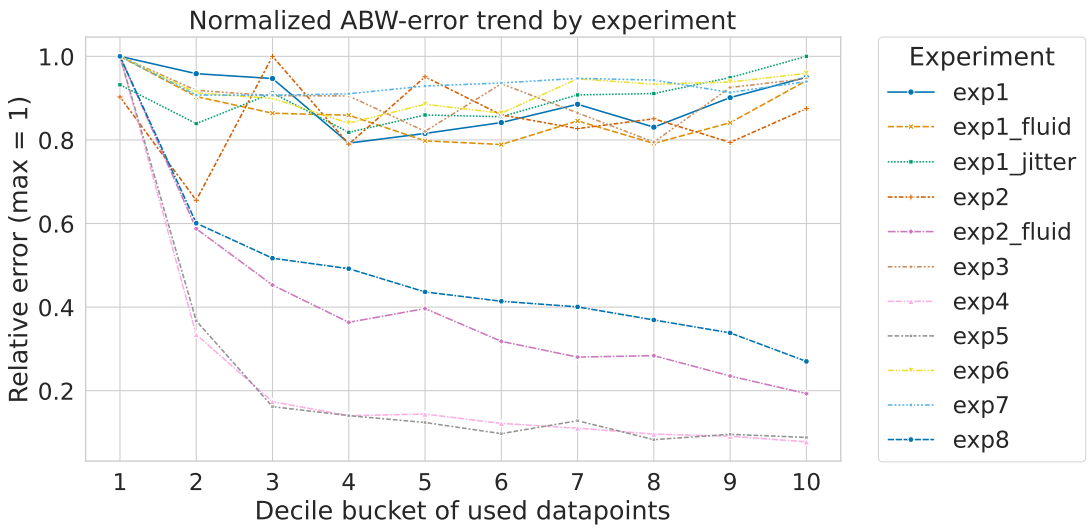

Figure 5.6: Plot showing the general trend of how the error evolves, depending on the number of datapoints used during regression.

The systematic bias above does not tell the whole story regarding accuracy. The magnitude of error still varied across the different experiments. In particular, Experiment 2 and its fluid-traffic variant stand out as exceptions to the overestimation pattern seen elsewhere. One yielded a large negative error and the other a very small positive

error (-17.84% and +4.52%). This prompted a deeper look at how the estimator's linear regression was functioning under various conditions, especially how the choice of datapoints (regression basis) affected the outcome.

One factor is the number of usable packet gap samples and their distribution available for the tool within each measurement window. When there are few packet pair samples to fit the regression model to, the estimation becomes more sensitive to each individual sample. In scenarios with more data points, the estimates yielded are more stable and tend to consistently overshoot by a similar margin. But in scenarios with very sparse data, like Experiments 4 and 5, which had low TCP throughput by design, the results were more erratic and dependent on which packets were left in by the filtration mechanism.

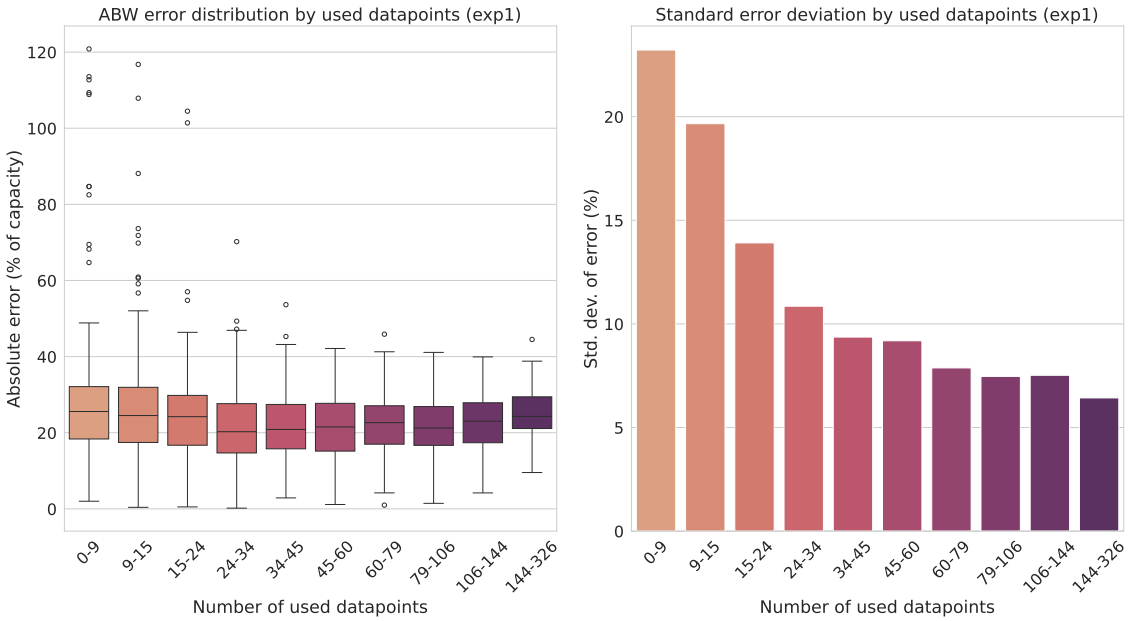

Figure 5.7: Absolute ABW error distribution and standard error deviation plotted against the number of datapoints used for regression.

We analyzed the error as a function of the number of data points used in the regression. In general, estimation error tended to be higher and more variable in cases of very little data, and all experiments show a trend of reduced estimate variance when the number of datapoints increases. However, even though the variance decreases, the error shows two distinct trends: Either the error decreases relative to the number of datapoints, or it trends in a U-shape, decreasing at first, then increasing slightly. Figure 5.7 shows this U-shaped trend, while Figure 5.6 gives an overview of the trends present in all experiments. Without exploring more permutations of the experiments, we cannot

say for certain the exact cause of these patterns. Our best interpretation is that it is related to a combination of latency jitter, cross-traffic intensity, and the relative capacity of the bottlenecks and the outgoing link capacities, which we discuss in more detail in Section 5.4.

5.3.3 Experiment 2: Underestimation

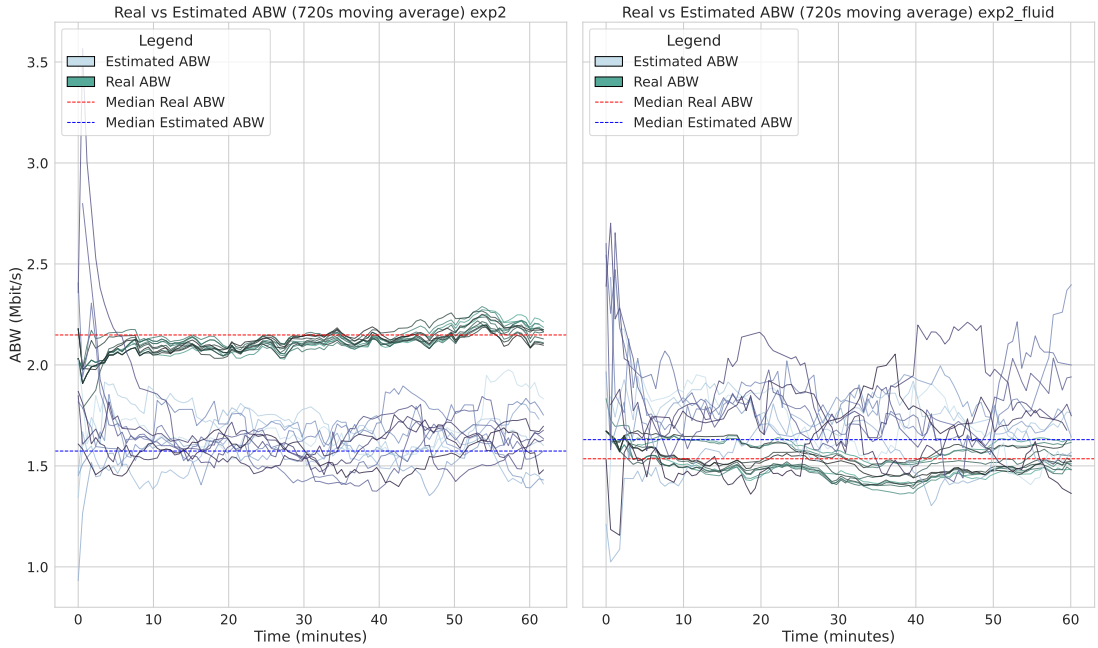

Figure 5.8: Experiment 2 variants showing the differences in real ABW and estimated ABW between the two.

As noted in Section 5.3.2, Experiment 2 and its fluid variant did not follow the general trend of overestimation. In the baseline, the median error was -17.8, while +4.5 in the fluid variant. These two results warrant explanation, especially since they had the added complications of latency jitter and packet loss in the network.

Experiment 2's base scenario combined a tighter link capacity gap with 2% random packet loss. Packet loss tended to manifest in our measurements as a shift of the y=1 intercept in the regression analysis towards zero. In other words, random losses cause the estimator to under-predict ABW; as the number of datapoints not crossing the congestion threshold increases, the output gaps corresponding to the 10% lowest input gaps are shifted towards a lower rate. This causes an effect illustrated in Figure 5.9, where the best fit line is skewed towards a lower ABW, due to a shift in the distribution of datapoints.

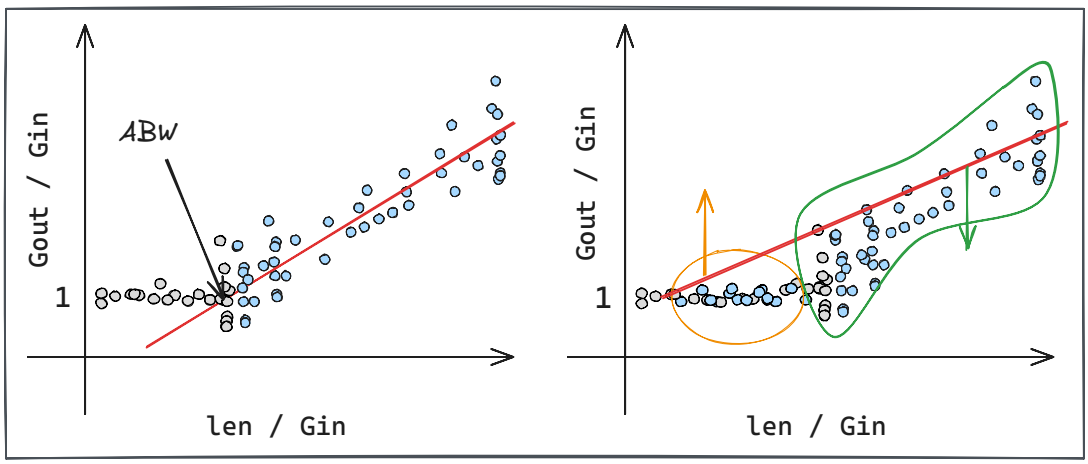

Figure 5.9: Illustration showing the impact of non-congested packet gap data on the regression analysis.

Now, when we switch to the fluid cross-traffic variant of Experiment 2, something interesting occurs. The fluid pattern means that there is an increase in the frequency of queueing events at the bottleneck links. With more frequent queueing, fewer datapoints will contribute to the shift occurring in the non-fluid variant, counteracting this effect. This likely explains the shift from underestimation to a slight overestimation. Figure 5.8 shows this effect; as ABW decreases, the estimated ABW increases, demonstrating that the link characteristics can change the estimator's bias.

5.3.4 Experiment 8: Narrowing the Capacity Gap

Our final experiment was motivated by the observations above. We hypothesized that the significant accuracy improvement seen in the fluid version of Experiment 2 might be a false indication of accuracy, due to just counteracting the bias from the failure to recognize the TCP bursts. Specifically, we wanted to see what would happen when we lower the gap between the bottleneck capacities and the outgoing link capacities. To isolate this effect, Experiment 8 was set up similarly to Experiment 7, which had a median error of 23% across several cross-traffic patterns and intensities, but with the bottleneck link capacity increased from 5 Mbit/s to 7 Mbit/s. This is similar to the relative capacities of Experiment 2 (3 and 5 Mbit/s), while also applying a minor timing jitter for all cross-traffic streams. The idea was to see if the accuracy would improve when narrowing the capacity gap would yield better accuracy.

The results from Experiment 8 confirmed our expectations. The estimation error

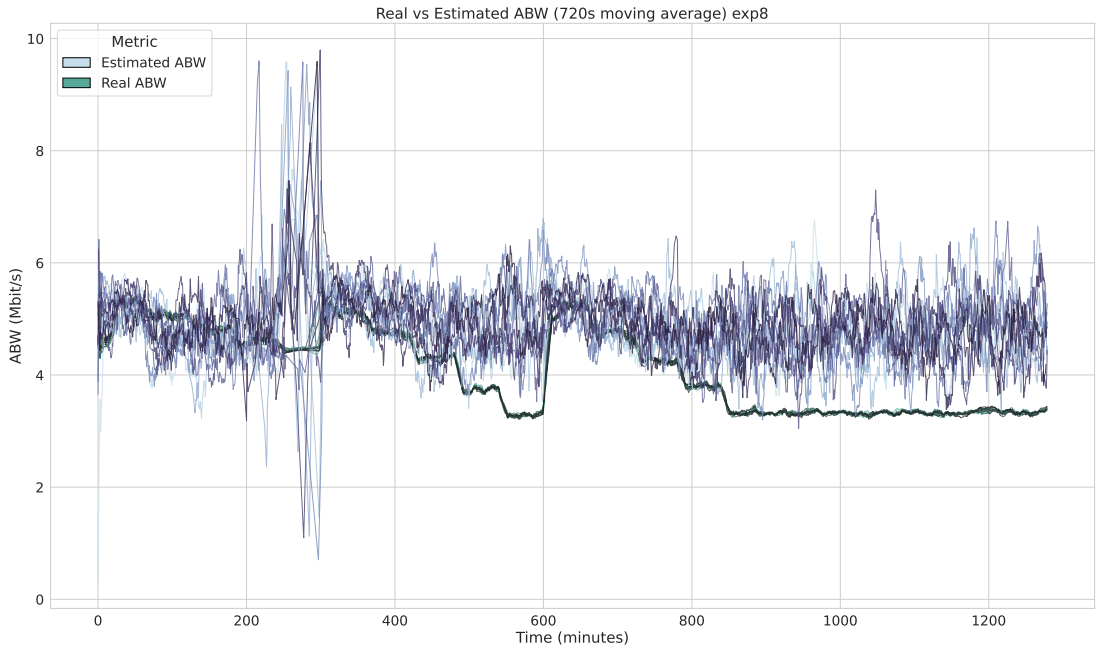

Figure 5.10: Experiment 8 estimated ABW plotted against the ground truth ABW.

dropped relative to Experiment 7. As shown in Table 5.2, the median error in Experiment 8 was about +10%, compared to +23% in Experiment 7, roughly cutting the error in half. Furthermore, when we perform the same analysis of regression-basis, as we did for Experiment 1, we see the accuracy increase in line with the amount of datapoints, shown in Figure 5.6.

5.4 Discussion

In this section, we analyze how network latency, packet loss, timing jitter, and the intensity of TCP traffic interact to influence accuracy. These factors intertwine in complex ways, and our experiments suggest that they can skew the directional bias of the estimation error. Rather than drawing firm conclusions from any single factor, we base our discussion on observed trends and behavior of TCP in our emulated environment.

As we discussed, the set of packet-gap samples used for regression can become skewed towards lower transmission rates during certain conditions. Figure 5.9 illustrates the effect this might have on the y=1 intersect of the regression line, where more datapoints correspond to transmission rates that rarely trigger congestion. In essence, if many packet pairs are observed at lower utilization, the regression model will reflect a lower ABW. We previously suggested that introducing random packet loss on a link can cause

this effect by frequently throttling TCP throughput. Random loss forces TCP flows to back off, so a larger fraction of samples end up in an uncongested state. However, random loss is not the only factor that can skew the data towards lower congestion levels. Any mechanism that prevents the bottleneck from being regularly saturated may produce a similar bias. The outcome is a shift in the systematic directional error towards lower available bandwidths.

The disparity between the bottleneck link's capacity and the upstream link's capacity emerged as a part of this relationship. By narrowing this gap (by increasing bottleneck capacity), we also increased the rate at which TCP needs to transmit for the packets to experience congestion, thereby introducing a shift in the directional bias. We made an attempt at verifying this by re-running Experiment 8 with lower TCP burst sizes, resulting in estimates with a median error of 0.5%. This indicated that the increase in accuracy might have been due to the directional bias shifting, and not the model capturing a more detailed picture of the available bandwidth. Figure C.2 visualizes this trend.

When introducing latency jitter to the network links, the resulting packet gap measurements end up being more distributed. The presence of jitter can blur the distinction between indicators of congestion and packets not influenced by queueing delay. Even when the link is not fully saturated, jitter can cause packet gaps to expand unpredictably, mimicking the effect of queuing delay. We did not perform enough experiments to verify the exact effect this had on the results, but as we outlined in Section 5.3.2, one of the common parameters for the experiments showing a trend of increasing accuracy, was the presence of latency jitter, indicating that when the number of used datapoints increase, the impact of jitter lowers as the trend becomes more apparent.

5.5 Using robust regression

All the analyses above considered only a simple linear regression model. The following section analyzes how switching to the robust regression model influenced the results. Robust regression was intended to reduce the impact of outliers on the yielded estimates.

In general, using robust regression did not drastically change the median accuracy of the estimator in most experiments, but it did have some benefits. The prior sections' findings hold for both regression types, with only slight numerical differences in median

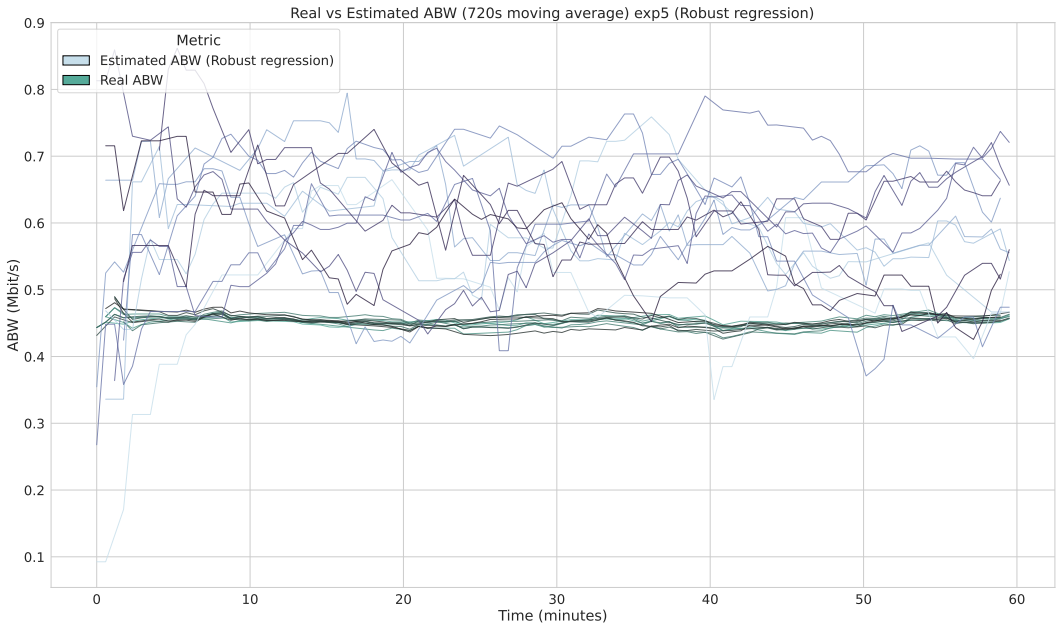

Figure 5.11: Ground truth ABW plotted with the estimated ABW using robust regression.

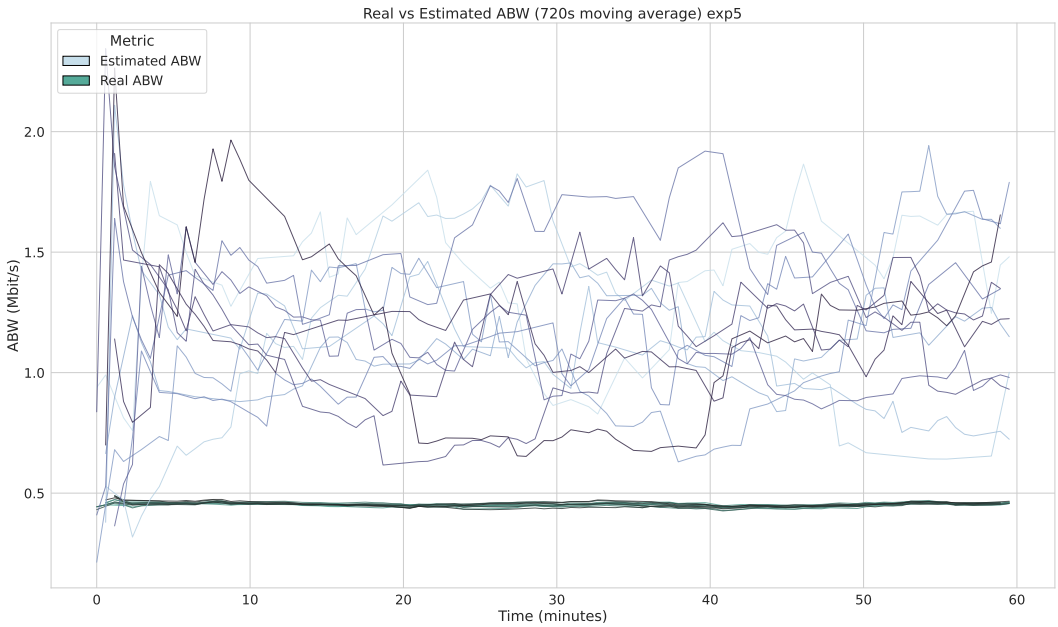

Figure 5.12: Ground truth ABW plotted with the estimated ABW using simple linear regression.

error as seen in Table 5.2. For instance, in experiments which generally had enough datapoints (all experiments except for 4 and 5), the median errors under robust regression were very close to those under simple regression. However, robust regression consistently reduced the variance of the error and the occurrence of extreme outliers.

Looking at the error distributions in Figure 5.3, we can see that the spread (especially the whiskers and outlier points) is generally smaller for the robust regression case.

The impact of robust regression became more significant in the challenging scenarios with sparse data. Experiments 4 and 5, as described, had very low TCP traffic volume and high variability due to loss and jitter. In those cases, using robust regression was able to almost halve the error by discounting some outlier points. Still, Experiment 5 remained one of the worst cases, even after robust fitting, though significantly improved. Figure 5.12 shows an overview of the performance during Experiment 5 using simple linear regression, and 5.11 shows what happened when we fitted a robust model instead.

It should be noted that the robust regression data was gathered after running the experiments by fitting the datapoints to a robust least squares model, instead of either running each experiment twice, or performing two calculations for each datapoint. The model used post data collection is based on the one that can be switched on through a configuration parameter in the tool.

5.6 Performance Evaluation

Across the eleven experiments, the estimator's absolute error ranged roughly from 5% up to 38% of the link capacity, depending on conditions, while most errors hovered around 20%. Based on the results, we identified four main factors that impacted the tool's performance the most:

- **Regression Model**:

  Switching from simple linear regression to a robust regression model had a limited impact on accuracy in most cases, but it improved performance in edge cases. In all experiments, using robust regression tended to tighten the error distribution by mitigating outliers, although the systematic bias remained mostly unchanged.

- **Cross-Traffic Pattern**:

  The cross-traffic pattern mostly influenced the accuracy by leading to fewer congestion indicators being present in the packet gap data. Introducing timing jitter and using a more fluid-like pattern of cross-traffic led to the tool capturing more nuanced details of the changes in ABW.

- **Capacity Ratio and link characteristics**:

  Initially, the results showed that decreasing the gap between the bottleneck capacity and the upstream link capacity made the results more accurate. However,

this turned out to be only partly true, as the increase in accuracy was likely due to TCP not filling up the bottleneck. This ratio, in combination with packet loss and latency jitter, skews the used datapoints towards lower rates, thereby shifting the bias to report lower ABW estimates.

- **Available Data Volume**:

  The amount of probe data per estimation window is another governing factor. In experiments with more frequent packet samples, the results were more consistent, though not necessarily more accurate. In contrast, when fewer samples were available, the estimation error became less predictable.

Concluding this chapter, the results show that our passive estimator can provide estimates within all our performed experiments, but estimation error varies with each scenario. The tool consistently overestimates available bandwidth, but it rarely yields estimates above the bottleneck link capacity. The main challenges identified are the sensitivity to network link parameters and degradation with sparse data or volatile conditions, which consequently impacts the error distribution's directional bias. Overall, it seems like even though accuracy tended to improve when making the links less stable, this is likely not due to the tool actually capturing a more detailed picture of ABW, but rather a flaw in how we determine the ground truth ABW.

Chapter 6

Conclusion

In this thesis, we set out to explore passive available bandwidth estimation within Kubernetes clusters deployed on the edge, guided by the overarching research question of how the natural flow of data on the edge can indicate the state of network links under unstable conditions.

6.1 Research Questions

(1) What are the challenges of implementing bandwidth estimation in radio networks?

Section 2.6 underscores that wireless networks introduce additional complexities such as interference, dynamic rate adaptation, and MAC-layer contention, making bandwidth estimation significantly harder. Chapter 2 first establishes the concept of bandwidth estimation and the theory behind most traditional algorithms, then demonstrates why these techniques break down in wireless settings, ultimately motivating our decision to pivot toward wired network experiments.

(2) How can data collected via passive network interface listening be used to estimate available bandwidth?

Our evaluation in Chapter 5 showed that given enough data and time, the tool provided estimates in all tested scenarios. This demonstrates that data collected via passive network interface listening can be used to gauge ABW. With some minor modifications to the filtration mechanism, we implemented our approach using the algorithm described by Khangura and Fidler (2017), enabling continuous, non-intrusive ABW estimation

without active probing. Importantly, the passive estimates rarely exceeded the true bottleneck capacity, indicating a level of reliability in the method.

However, the accuracy and consistency of the estimates were limited. The results reveal a systematic tendency to overestimate the available bandwidth, with the magnitude of estimation error varying across network conditions. In stable network environments, the estimator's readings were consistently around 20% higher than the real ABW, while its output became less precise under sparse or highly variable traffic. Interestingly, there were cases where estimation appeared to improve when network links were less stable, but upon reflection, this was likely due to limitations in our ground truth measurements rather than a genuine improvement in estimation accuracy. These findings highlight the inherent challenges of passive bandwidth estimation under unpredictable conditions and show that high accuracy is not easy to achieve even under stable conditions.

(3) How can we communicate the passively gathered data with Kubernetes without causing significant strain on the network?

To address this question, we designed and implemented a simple gRPC-based API that could feed metrics to the scheduler, and by using this API for data collection, we showed that it works. However, we did not perform an experimental evaluation of the network overhead introduced by sending these updates through the network. Quantifying this overhead and its impact remains to be tested in future work.

How can we utilize the natural flow of data on the edge to indicate the state of network links in an unstable network?

Taken together, these results partially answer the overarching research question. In the scenarios we tested, we could utilize the natural flow of data to indicate the state of network links in terms of ABW, but with accuracy trade-offs and unmeasured export overhead. The work (1) clarifies why wireless ABW estimation is such a complex problem to solve, (2) explains how passive bandwidth estimation can be done, verifying it through experimental evaluation, and (3) supplies an API for data distribution, developed with network overhead in mind, ready for future overhead profiling. Having addressed the research questions, we now highlight some of the main contributions of this work.

6.2 Contributions

6.2.1 Passive Bandwidth Estimation Framework

We designed and implemented a framework that enables passive bandwidth estimation. By adapting the transport-layer analysis approach of Khangura and Fidler (2017) to our use case, we demonstrated that real-time capacity estimation is possible, even in less optimal network environments. This framework operates alongside application traffic, fulfilling the goal of a non-intrusive tool for edge deployments.

6.2.2 Experimental Evaluation setup

We developed a script-driven testing system on an emulated network using CORE for network emulation and MGEN for traffic generation. This toolkit automates the deployment of test cases and the collection of measurements into structured storage with PostgreSQL, enabling efficient experimentation and evaluation of our tool. Additionally, it provides a foundation for future research to validate new iterations of the estimator under new conditions.

6.2.3 Dataset for Analysis

Over the course of our experiments, we compiled a large dataset of the data used by our tool, and the provided estimates, together with the ground truth throughput measurements. This dataset has been made available as part of the research outputs. It offers a resource for further analysis, such as comparative evaluation of estimation techniques or data-driven refinement of our algorithm.

6.2.4 Proof-of-Concept gRPC API

We implemented a lightweight interface for accessing the ABW measurements discussed in Section 4.5.2. This shows how such a distributed tool can be used to supply the Kubernetes scheduler with additional live network metrics without significant overhead. While not a full demonstration of how well this would work in a Kubernetes cluster deployed on an edge network, it lays a foundation that can be later built upon to improve the concept of a network-aware Kubernetes scheduler.

6.3 Experiences

Alongside these contributions, the project yielded practical insights from the implementation experience. Our work spanned a diverse set of technologies, and most were unfamiliar to us at the beginning of the project. This resulted in a lot of trial-and-error in the early stages of development. On the positive side, the passive monitoring approach successfully achieved its design objectives in our testing environment. The estimator was able to run continuously on each node without injecting any probe traffic. Another aspect that went well was the use of an emulation and automation framework, allowing us to systematically vary network parameters and load patterns. The overall system architecture and methodology worked out, as we were able to combine emulation, traffic generation, and passive data capture into a testbed that produced meaningful data.

Still, there were some parts of the project that halted our progress. Due to the approach we took for the tool's implementation, we ended up implementing all packet parsing and stream decomposition logic partly from scratch, which could have been avoided using a tool like TShark. The reason why we didn't use an existing tool was the idea that we wanted the tool to be less resource-intensive, but we were not able to confirm this during the project. Furthermore, the initial idea of implementing passive ABW estimation for radio networks led to us spending a significant amount of time on developing functionality we didn't use. One of the features related to wireless estimation was left in to help with future research, which is the part that reads information about seen stations from `/proc/net` with metrics such as signal quality, bitrate, and Basic Service Set Identifier (BSSID).

6.4 Future work

Building on this research, several directions for future work can be pursued to deepen and extend this research:

1. Improving Estimation in Idle Periods

A clear next step is to address the estimator's blind spot when traffic is sparse or absent. Purely passive monitoring fails to produce meaningful data when a link carries little

to no traffic. One potential enhancement is to adopt a hybrid approach that remains passive during busy periods but performs lightweight active measurements during quiet periods. The challenge is to do this in a minimal way without wasting capacity so that it doesn't defeat the purpose of low overhead. Our tool implements a starting point for this type of behavior through a module that periodically performs Iperf3 bandwidth tests to other nodes running an instance of the same tool, but it needs to be extended with additional logic to avoid probing when the network is busy.

2. Hardware Deployment and Virtualization Considerations

While our evaluation utilized an emulated wired testbed, testing the bandwidth estimation tool on physical hardware is an important step towards verifying the merit of our results. Running the estimator within a virtual environment offered flexibility and ease of deployment, and using NREC was helpful, as it allowed us to run experiments on a stable virtual machine with a set amount of virtual CPUs and Memory. However, virtualization introduces additional layers between the application and the physical network hardware, potentially affecting the precision of bandwidth measurements. When using hardware-based timestamping, reliant on the NIC's ability to generate timestamps, depending on how the virtualization is done, the timestamping accuracy can be significantly affected. By inspecting the PCI-buses with the `lspci` tool (Mares et al., 2025), we found that the testbed uses a *Virtio network device*, or a virtual NIC (Russell, 2008). As per Red Hat's documentation, virtual network devices typically do not support hardware timestamping, making it challenging to achieve synchronization accuracy better than tens of microseconds (Red Hat Customer Content Services, 2025).

To mitigate the inaccuracies mentioned above, two future research directions should be considered. The first step is to run the emulation directly on a physical machine, where the only virtualization layer is the emulation itself. This would mitigate potential issues related to having multiple virtualization layers. The second step would be to deploy the estimator on a physical testbed with several independent devices, allowing for direct interaction with hardware. This would enable assessment of the tool's performance under real-world conditions.

3. Kubernetes Integration

We have created a prototype tool, performing available bandwidth estimation with functionality to publish this data through a lightweight metrics interface. The next step is to bridge the gap from these metrics to a network-aware scheduler to validate its effectiveness. In practice, this means implementing a scheduler capable of using network information to improve scheduling decisions, but further tuning might also be needed for the tool to filter out outlier estimates that are unlikely to represent the true available bandwidth.

4. Support for wireless networks

Available bandwidth estimation in wireless networks is a complex problem, likely requiring a combination of several techniques to be generic enough for heterogeneous networks. Edge scenarios often involve radio technologies, where available bandwidth can fluctuate due to signal strength changes, interference, and user mobility as discussed in Section 2.6. Passive transport-layer packet analysis alone might not be sufficient in such conditions, as wireless capacity estimation may require the use of algorithms tailored to each specific technology.

Bibliography

Abdelaziz, A., Mean, H. K., & Jean-Pierre, C. (2007). Evaluation of the performance of the SLoPS: Available Bandwidth Estimation Technique in IEEE 802.11b Wireless Networks. In H. Labiod & M. Badra (Eds.), *New Technologies, Mobility and Security* (pp. 123–132). Springer Netherlands. https://doi.org/10.1007/978-1-4020-6270-4_11

Aboba, D. B. D. (2007, June). Architectural Implications of Link Indications. https://doi.org/10.17487/RFC4907

Acharya, P. A., Sharma, A., Belding, E. M., Almeroth, K. C., & Papagiannaki, K. (2010). Rate Adaptation in Congested Wireless Networks through Real-Time Measurements. *IEEE Transactions on Mobile Computing*, *9*(11), 1535–1550. https://doi.org/10.1109/TMC.2010.108

Agarwal, S., Katz, R., Krishnamurthy, S., & Dao, S. (2001). Distributed power control in ad-hoc wireless networks. *12th IEEE International Symposium on Personal, Indoor and Mobile Radio Communications. PIMRC 2001. Proceedings (Cat. No.01TH8598)*, F–59–F–66. https://doi.org/10.1109/PIMRC.2001.965296

Ahrenholz, J., Danilov, C., Henderson, T. R., & Kim, J. H. (2008). CORE: A real-time network emulator. *MILCOM 2008 - 2008 IEEE Military Communications Conference*, 1–7. https://doi.org/10.1109/MILCOM.2008.4753614

Ahrenholz, J., Goff, T., & Adamson, B. (2011). Integration of the CORE and EMANE Network Emulators [ISSN: 2155-7586]. *2011 - MILCOM 2011 Military Communications Conference*, 1870–1875. https://doi.org/10.1109/MILCOM.2011.6127585

Aina, F., Yousef, S., & Osanaiye, O. (2021, February). Be-Ozr: An Efficient Available Bandwidth Estimation In Ieee802.11E Network. https://doi.org/10.21203/rs.3.rs-193297/v1

Airon, M., & Gupta, N. (2017, October). Bandwidth Estimation Tools and Techniques: A Review. https://doi.org/10.20944/preprints201710.0060.v1

Akyildiz, I., Weilian Su, Sankarasubramaniam, Y., & Cayirci, E. (2002). A survey on sensor networks [Publisher: Institute of Electrical and Electronics Engineers (IEEE)]. *IEEE Communications Magazine*, *40*(8), 102–114. https://doi.org/10.1109/mcom.2002.1024422

Arsan, T. (2012). Review of bandwidth estimation tools and application to bandwidth adaptive video streaming [ISSN: 1949-4106]. *High Capacity Optical Networks and Emerging/Enabling Technologies*, 152–156. https://doi.org/10.1109/HONET.2012.6421453

Authors, P. B. (2025). *Protocol buffers language guide (proto3)* [Accessed 6 May 2025]. https://protobuf.dev/programming-guides/proto3/

Azevedo, D., Bonfim, M., Lima, L., & Fernandes, S. (2018). Towards an Accurate Bandwidth Estimation Tool for 802.11n Wireless Networks. *2018 IEEE Symposium on Computers and Communications (ISCC)*, 00486–00491. https://doi.org/10.1109/iscc.2018.8538751

Bergfeldt, E., Ekelin, S., & Karlsson, J. M. (2011). Real-time bandwidth measurements over mobile connections. *European Transactions on Telecommunications*, *22*(6), 255–267. https://doi.org/10.1002/ett.1474

Bernstein, D. (2014). Containers and Cloud: From LXC to Docker to Kubernetes. *IEEE Cloud Computing*, *1*(3), 81–84. https://doi.org/10.1109/MCC.2014.51

Bianchi, G. (2000). Performance analysis of the IEEE 802.11 distributed coordination function. *IEEE Journal on Selected Areas in Communications*, *18*(3), 535–547. https://doi.org/10.1109/49.840210

Boeing Research & Technology division. (2025). *Core architecture* [Accessed: 2025-05-10]. https://coreemu.github.io/core/architecture.html

Bowe, S., Kozlowski, W., & Sekine, H. (2024, September). *Pcap v2.2.0* (Version 2.2.0) [GitHub release]. https://github.com/rust-pcap/pcap/releases/tag/v2.2.0

Brakmo, L., & Peterson, L. (1995). TCP Vegas: End to end congestion avoidance on a global Internet [Conference Name: IEEE Journal on Selected Areas in Communications]. *IEEE Journal on Selected Areas in Communications*, *13*(8), 1465–1480. https://doi.org/10.1109/49.464716

Bray, T. (2017, December). The JavaScript Object Notation (JSON) Data Interchange Format. https://doi.org/10.17487/RFC8259

Bredel, M., & Fidler, M. (2008). A measurement study of bandwidth estimation in IEEE 802.11g wireless LANs using the DCF. *Proceedings of the 7th international IFIP-TC6 networking conference on AdHoc and sensor networks, wireless networks, next generation internet*, 314–325.

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes: Lessons learned from three container-management systems over a decade. *Queue*, *14*(1), 70–93. https://doi.org/10.1145/2898442.2898444

Canonical Ltd. (2024, December). *Microk8s documentation* (Version 1.32). https://github.com/canonical/microk8s/releases/tag/v1.32

Carr, M., & Verner, J. (1997). Prototyping and software development approaches [Publisher: Citeseer]. *Department of Information Systems, City University of Hong Kong, Hong Kong*, 319–338.

Carson, M., & Santay, D. (2003). NIST Net: A Linux-based network emulation tool. *SIGCOMM Comput. Commun. Rev.*, *33*(3), 111–126. https://doi.org/10.1145/956993.957007

Chan, K. C., & Haythorne, S. (2017). A networking laboratory using an integrated mixture of physical equipment and simulators. *Computer Applications in Engineering Education*, *25*(5), 681–692. https://doi.org/10.1002/cae.21829

Chaudhari, S. S., & Biradar, R. C. (2015). Survey of Bandwidth Estimation Techniques in Communication Networks. *Wireless Personal Communications*, *83*(2), 1425–1476. https://doi.org/10.1007/s11277-015-2459-2

Chimento, P., & Ishac, J. (2008, February). *Defining Network Capacity* (Request for Comments No. RFC5136). Internet Engineering Task Force. https://doi.org/10.17487/rfc5136

Chin, A. (2024, October 7). *Procfs* (Version 0.17.0) [Rust crate for Linux /proc filesystem access]. https://github.com/eminence/procfs/releases/tag/v0.17.0

Ciaccia, F., Ciaccia, F., Ciaccia, F., Romero, I., Romero, I., Arcas-Abella, O., Arcas-Abella, O., Arcas-Abella, O., Montero, D., Montero, D., Serral-Gracià, R., Serral-Gracia, R., Nemirovsky, M., Nemirovsky, M., & Nemirovsky, M. (2020). SABES: Statistical Available Bandwidth EStimation from passive TCP measurements. *2020 IFIP Networking Conference (Networking)*. https://doi.org/null

Cottrell, L. (2003, April). *pathChirp: Efficient Available Bandwidth Estimation for Network Paths* (tech. rep. No. SLAC-PUB-9732, 813038). https://doi.org/10.2172/813038

Dovrolis, C., Ramanathan, P., & Moore, D. (2001). What do packet dispersion techniques measure? *Proceedings IEEE INFOCOM 2001. Conference on Computer Communications. Twentieth Annual Joint Conference of the IEEE Computer and Communications Society (Cat. No.01CH37213)*, *2*, 905–914. https://doi.org/10.1109/INFCOM.2001.916282

Dovrolis, C., Ramanathan, P., & Moore, D. (2004). Packet-Dispersion Techniques and a Capacity-Estimation Methodology. *IEEE/ACM Transactions on Networking*, *12*(6), 963–977. https://doi.org/10.1109/TNET.2004.838606

Dubey, A. (2021, August). All about k3s - Lightweight Kubernetes. Retrieved May 26, 2024, from https://dev.to/abhinavd26/all-about-k3s-lightweight-kubernetes-3ell

ESnet. (2024). *Iperf3* (Version 3.18) [Release date: 13 December 2024]. https://software.es.net/iperf/

Fall, K. (1999). Network emulation in the VINT/NS simulator. *Proceedings IEEE International Symposium on Computers and Communications (Cat. No.PR00250)*, 244–250. https://doi.org/10.1109/ISCC.1999.780820

Farshad, A., Lee, M., Marina, M. K., & Garcia, F. (2014). On the impact of 802.11n frame aggregation on end-to-end available bandwidth estimation. *2014 Eleventh Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*, 108–116. https://doi.org/10.1109/SAHCN.2014.6990333

Feng, H., & Shu, Y. (2007). Statistical Analysis of Packet Interarrival Times in Wireless LAN. *2007 International Conference on Wireless Communications, Networking and Mobile Computing*, 1888–1891. https://doi.org/10.1109/WICOM.2007.473

Fontes, H., Campos, R., & Ricardo, M. (2015). Improving ns-3 Emulation Support in Real-World Networking Scenarios. *Proceedings of the Eighth EAI International Conference on Simulation Tools and Techniques*. https://doi.org/10.4108/eai.24-8-2015.2261074

Fontes, R. R., Afzal, S., Brito, S. H. B., Santos, M. A. S., & Rothenberg, C. E. (2015). Mininet-WiFi: Emulating software-defined wireless networks. *2015 11th*

International Conference on Network and Service Management (CNSM), 384–389. https://doi.org/10.1109/CNSM.2015.7367387

Fronteddu, R., Morelli, A., & Suri, N. (2022). Enhancing network link state estimation by incorporating passive packet pair measurement into the Mockets transport protocol. *Procedia Computer Science*, *205*, 1–9. https://doi.org/10.1016/j.procs.2022.09.001

Garg, V. K. (2007). Fundamentals of Cellular Communications. In *Wireless Communications & Networking* (pp. 123–147). Elsevier. https://doi.org/10.1016/B978-012373580-5/50039-9

Gerla, M., Sanadidi, M., Wang, R., Zanella, A., Casetti, C., & Mascolo, S. (2001). TCP Westwood: Congestion window control using bandwidth estimation. *GLOBECOM'01. IEEE Global Telecommunications Conference (Cat. No.01CH37270)*, *3*, 1698–1702 vol.3. https://doi.org/10.1109/GLOCOM.2001.965869

Google LLC. (2025). Protocol Buffers. Retrieved April 8, 2025, from https://protobuf.dev/

Group, P. G. D. (2025). *Postgresql 17* (Version 17.4) [Release date: 20 February 2025]. https://www.postgresql.org/

gRPC Authors. (2021, November). gRPC Documentation. Retrieved April 8, 2025, from https://grpc.io/docs/

gRPC Authors. (2025). *Client-streaming rpc* [Accessed: 2025-05-11]. https://grpc.io/docs/what-is-grpc/core-concepts/#client-streaming-rpc

Guerrero, C. D., & Labrador, M. A. (2008). A Hidden Markov Model approach to available bandwidth estimation and monitoring. *2008 IEEE Internet Network Management Workshop (INM)*, 1–6. https://doi.org/10.1109/INETMW.2008.4660326

Gupta, D., Wu, D., Mohapatra, P., & Chuah, C.-N. (2009). Experimental Comparison of Bandwidth Estimation Tools for Wireless Mesh Networks [ISSN: 0743-166X]. *IEEE INFOCOM 2009*, 2891–2895. https://doi.org/10.1109/INFCOM.2009.5062253

Gupta, P., & Kumar, P. (2000). The capacity of wireless networks. *IEEE Transactions on Information Theory*, *46*(2), 388–404. https://doi.org/10.1109/18.825799

Haider, Z., Saleem, M., & Jamal, T. (2018, October). Analysis of Interference in Wireless Networks [arXiv:1810.13164 [cs]]. https://doi.org/10.48550/arXiv.1810.13164

Han, M. P., Htet, S. Y., & Wuttisttikulkij, L. (2022). Hybrid GNS3 and Mininet-WiFi Emulator for SDN Backbone Network Supporting Wireless IoT Traffic. *2022 37th International Technical Conference on Circuits/Systems, Computers and Communications (ITC-CSCC)*, 768–771. https://doi.org/10.1109/ITC-CSCC55581.2022.9895019

Huang, J., Ito, N., Oshiba, T., Satoda, K., & Murase, T. (2024). PathRefiner: Accurate Bandwidth Estimation Using Large-Sized Virtual Packets for High-Speed Networks. *2024 IEEE International Conference on Consumer Electronics (ICCE)*, 1–6. https://doi.org/10.1109/ICCE59016.2024.10444457

Introduction to Linux Traffic Control. (n.d.). Retrieved March 25, 2025, from https://tldp.org/HOWTO/Traffic-Control-HOWTO/intro.html

Iyer, A., Rosenberg, C., & Karnik, A. (2006). What is the right model for wireless channel interference? *Proceedings of the 3rd international conference on Quality of service in heterogeneous wired/wireless networks - QShine '06*, 2. https://doi.org/10.1145/1185373.1185376

Jain, K., Padhye, J., Padmanabhan, V. N., & Qiu, L. (2003). Impact of interference on multi-hop wireless network performance. *Proceedings of the 9th annual international conference on Mobile computing and networking*, 66–80. https://doi.org/10.1145/938985.938993

Jain, M., & Dovrolis, C. (2003). End-to-end available bandwidth: Measurement methodology, dynamics, and relation with TCP throughput. *IEEE/ACM Transactions on Networking*, *11*(4), 537–549. https://doi.org/10.1109/TNET.2003.815304

Jain, M., & Dovrolis, C. (2002). Pathload: A Measurement Tool for End-to-End Available Bandwidth. *Proceedings of Passive and Active Measurement Workshop*.

Jain, M., & Dovrolis, C. (2004). Ten fallacies and pitfalls on end-to-end available bandwidth estimation. *Proceedings of the 4th ACM SIGCOMM conference on Internet measurement*, 272–277. https://doi.org/10.1145/1028788.1028825

Jain, M., & Dovrolis, C. (2005). End-to-end estimation of the available bandwidth variation range. *Proceedings of the 2005 ACM SIGMETRICS international conference on Measurement and modeling of computer systems*, 265–276. https://doi.org/10.1145/1064212.1064242

Johnsen, F. T., Hauge, M., Landmark, L., Andersen, E. P., Mjelde, T. M., Sander, J., & Rytir, M. (2023). Adaptive Networking at the Tactical Edge. *2023 International Conference on Military Communications and Information Systems (ICMCIS)*, 1–10. https://doi.org/10.1109/ICMCIS59922.2023.10253558

Jurgelionis, A., Laulajainen, J.-P., Hirvonen, M., & Wang, A. I. (2011). An Empirical Study of NetEm Network Emulation Functionalities. *2011 Proceedings of 20th International Conference on Computer Communications and Networks (ICCCN)*, 1–6. https://doi.org/10.1109/ICCCN.2011.6005933

K3s Project Authors. (2024, September). K3s - Lightweight Kubernetes. Retrieved September 9, 2024, from https://docs.k3s.io/

Kayal, P. (2020). Kubernetes in Fog Computing: Feasibility Demonstration, Limitations and Improvement Scope : Invited Paper. *2020 IEEE 6th World Forum on Internet of Things (WF-IoT)*, 1–6. https://doi.org/10.1109/WF-IoT48130.2020.9221340

Kerrisk, M. (2025). *select(2) - synchronous i/o multiplexing*. The Linux man-pages Project. Retrieved April 27, 2025, from %5Curl%7Bhttps://man7.org/linux/man-pages/man2/select.2.html%7D

Khan, M. A. Y., & Veitch, D. (2008). Speedo: Realistic achievable bandwidth in 802.11 through passive monitoring. *2008 33rd IEEE Conference on Local Computer Networks (LCN)*, 892–899. https://doi.org/10.1109/LCN.2008.4664299

Khangura, S. K. (2019). Machine learning-based available bandwidth estimation [Publisher: Hannover : Institutionelles Repositorium der Leibniz Universität Hannover]. https://doi.org/10.15488/9166

Khangura, S. K., & Akın, S. (2021). Online available bandwidth estimation using multiclass supervised learning techniques. *Computer Communications*, *170*, 177–189. https://doi.org/10.1016/j.comcom.2021.02.009

Khangura, S. K., & Fidler, M. (2017). Available bandwidth estimation from passive TCP measurements using the probe gap model. *2017 IFIP Networking Conference (IFIP Networking) and Workshops*, 1–9. https://doi.org/10.23919/IFIPNetworking.2017.8264826

Klabnik, S. (2018). *The Rust programming language*. No Starch Press, Inc. https://doi.org/10.5555/3271463

Kosek-Szott, K. (2012). A survey of MAC layer solutions to the hidden node problem in ad-hoc networks. *Ad Hoc Networks*, *10*(3), 635–660. https://doi.org/10.1016/j.adhoc.2011.10.003

Koziolek, H., & Eskandani, N. (2023). Lightweight Kubernetes Distributions: A Performance Comparison of MicroK8s, k3s, k0s, and Microshift. *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering*, 17–29. https://doi.org/10.1145/3578244.3583737

Kumar, A., Manjunath, D., & Kuri, J. (2004). Multiple Access: Wireless Networks. In *Communication Networking* (pp. 435–531). Elsevier. https://doi.org/10.1016/B978-012428751-8/50008-2

Labs, G. (2025). *Grafana* (Version 12.0.0) [Release date: 5 May 2025]. https://grafana.com

Lakshminarayanan, K., Padmanabhan, V. N., & Padhye, J. (2004). Bandwidth estimation in broadband access networks. *Proceedings of the 4th ACM SIGCOMM conference on Internet measurement*, 314–321. https://doi.org/10.1145/1028788.1028832

Lantz, B., Heller, B., & McKeown, N. (2010). A network in a laptop: Rapid prototyping for software-defined networks. *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, 1–6. https://doi.org/10.1145/1868447.1868466

Liu, M. (2006). An End-to-End Available Bandwidth Estimation Methodology. *Journal of Software*, *17*(1), 108. https://doi.org/10.1360/jos170108

Liu, X., Ravindran, K., & Loguinov, D. (2007). A Queueing-Theoretic Foundation of Available Bandwidth Estimation: Single-Hop Analysis. *IEEE/ACM Transactions on Networking*, *15*(4), 918–931. https://doi.org/10.1109/TNET.2007.896235

Luo, P., Huang, H., Shu, W., Li, M., & Wu, M.-Y. (2008). Performance Evaluation of Vehicular DTN Routing under Realistic Mobility Models. *2008 IEEE Wireless Communications and Networking Conference*, *3158*, 2206–2211. https://doi.org/10.1109/wcnc.2008.390

Mares, M., et al. (2025). *Lspci(8) – list all pci devices* [Accessed May 8, 2025]. The PCI Utilities. https://man7.org/linux/man-pages/man8/lspci.8.html

Melander, B., Melander, B., Björkman, M., Björkman, M., Gunningberg, P., & Gunningberg, P. (2000). A new end-to-end probing and analysis method for

estimating bandwidth bottlenecks. *null*. https://doi.org/10.1109/glocom.2000.892039

Mingzhe Li, Claypool, M., & Kinicki, R. (2008). WBest: A bandwidth estimation tool for IEEE 802.11 wireless networks. *2008 33rd IEEE Conference on Local Computer Networks (LCN)*, 374–381. https://doi.org/10.1109/LCN.2008.4664193

Mishra, G., & Rath, H. K. (2020). Scalability Analysis of an Emulated Platform for Heterogeneous Networks. *2020 International Conference on COMmunication Systems & NETworkS (COMSNETS)*, 702–707. https://doi.org/10.1109/COMSNETS48256.2020.9027435

MQTT Version 5.0. (2019, March). Retrieved September 9, 2024, from https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

Muelas, D., Ramos, J., & Vergara, J. E. L. D. (2018). Assessing the Limits of Mininet-Based Environments for Network Experimentation. *IEEE Network*, *32*(6), 168–176. https://doi.org/10.1109/MNET.2018.1700277

Mukta & Gupta, N. (2020). Analytical approach towards available bandwidth estimation in wireless ad hoc networks. *Wireless Networks*, *26*(4), 2957–2982. https://doi.org/10.1007/s11276-020-02249-8

Nam, S. Y., Kim, S. J., Lee, S., & Kim, H. S. (2013). Estimation of the available bandwidth ratio of a remote link or path segments. *Computer Networks*, *57*(1), 61–77. https://doi.org/10.1016/j.comnet.2012.08.015

Naval Research Labratory. (2021, August). Multi-Generator. Retrieved April 8, 2025, from https://www.nrl.navy.mil/Our-Work/Areas-of-Research/Information-Technology/NCS/MGEN/

Nishimori, K., Kitao, K., & Imai, T. (2012). Interference-Based Decode and Forward Scheme Using Relay Nodes in Heterogeneous Networks. *International Journal of Antennas and Propagation*, *2012*, 1–10. https://doi.org/10.1155/2012/103501

Obara, H., Koseki, S., & Selin, P. (2012). Packet train pair: A fast and efficient technique for measuring available bandwidth in the Internet. *2012 Proceedings of SICE Annual Conference (SICE)*, 1833–1836. Retrieved May 28, 2024, from https://ieeexplore.ieee.org/abstract/document/6318753

Overby, T., Koenders, V., & R, Z. (2025). *Bincode* [Version 2.0.1]. https://crates.io/crates/bincode

Padhye, J., Agarwal, S., Padmanabhan, V. N., Qiu, L., Rao, A., & Zill, B. (2005). Estimation of link interference in static multi-hop wireless networks. *Proceedings of the 5th ACM SIGCOMM conference on Internet measurement*, 28.

Paris, S., Facchi, N., & Gringoli, F. (2015). Rate adaptation algorithms for reliable multicast transmissions in wireless LANs. In *Modeling and Simulation of Computer Networks and Systems* (pp. 55–94). Elsevier. https://doi.org/10.1016/B978-0-12-800887-4.00003-1

Park, H. J., Park, H. J., Roh, B.-h., & Roh, B.-h. (2010). Accurate Passive Bandwidth Estimation (APBE) in IEEE 802.11 Wireless LANs. *Proceedings of the International Conference on Ubiquitous Information Technologies & Applications*. https://doi.org/10.1109/icut.2010.5677782

Park, K.-J., Lim, H., Hou, J. C., & Choi, C.-H. (2009). Feedback-assisted robust estimation of available bandwidth. *Computer Networks*, *53*(7), 896–912. https://doi.org/10.1016/j.comnet.2008.10.023

pnet Contributors. (2024, May). *Libpnet: Cross-platform, low level networking using the Rust programming language* (Version 0.35.0). https://docs.rs/pnet/0.35.0/pnet/index.html

Postel, J. (1981, September). *Internet Protocol* (tech. rep. No. RFC0791). RFC Editor. https://doi.org/10.17487/rfc0791

Prasad, R., Davrolis, C., Murray, M., & Claffy, K. (2003). Bandwidth estimation: Metrics, measurement techniques, and tools. *IEEE Network*, *17*(6), 27–35. https://doi.org/10.1109/MNET.2003.1248658

Red Hat Customer Content Services. (2025). *Kvm guest timing management* [Accessed May 8, 2025]. Red Hat, Inc. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/virtualization_deployment_and_administration_guide/chap-kvm_guest_timing_management

Rensin, D. (2015). *Kubernetes*. O'Reilly Media, Incorporated.

Rescorla, E. (2018, August). The Transport Layer Security (TLS) Protocol Version 1.3. https://doi.org/10.17487/RFC8446

Riley, G. F., & Henderson, T. R. (2010). The ns-3 Network Simulator. In K. Wehrle, M. Güneş, & J. Gross (Eds.), *Modeling and Tools for Network Simulation* (pp. 15–34). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-12331-3_2

Rizzo, L. (1997). Dummynet: A Simple Approach to the Evaluation of Network Protocols. *27*.

Russell, R. (2008). Virtio: Towards a de-facto standard for virtual I/O devices. *ACM SIGOPS Operating Systems Review*, *42*(5), 95–103. https://doi.org/10.1145/1400097.1400108

Sarr, C., Chaudet, C., Chelius, G., & Guérin Lassous, I. (2008). Bandwidth Estimation for IEEE 802.11-Based Ad Hoc Networks [Conference Name: IEEE Transactions on Mobile Computing]. *IEEE Transactions on Mobile Computing*, *7*(10), 1228–1241. https://doi.org/10.1109/TMC.2008.41

Serde Contributors. (2025, March). *Serde* (Version 1.0.219). https://docs.rs/serde/1.0.219/serde/index.html

Shah, S. H., Chen, K., & Nahrstedt, K. (2003). Available Bandwidth Estimation in IEEE 802.11-based Wireless Networks. Retrieved March 18, 2025, from https://www.semanticscholar.org/paper/Available-Bandwidth-Estimation-in-IEEE-802.11-based-Shah-Chen/93062b94fa2246654dedd9b5d691bf3328be89b2#citing-papers

Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal*, *3*(5), 637–646. https://doi.org/10.1109/JIOT.2016.2579198

Skordoulis, D., Qiang Ni, Hsiao-Hwa Chen, Stephens, A., Changwen Liu, & Jamalipour, A. (2008). IEEE 802.11n MAC frame aggregation mechanisms for next-generation high-throughput WLANs. *IEEE Wireless Communications*, *15*(1), 40–47. https://doi.org/10.1109/MWC.2008.4454703

Snyder, H. (2019). Literature review as a research methodology: An overview and guidelines. *Journal of Business Research*, *104*, 333–339. https://doi.org/https://doi.org/10.1016/j.jbusres.2019.07.039

Sooriyabandara, M., Fairhurst, G., Padmanabhan, V., & Balakrishnan, H. (2002, December). TCP Performance Implications of Network Path Asymmetry. https://doi.org/10.17487/RFC3449

Strauss, J., Katabi, D., & Kaashoek, F. (2003). A measurement study of available bandwidth estimation tools. *Proceedings of the 2003 ACM SIGCOMM conference on Internet measurement - IMC '03*, 39. https://doi.org/10.1145/948205.948211

Swann, M., Rose, J., Bendiab, G., Shiaeles, S., & Savage, N. (2020). Tools for Network Traffic Generation - A Quantitative Comparison. *World Congress on Internet*

Security (WorldCIS-2020) in collaboration with the International Conference for Internet Technology and Secured Transactions (ICITST-2020), World Congress on Sustainable Technologies (WCST-2020) and World Congress on Industrial Control Systems Security (WCICSS-2020), 24–29. https://doi.org/10.20533/WorldCIS.2020.0003

Systems, C. (2024). *Trex traffic generator* (Version 3.06) [Release date: 17 September 2024]. https://github.com/cisco-system-traffic-generator/trex-core

Takano, Y., Mutoh, R., Oguchi, N., & Abe, S. (2016). Estimating available bandwidth in mobile networks by correlation coefficient. *2016 18th Asia-Pacific Network Operations and Management Symposium (APNOMS)*, 1–4. https://doi.org/10.1109/APNOMS.2016.7737275

The Kubernetes Authors. (2024a). *Overview* [Accessed: 2024-09-09]. https://kubernetes.io/docs/concepts/overview/

The Kubernetes Authors. (2024b, August). Etcd. Retrieved September 16, 2024, from https://kubernetes.io/docs/concepts/architecture/#etcd

The Kubernetes Authors. (2024c, August). Kube-apiserver. Retrieved September 16, 2024, from https://kubernetes.io/docs/concepts/architecture/#kube-apiserver

The Kubernetes Authors. (2024d, August). Kube-scheduler. Retrieved September 16, 2024, from https://kubernetes.io/docs/concepts/architecture/#kube-scheduler

The Kubernetes Authors. (2025a). *Deployment* [Accessed May 9, 2025]. Cloud Native Computing Foundation. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

The Kubernetes Authors. (2025b). *Pods* [Accessed May 9, 2025]. Cloud Native Computing Foundation. https://kubernetes.io/docs/concepts/workloads/pods/pod/

The Kubernetes Authors. (2025c). *Service* [Accessed May 9, 2025]. Cloud Native Computing Foundation. https://kubernetes.io/docs/concepts/services-networking/service/

The Tcpdump Group. (2025, March). Tcpdump(1) man page. Retrieved April 8, 2025, from https://www.tcpdump.org/manpages/tcpdump.1.html

Timescale, I. (2025, April 15). *Timescaledb: A time-series database for high-performance real-time analytics packaged as a postgres extension* (Version 2.19.3). https://github.com/timescale/timescaledb/releases/tag/2.19.3

Tokio Contributors. (2025, April). Rust's asynchronous runtime. https://docs.rs/tokio/1.44.2/tokio/index.html

tonic Contributors. (2025, April). *A native gRPC client & server implementation with async/await support* (Version 0.13.0). https://docs.rs/tonic/0.13.0/tonic/index.html

Turrubiartes, M., Torres, D., Angulo, M., & Munoz, D. (2005). Analysis of IP Network Path Capacity Estimation Using a Variable Packet Size Method. *15th International Conference on Electronics, Communications and Computers (CONIELECOMP'05)*, 177–182. https://doi.org/10.1109/CONIEL.2005.20

Tursunova, S., Inoyatov, K., & Kim, Y.-T. (2010). Cognitive passive estimation of available bandwidth (cPEAB) in overlapped IEEE 802.11 WiFi WLANs [ISSN: 2374-9709]. *2010 IEEE Network Operations and Management Symposium - NOMS 2010*, 448–454. https://doi.org/10.1109/NOMS.2010.5488496

Vatne, N. (2022). Edge computing for disaster relief operations.

Wong, S. H. Y., Yang, H., Lu, S., & Bharghavan, V. (2006). Robust rate adaptation for 802.11 wireless networks. *Proceedings of the 12th annual international conference on Mobile computing and networking*, 146–157. https://doi.org/10.1145/1161089.1161107

Xiliang Liu, Ravindran, K., & Loguinov, D. (2008). A Stochastic Foundation of Available Bandwidth Estimation: Multi-Hop Analysis. *IEEE/ACM Transactions on Networking*, *16*(1), 130–143. https://doi.org/10.1109/TNET.2007.899014

Xiong, Y., Sun, Y., Xing, L., & Huang, Y. (2018). Extend Cloud to Edge with KubeEdge. *2018 IEEE/ACM Symposium on Edge Computing (SEC)*, 373–377. https://doi.org/10.1109/SEC.2018.00048

Yakubov, D., & Hästbacka, D. (2025). Comparative Analysis of Lightweight Kubernetes Distributions for Edge Computing: Performance and Resource Efficiency [Publisher: arXiv Version Number: 1]. https://doi.org/10.48550/ARXIV.2504.03656

Zhao, H., Garcia-Palacios, E., Wei, J., & Xi, Y. (2009). Accurate available bandwidth estimation in IEEE 802.11-based ad hoc networks. *Computer Communications*, *32*(6), 1050–1057. https://doi.org/10.1016/j.comcom.2008.12.031

Appendices

Appendix A

Tool Configuration

```
[client]
iface = "eth0"
listen_port = 40042
link_phy_cap = 10000000
measurement_window = 35
tstamp_type = "adapter"
precision = "micro"
regression_type = "simple"

[server]
ip = "172.16.0.254"
port = 50041
send_rtts = false
send_link_states = true
send_pgm_dps = true
```

Listing 5: Configuration file sample for `network_listener`.

Listing 5 shows a sample configuration file for the implemented tool. Here, we define parameters such as which interface to listen on, which port to use for gRPC when sending and receiving from other clients, and different packet capture settings. It also includes some true/false switches that trigger different experimental behaviors. For instance, setting `send_rtts` to true makes the tool send calculated RTTs to the IP and port, set through the `server.ip` and `server.port` fields. This was mainly used for testing purposes and to visualize TCP behavior prior to implementing the ABW estimation

algorithm. The `send_link_states` and `send_pgm_dps` fields work in the same way.

Some parts of the tool's implementation are unfinished, such as performing active measurements during times when the network is idle. The `listen_port` defines which port the tool should use for communicating with other instances of itself, and is currently used to identify other running instances of the tool.

Appendix B

Installation

The tool itself, experiment configurations, figures, plots, and raw data are all present in the GitHub repository found at: https://github.com/kjetilom/network_listener/releases/tag/v0.1. The following presents a guide for the installation and execution of the implemented tool. To run the tool itself without the emulation or data collection framework, you should only need to clone the repository, install Rust, Iperf3, and net-tools. These instructions have been tested on a fresh Ubuntu 22.04 LTS installation. We did not manage to install it on Ubuntu 24. We recommend performing this installation on a virtual machine.

After installing the dependencies described in Listing 6, and starting and configuring the Grafana and PostgreSQL pods as described in Listing 7, we can add PostgreSQL as a data source in Grafana. After logging in, navigate to the "Connections" tab, select "Data Sources", and search for and select the "PostgreSQL" data source.

These steps might differ depending on how the pods were configured, but if none were changed, the following should work:

- Host URL: Use the IP address of the docker image with the configured port `172.17.0.1:5432`.
- Set user and password accordingly
- TLS/SSL Mode: Set to disabled
- Enable TimescaleDB
- The rest can be left as default.
- Press the Save and test button to verify that it works.

```
# Install requirements for experimental setup
# Client version 17 should be installed if available, but as
# of writing this guide, it was not present through apt-get.
sudo apt-get update -y
sudo apt-get install -y \
    postgresql-client-common \
    postgresql-client-14 \
    docker \
    libpcap-dev \
    iperf3 \
    net-tools \
    make \
    gcc \
    protobuf-compiler \
    docker.io

# Install rustc and cargo
cd $HOME
# Check if Rust is installed; if not, install it.
if ! command -v cargo &> /dev/null; then
    curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
    source $HOME/.cargo/env
fi

# Install and build the project:
cd $HOME
git clone https://github.com/kjetilom/network_listener.git
cd network_listener
make build # cargo build --release
```

Listing 6: Installation script for test setup requirements.

If all the prior steps worked as expected, PostgreSQL should now be available as an option during dashboard creation (Dashboards → Create dashboard → Add visualization → name-of-the-datasource).

```
# Install Docker image configured with TimescaleDB
# This is required to be able to upload experiment data
sudo docker pull timescale/timescaledb-ha:pg17

# Run the container:
# username, database name and password can be changed
# This should work with the preconfigured
# `database_cfg.toml` file
sudo docker run --name pgrdb \
          -e POSTGRES_USER=user \
          -e POSTGRES_PASSWORD=password \
          -e POSTGRES_DB=metricsdb \
          -p 5432:5432 \
          -d timescale/timescaledb-ha:pg17

# Check if the container is running:
sudo docker ps | grep pgrdb

# Try to connect to the database
psql -h localhost -U user -d metricsdb

# Pull the latest Grafana Docker image and run it.
sudo docker run -d \
    -p 3000:3000 \
    --name=grafana \
    grafana/grafana-enterprise
# Accessing http://localhost:3000 should now yield a login page
# with both username and password set to admin.
```

Listing 7: Start required pods and verify that they are running correctly.

B.1 Database configuration

Before adding visualizations, the database needs to be configured with the correct tables, views, and functions. This can either be achieved through loading the dataset provided in `experiment_data.sql.gz`, or through executing the `up.sql` and `views.sql` files present under `src/scheduler/`. Instructions on how to do this are present in Listing 8.

If you choose to load the dataset provided, it can be viewed through Grafana by importing the JSON file `grafana_dashboard.json` through the option under

```
# Install git-lfs and gzip to install the dataset-gzip file
sudo apt-get install -y \
    git-lfs \
    gzip

# Clone the tool repository
cd $HOME
git clone https://github.com/kjetilom/network_listener.git

# Install the dataset file.
# This will also sync the folder with all of the plots and figures.
cd network_listener
git lfs pull

# Extract and load the dataset
gzip -d experiment_data.sql.gz
psql -h localhost -U user -d metricsdb < experiment_data.sql

# Alternative (assuming the repo has been cloned):
# This only sets up the database schema for experiment data.
cd network_listener
psql -h localhost -U user -d metricsdb # Enter password

# In the psql shell:
\i src/scheduler/up.sql
\i src/scheduler/views.sql
```

Listing 8: Set up the database either with the experiment data dump or with the schema alone.

Dashboards → Create dashboard → Import dashboard. The visualizations might need to be configured with the correct datasource after loading to show the visualizations, and Grafana shows timeseries data, so the time window needs to be configured according to when the experiment was performed (An example would be Experiment 1, which was started on 2025-04-21 16:18:00).

B.2 CORE Installation

CORE and MGEN need to be installed to run the experiments. The installation instructions for CORE are available at https://coreemu.github.io/core/install.html, and

MGEN at https://github.com/USNavalResearchLaboratory/mgen. Listing 9 shows an example of how to install CORE and MGEN.

```
# The repository contains a setup script for CORE fetched from
# the CORE documentation at https://coreemu.github.io/core/install_ubuntu.html
# with some minor modifications regarding installation location.
cd $HOME/network_listener
./setup.sh

cd $HOME
# Install mgen
git clone https://github.com/USNavalResearchLaboratory/mgen.git
cd mgen
git submodule update --init
cd makefiles
make -f Makefile.linux

# Create a link to the executable
sudo ln mgen /usr/bin/mgen
```

Listing 9: CORE and MGEN installation.

To get CORE working alongside Docker, we also need to add the contents of Listing 10 to `/etc/docker/docker.json`, although this might break normal Docker networking.

```
{
  "iptables": false
}
```

Listing 10: Allow CORE to run alongside Docker.

Appendix C

Additional Figures

The following is a collection of plots that were not included in the results chapter of this thesis.

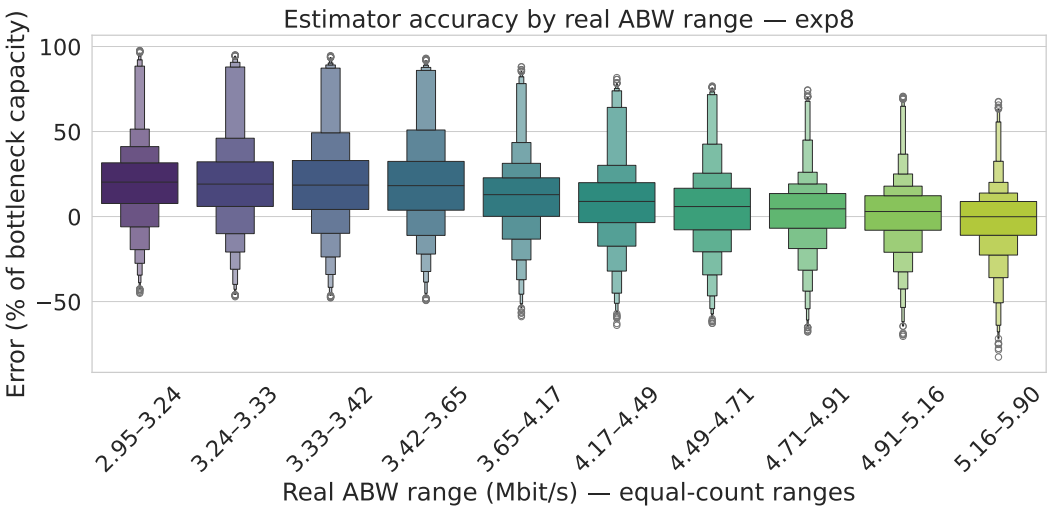

Figure C.1: Plot showing the estimation accuracy, grouped by ground truth ABW for Experiment 8. It indicates that the tool can quite accurately estimate bandwidth when cross-traffic load is low, but is less accurate during higher loads.

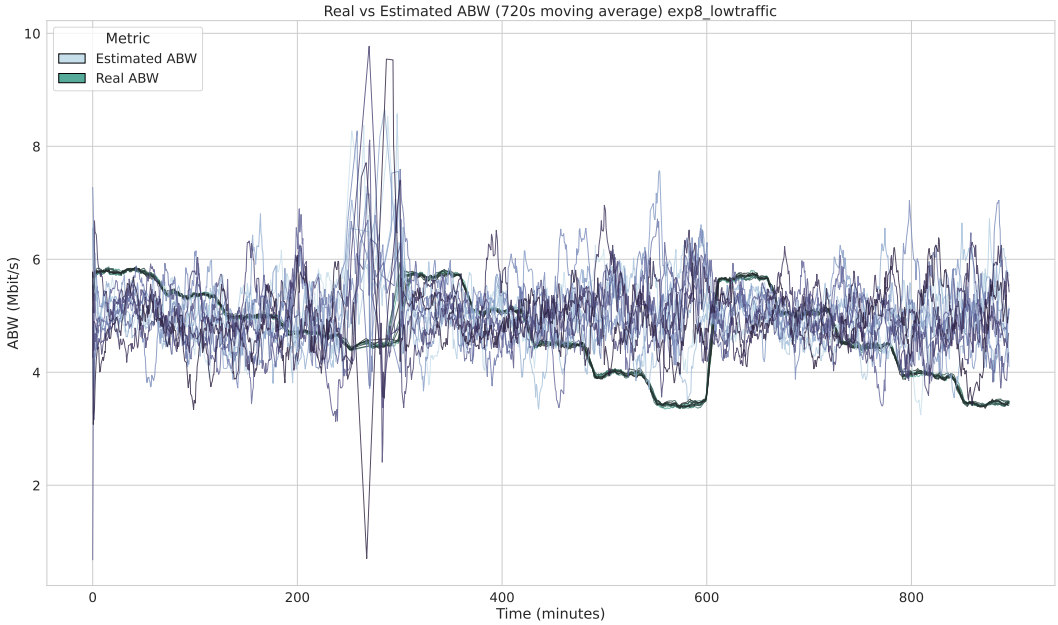

Figure C.2: We re-ran experiment 8 with smaller TCP bursts; the outcome was that the median error lowered to 0.5%, but as this plot indicates, the estimated ABW does not follow the trend of the ground truth ABW. The clearest trend is that when ABW lowers, the estimates become less precise.

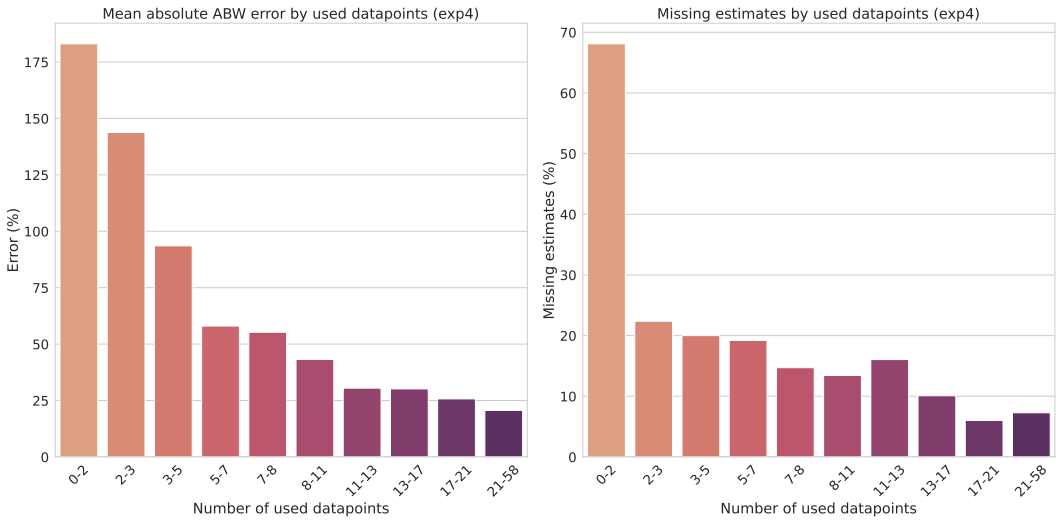

Figure C.3: Absolute error, against the number of used datapoints using simple linear regression for Experiment 4.

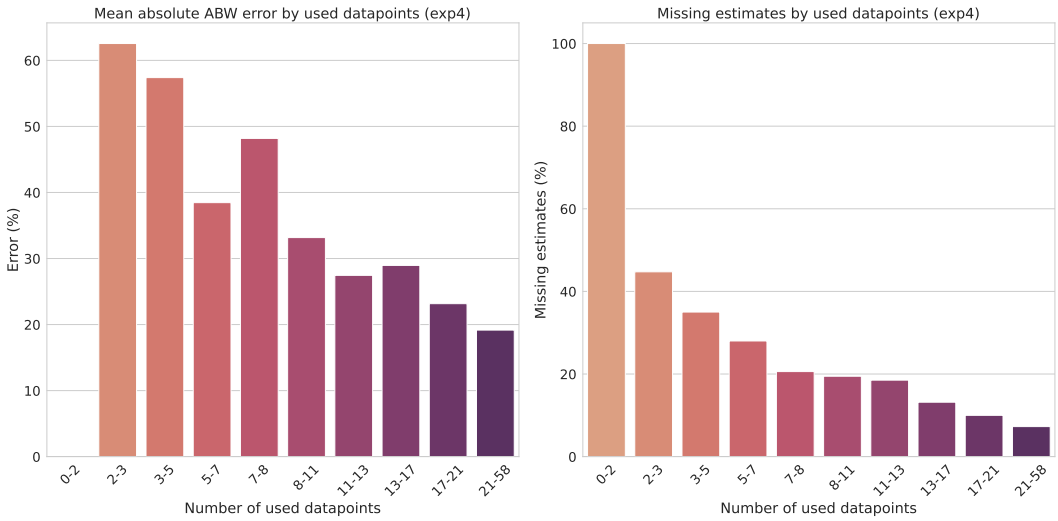

Figure C.4: Same plot as above, but with robust regression. The error still lowers with the number of used datapoints, but most errors are reduced significantly. Interestingly, the number of estimates lower than 0 or higher than the upstream link capacity increases for most of the datapoint buckets.

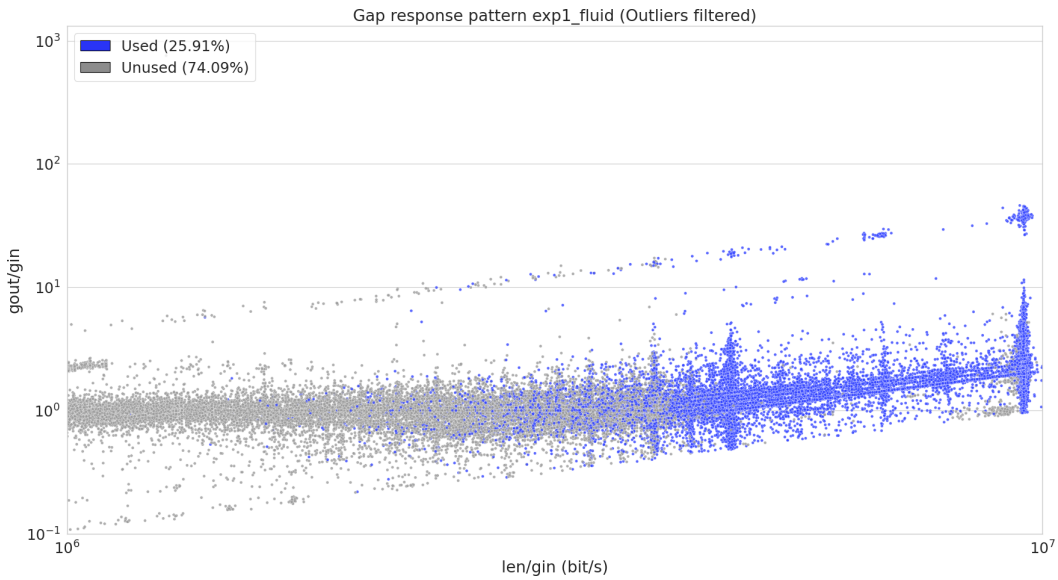

Figure C.5: Scatterplot of packet gap measurements from experiment 1 showing several diagonal lines, closer to each other, more closely representing the gap response curve while accounting for cross traffic.

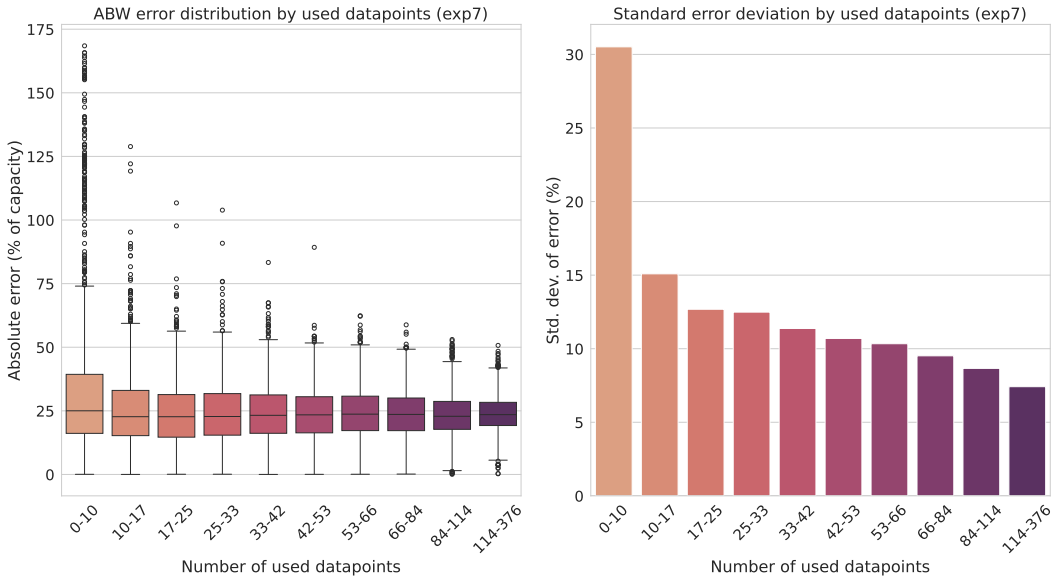

Figure C.6: Experiment 7 error distribution by number of used datapoints. Results are consistently at 25%.

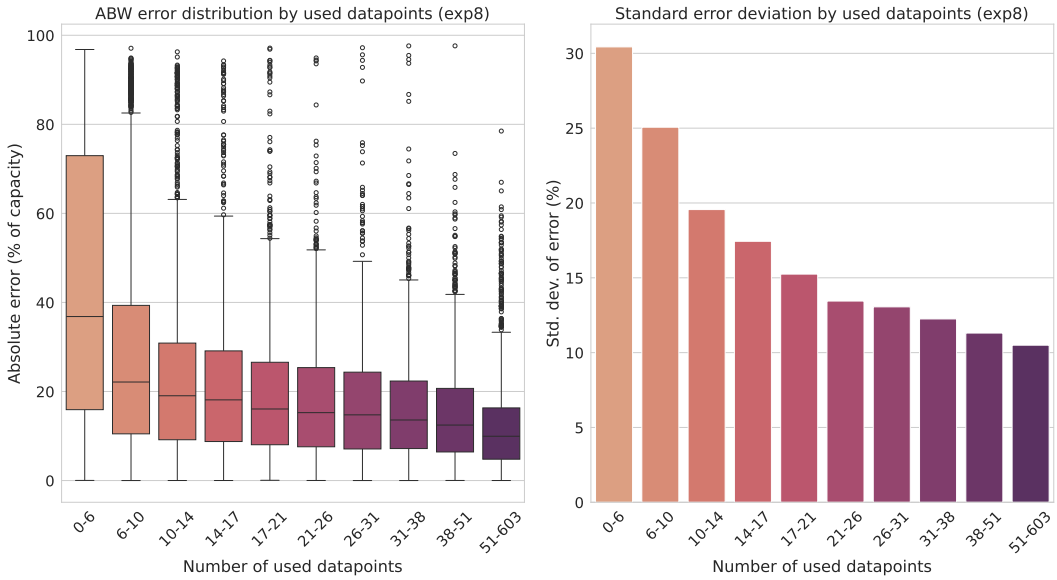

Figure C.7: Likely due to the introduced latency jitter, the error lowers when more data is available. The values are generally less regular compared to Experiment 7.

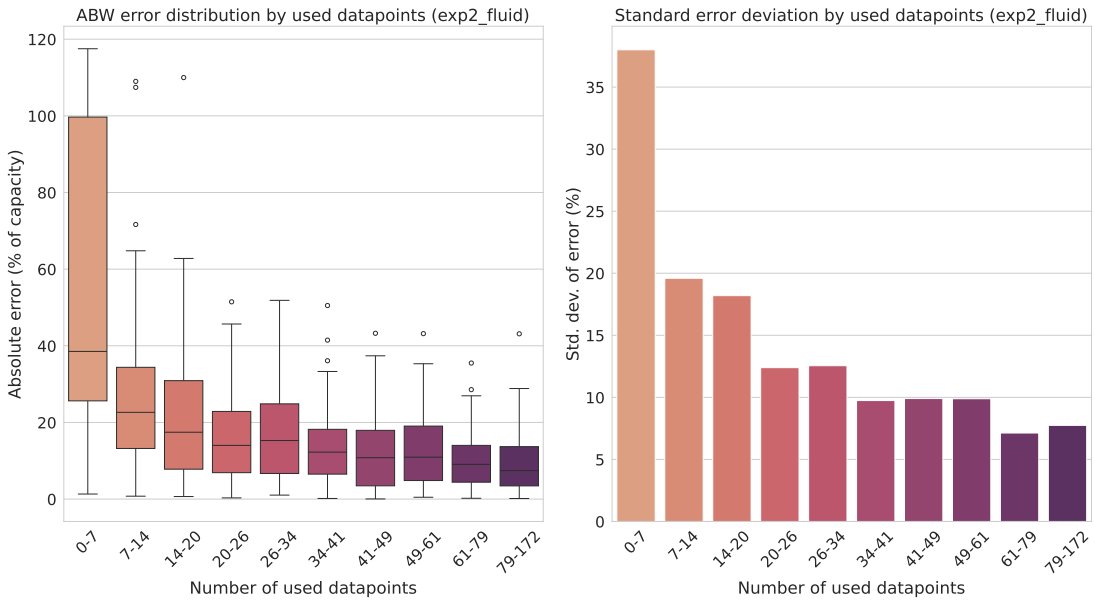

Figure C.8: Distribution of absolute error by the number of used datapoints, plotted next to the standard deviation of the absolute error by the number of used datapoints.